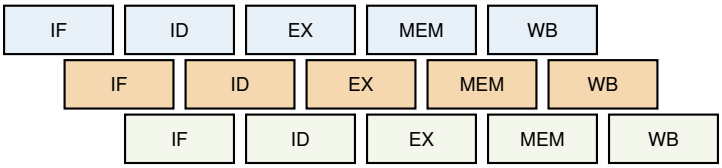
MIPS Instructions
The Pipelined Processor
Floating-Point Arithmetic

A first-time learner's textbook

with topic-wise solved past term-final questions



Course: CSE 209 Computer Architecture

Syllabus boundary: *Chapter 2 — Instructions: Language of the Computer* (69 slides), *Chapter 4 Part 2 — The Processor: Pipelining and ILP* (111 slides) and *Chapter 3 — Arithmetic: Floating Point* (slides 9–22)

Prepared for Akif · October 2026

How to Use This Book

What this book is

This is a textbook for a reader who has *not* yet studied the topics. It explains three groups of lecture material in full, in the order in which a newcomer can follow them, and then solves the past term-final questions on exactly these topics, grouped by topic rather than by year.

Part	Content	Lecture source
I. MIPS	instructions, registers, memory, number encoding, logic and branches, procedures and the stack, addressing, synchronization, translation/linking/loading; solved past questions (Chapters 1–7)	*Chapter 2*, 69 slides
II. The Processor, Part 2	pipelining, hazards, forwarding, hazard detection, control hazards, branch prediction, instruction-level parallelism, real processors; solved past questions (Chapters 8–15)	*Chapter 4 Part 2*, 111 slides
III. Floating point	IEEE 754 representation, addition, solved past questions (Chapters 16–18)	*Chapter 3*, slides 9–22

Topics that are not in these parts (for instance Chapter 1 performance, the single-cycle datapath design, integer multiplication and division, I/O) are deliberately left out; Appendix A lists them.

How the explanation chapters are built

Each chapter starts with a short *goal* box, defines every term the first time it is used, gives intuition before formulas, and works at least one full example. Boxes:

- **Definition** — a term in plain words, with an example.
- **Intuition** — why the idea is needed or how to picture it.
- **Worked Example** — every step written out.
- **Common mistake** — an error students often make.
- **Correction / clarification to the lecture** — places where a slide has a misprint or an inexact statement; the corrected version is explained, not silently adopted.
- **Past Exam Question** followed by **Solution**.

Slide references. Small grey tags such as [MIPS slide 12], [Processor slide 66] and [FP slide 15] point to the lecture slide the material comes from: `MIPS slide`, `Processor slide` and `FP slide` followed by the slide's own number. A figure captioned "slide *n*" is an image of that slide, included so you can compare the explanation with the original.

What was checked, and how

- **Assembly code.** Every complete assembly program in the book (solved questions) was executed on a small MIPS simulator I wrote in Python and compared with the result of the

equivalent C logic. Machine-code fields, branch offsets and jump targets were recomputed by script.

- **Pipeline cycle counts and timing diagrams.** A small timing model of the five-stage pipeline (forwarding, load-use stalls, branch penalty) was written; it reproduces the lecture's own examples (for instance the 13-cycle and 11-cycle schedules), and all pipeline diagrams in the solutions are generated from it, so the diagram and the stated cycle count agree.
- **Floating point.** Bit patterns, ranges and custom formats were recomputed in Python with exact fractions and with IEEE single precision (`struct`, NumPy). The branch-predictor accuracy figures were checked by simulation.
- **External sources.** The IEEE 754 bias, special values and the guard/sticky-bit statements were checked against Goldberg's paper (Appendix, *Sources*). I did not use any official solution set or marking scheme, so the answers are my own derivations and may differ from your instructor's expected wording.
- **Visual inspection.** The compiled PDF was checked by rendering it: the cover, contents, representative figure-heavy and equation-heavy pages and the last pages were looked at, and every page was scanned automatically for text or figures running into the margins (the problems found were fixed). I did not read all 130 pages one by one on screen.

Honest limits

- Several past papers were scanned images; where text or a figure was unreadable, the solution states the assumption it made (for instance a garbled number, or a figure that was not visible). These are collected in Appendix A.
- Questions on MIPS floating-point instructions use instructions that are not in the supplied slides. They are answered in a clearly marked box, from standard MIPS documentation, and were checked by hand rather than run.
- Slide statements that were wrong or inexact are corrected in a marked box. Nothing from the slides was changed silently.

Contents

How to Use This Book ii

I MIPS: The Language of the Computer 1

1 Talking to the Hardware: Instructions, Registers and Memory 2
- 1.1 What is an instruction set? . . . 2
  - 1.1.1 Why MIPS? . . . 2
- 1.2 Arithmetic operations and Design Principle 1 . . . 2
- 1.3 Register operands and Design Principle 2 . . . 3
- 1.4 Memory operands . . . 4
  - 1.4.1 How a memory address is formed . . . 4
  - 1.4.2 Alignment and endianness . . . 4
  - 1.4.3 Registers versus memory . . . 5
- 1.5 Immediate operands and Design Principle 3 . . . 5
- 1.6 The constant zero . . . 5
- 1.7 Summary of the chapter . . . 5

2 Numbers and Machine Code: How Instructions Are Encoded 7
- 2.1 Unsigned binary integers . . . 7
- 2.2 Two's-complement signed integers . . . 7
  - 2.2.1 Negation: complement and add 1 . . . 8
  - 2.2.2 Sign extension . . . 8
- 2.3 Representing instructions . . . 8
  - 2.3.1 Register numbers . . . 8
- 2.4 R-format (register format) . . . 8
- 2.5 Hexadecimal . . . 9
- 2.6 I-format (immediate format) and Design Principle 4 . . . 9
- 2.7 Summary . . . 10

3 Logical Operations and Making Decisions 11
- 3.1 Logical (bitwise) operations . . . 11
  - 3.1.1 Shifts . . . 11
  - 3.1.2 AND, OR, NOT . . . 11
- 3.2 Conditional operations: branches . . . 12
  - 3.2.1 Basic blocks . . . 12
- 3.3 Comparison: `slt` . . . 13

3.3.1 Signed versus unsigned comparison . . . 13
3.4 Worked examples: translating conditions . . . 13
3.5 Summary . . . 13

4 Procedures, the Stack and the Memory Layout 15
4.1 Terms . . . 15
4.2 Register usage conventions . . . 15
4.3 Calling and returning: `jal` and `jr` . . . 16
4.4 The stack . . . 16
4.5 Non-leaf procedures and recursion . . . 16
4.6 Local data, frames and the memory layout . . . 17
4.6.1 The MIPS memory layout [MIPS slide 45] . . . 17
4.7 Jump address tables (`switch/case`) . . . 18
4.8 Summary . . . 19

5 Characters, Big Constants and Addressing Modes 20
5.1 Character data . . . 20
5.2 32-bit constants . . . 21
5.3 Branch addressing . . . 21
5.4 Jump addressing . . . 22
5.4.1 Branching far away . . . 22
5.5 Summary of MIPS addressing modes [MIPS slide 55] . . . 23
5.6 Summary . . . 23

6 Synchronization, Translating and Starting a Program 24
6.1 Synchronization . . . 24
6.1.1 MIPS: `ll` and `sc` [MIPS slide 57] . . . 24
6.2 Translation and startup [MIPS slide 58] . . . 25
6.2.1 Assembler pseudoinstructions [MIPS slide 59] . . . 25
6.2.2 Producing an object module [MIPS slide 60] . . . 26
6.2.3 Linking object modules [MIPS slide 61] . . . 26
6.2.4 Loading a program [MIPS slide 62] . . . 26
6.2.5 Dynamic linking [MIPS slide 63–64] . . . 27
6.2.6 Starting Java applications [MIPS slide 65] . . . 27
6.3 Fallacies and pitfalls [MIPS slide 66–68] . . . 28
6.4 Concluding remarks [MIPS slide 69] . . . 28

7 MIPS: Solved Past Examination Questions (Topic-wise) 29
7.1 T1: Design principles and ISA-design reasoning . . . 29
7.2 T2: Instruction formats, machine code and instruction design . . . 32
7.3 T3: Addressing modes . . . 36

7.4 T4: Branch and jump reach; big constants; pseudoinstructions . . . 37
7.5 T5: Translating C into MIPS . . . 40
7.6 T6: Procedures, recursion and the stack . . . 43
7.7 T7: Synchronization . . . 47
7.8 T8: Translation, linking, loading, dynamic linking and Java . . . 47

II The Processor, Part 2: Pipelining 49

8 Pipelining: The Big Idea 50
8.1 What you need from the single-cycle processor . . . 50
8.1.1 How slow is it? The lecture's timing assumptions [Processor slide 5] . . . 50
8.2 The pipelining analogy [Processor slide 2] . . . 51
8.3 The five-stage MIPS pipeline [Processor slide 3–4] . . . 51
8.3.1 Pipeline performance [Processor slide 5–7] . . . 52
8.4 Why MIPS suits pipelining [Processor slide 9] . . . 52
8.5 Hazards: what can go wrong [Processor slide 10–11] . . . 53
8.6 Summary: the big picture [Processor slide 32] . . . 53

9 Structural and Data Hazards, Forwarding and Code Scheduling 54
9.1 Reading a pipeline diagram . . . 54
9.2 Structural hazards [Processor slide 11] . . . 54
9.3 Data hazards [Processor slide 12–16] . . . 55
9.3.1 Solution 1: stall . . . 55
9.3.2 Solution 2: forwarding (bypassing) [Processor slide 16] . . . 55
9.3.3 Load-use data hazard [Processor slide 17–21] . . . 56
9.4 Code scheduling to avoid stalls [Processor slide 22–24] . . . 56
9.5 Summary . . . 57

10 The Pipelined Datapath and Its Control 59
10.1 From one long cycle to five short ones . . . 59
10.2 Pipeline registers . . . 59
10.3 How an instruction moves through the pipeline . . . 61
10.4 Two ways of drawing pipeline behaviour [Processor slide 35, 46–49] . . . 62
10.5 Pipelined control [Processor slide 50–52] . . . 62
10.6 Summary . . . 63

11 Data Hazards: Forwarding Unit and Hazard Detection Unit 65
11.1 Which dependences are hazards? . . . 65
11.2 Detecting the need to forward [Processor slide 56–57] . . . 65
11.3 The forwarding unit and the forwarding paths [Processor slide 58–61] . . . 66
11.3.1 The conditions, written out [Processor slide 61] . . . 66

11.4 The double data hazard [Processor slide 62–64] . . . 68
11.5 Load-use hazards: detection and stall [Processor slide 66–72] . . . 69
11.5.1 When is the data ready? . . . 69
11.5.2 The detection condition [Processor slide 68–69] . . . 69
11.5.3 How to stall [Processor slide 70–74] . . . 69
11.5.4 The hazard detection unit in the datapath [Processor slide 75] . . . 69
11.6 Summary . . . 69

12 Control Hazards and Branch Prediction **72**
12.1 Why branches are a problem [Processor slide 77] . . . 72
12.2 Branch resolved in MEM: three lost cycles [Processor slide 78–79] . . . 72
12.2.1 A subtle interaction . . . 73
12.3 Reducing the delay: decide the branch in ID [Processor slide 80–83] . . . 73
12.3.1 New data hazards for branches [Processor slide 84–86] . . . 74
12.4 Branch prediction [Processor slide 87–93] . . . 74
12.4.1 Static prediction . . . 74
12.4.2 Dynamic prediction [Processor slide 88] . . . 75
12.4.3 Branch target buffer [Processor slide 92] . . . 76
12.4.4 Even a perfect predictor needs a cycle? . . . 76
12.5 Delayed branch [Processor slide 94] . . . 76
12.6 Summary . . . 77

13 Instruction-Level Parallelism and Real Processors **78**
13.1 Instruction-level parallelism [Processor slide 86] . . . 78
13.2 Two ways to issue several instructions [Processor slide 87] . . . 78
13.3 Speculation [Processor slide 88–89] . . . 79
13.4 Static multiple issue [Processor slide 90–94] . . . 79
13.4.1 MIPS with static dual issue [Processor slide 92–93] . . . 79
13.4.2 New hazards in the dual-issue machine [Processor slide 94] . . . 79
13.4.3 Scheduling example [Processor slide 95] . . . 79
13.4.4 Loop unrolling and register renaming [Processor slide 96–97] . . . 80
13.5 Dynamic multiple issue: superscalar processors [Processor slide 98–101] . . . 81
13.5.1 Dynamic pipeline scheduling [Processor slide 99] . . . 81
13.5.2 Why not just let the compiler do it? [Processor slide 101] . . . 82
13.6 Does multiple issue work? [Processor slide 102] . . . 82
13.7 Power efficiency [Processor slide 103] . . . 83
13.8 Real processors: ARM Cortex-A8 and Intel Core i7 [Processor slide 104–108] . . . 83
13.9 Fallacies, pitfalls and conclusion [Processor slide 109–111] . . . 83
13.10Summary . . . 84

14 Pipelining: Solved Past Examination Questions (Part 1) **85**

14.1 P1: Pipeline speed-up and why single-cycle is inefficient . . . 85
14.2 P2: Cycle counts and code scheduling . . . 86
14.3 P3: Data hazards and the forwarding unit . . . 89

15 Pipelining: Solved Past Examination Questions (Part 2) 91
15.1 P4: Hazard detection unit, hazard types . . . 91
15.2 P5: Control hazards . . . 93
15.3 P6: Branch prediction . . . 95
15.4 P7: Pipelined datapath and control . . . 96
15.5 P8: Multiple issue, scheduling and superscalar . . . 97
15.6 P9: CISC versus RISC . . . 99

III Floating-Point Arithmetic 100

16 Floating-Point Numbers: Representation 101
16.1 Prerequisites . . . 101
16.1.1 Binary fractions . . . 101
16.1.2 Scientific notation and normalization . . . 101
16.2 The IEEE 754 standard [FP slide 10–11] . . . 102
16.2.1 Why a bias? . . . 102
16.3 Encoding and decoding . . . 102
16.4 Range and precision [FP slide 12–14] . . . 103
16.4.1 The reserved exponents . . . 103
16.4.2 Single precision . . . 103
16.4.3 Double precision . . . 103
16.4.4 Precision . . . 104
16.5 Denormals, zero, infinity and NaN [FP slide 17–18] . . . 104
16.6 Comparing floating-point numbers as integers . . . 105
16.7 General formulae for any format . . . 105
16.8 Practice questions . . . 106

17 Floating-Point Addition 107
17.1 The idea . . . 107
17.2 A decimal example [FP slide 19] . . . 107
17.3 A binary example [FP slide 20] . . . 108
17.4 Sign handling . . . 108
17.5 The flowchart . . . 108
17.6 Overflow and underflow [FP slide 19] . . . 108
17.7 Floating-point adder hardware [FP slide 21–22] . . . 110
17.8 Extra bits for accurate rounding . . . 110
17.9 Floating-point addition is not associative . . . 111

17.10 Practice question 111

18 Floating Point: Solved Past Examination Questions **113**

18.1 F1: Bias and special encodings 113
18.2 F2: Range of a format 113
18.3 F3: Encoding and decoding 115
18.4 F4: Integers in floating point; comparing floats 116
18.5 F5: Floating-point addition 116
18.6 F6: MIPS code with floating-point instructions (outside the lecture slides) 118

A Topics Left Out, Assumptions and Sources **120**

Sources **122**

PART I

MIPS: The Language of the Computer

CHAPTER 1

Talking to the Hardware: Instructions, Registers and Memory

Chapter goal

A processor is a machine that can only do a few very simple things, each triggered by a binary code called an *instruction*. This chapter introduces the vocabulary of instructions, the MIPS instruction set used throughout the course, and the two places a MIPS instruction can find its data: *registers* and *memory*. You will be able to translate simple arithmetic and array statements from C into MIPS. *Lecture source: MIPS slides 1–13.*

1.1 What is an instruction set?

A computer's hardware is built from circuits that understand only binary numbers. One pattern of 32 bits might mean "add two numbers", another might mean "fetch a number from memory". Each such command is an **instruction**. The complete collection of commands a particular processor understands is its **instruction set**, and the formal description of that collection (which instructions exist, how they are written in binary, what they do) is the **instruction set architecture**, abbreviated **ISA**. [MIPS slide 2]

Different processor families have different ISAs: the processor in a phone (ARM), in a laptop (x86) and in a router (MIPS) do not understand each other's binary programs. But the lecture notes that they have "many aspects in common". Early computers had very simple instruction sets because they were easier to build, and — an important modern lesson — many modern computers again prefer simple instruction sets.

Humans do not write the binary patterns directly. We write **assembly language**, a text form in which each instruction has a short name such as `add` and operands such as `$s1`. A program called the **assembler** turns it into the binary **machine code** (Chapter 6).

1.1.1 Why MIPS?

MIPS is a real, commercially successful ISA (originating at Stanford, commercialised by MIPS Technologies) used widely in embedded devices: consumer electronics, network and storage equipment, cameras, printers [MIPS slide 3]. It belongs to the family called **RISC** (*reduced instruction set computer*) — an ISA with a small number of simple, regular instructions. It is "typical of many modern ISAs", so what you learn on MIPS transfers to ARM, RISC-V and others. (The contrasting style, such as x86, offers many complicated instructions; see Section 6.3.)

1.2 Arithmetic operations and Design Principle 1

Consider adding two numbers. MIPS writes the instruction as

```
add a, b, c # a gets b + c
```

The text after # is a *comment* ignored by the machine. The instruction names *three* operands: two **source** operands (`b` and `c`), which are read, and one **destination** (`a`), which receives the result. Subtraction `sub a, b, c` has the same shape (`a = b - c`). [MIPS slide 4]

Definition: Design Principle 1: Simplicity favours regularity [MIPS slide 4]

Making every arithmetic instruction have exactly the same form (one operation, two sources, one destination) makes the hardware simpler (the circuits that decode and execute instructions can be uniform), and simplicity enables higher performance at lower cost.

One might object: why not allow `add a, b, c, d` to add three numbers at once? That would be a different, irregular format that needs more hardware to decode. Instead, to add four numbers we use three instructions. This is the point of the principle, and it is the standard answer to exam questions such as "why does MIPS require three operands?".

Worked Example: Compiling a C expression [MIPS slide 5]

C code: `f = (g + h) - (i + j);` Because each instruction handles one operation, we need temporary storage for the two partial sums:

```
add t0, g, h # temp t0 = g + h
add t1, i, j # temp t1 = i + j
sub f, t0, t1 # f = t0 - t1
```

Three operations in C ⇒ three MIPS instructions.

1.3 Register operands and Design Principle 2

In the example above `g, h, i, j, t0` were just names. In real MIPS, arithmetic instructions may only use **registers** as operands.

Definition: Register

A register is a very small, very fast storage location *inside* the processor. MIPS has a **register file** of 32 registers, each 32 bits wide, numbered 0 to 31 [MIPS slide 6].

A 32-bit unit of data is called a **word**. MIPS registers hold exactly one word. Registers are referred to by names starting with a dollar sign:

- `$t0`–`$t9`: *temporary* values, for short-lived intermediate results;
- `$s0`–`$s7`: *saved* registers, for variables whose values must survive across procedure calls (explained in Chapter 4).

(The full table is in Chapter 4.)

Definition: Design Principle 2: Smaller is faster [MIPS slide 6]

A small number of registers can be accessed more quickly than the millions of locations in main memory: fewer places means shorter wires and simpler selection logic. This is why MIPS has only 32 registers, not thousands. (This is the answer to the exam question "why has the number of registers been restricted to 32?"; the other half of the answer is that 32 registers need only 5 bits to name, which keeps instructions short — Chapter 2.)

Worked Example: Register operands [MIPS slide 7]

C: `f = (g + h) - (i + j);` with `f,...,j` in `$s0,...,$s4`.

```
add $t0, $s1, $s2 # t0 = g + h
add $t1, $s3, $s4 # t1 = i + j
sub $s0, $t0, $t1 # f = t0 - t1
```

1.4 Memory operands

Only 32 words fit in the registers, but programs use arrays, structures and dynamic data far larger than that. Those live in **main memory** (how it is built is the subject of the separate memory-hierarchy book). To compute on such data, MIPS has the pattern: *load* a value from memory into a register, operate on registers, *store* the result back to memory [MIPS slide 8]. **Data transfer instructions** do this:

- `lw rt, offset(rs)` — *load word*: read a word from memory into register `rt`;
- `sw rt, offset(rs)` — *store word*: write register `rt` into memory.

1.4.1 How a memory address is formed

Memory is a long sequence of numbered boxes. In MIPS each box holds one **byte** (8 bits), and the number of the box is its **address**; this is called **byte addressing**. A word is 4 bytes, so the words of an array occupy addresses $0, 4, 8, 12, \ldots$ — *not* $0, 1, 2, 3$. (A classic pitfall: "sequential words are not at sequential addresses; increment by 4, not 1" [MIPS slide 68].)

In `lw $t0, 32($s3)`, the number 32 is the **offset** (a constant in the instruction) and `$s3` is the **base register** (a register holding the starting address of, say, an array). The hardware adds them: the address is 32 + contents of $s3. This is the **base (displacement) addressing** mode (Chapter 5).

1.4.2 Alignment and endianness

Two more facts from [MIPS slide 8]:

- Words are **aligned**: the address of a word must be a multiple of 4. A word at address 8 occupies bytes 8,9,10,11. A word "at address 10" is illegal because it would straddle two memory words; this restriction lets memory deliver a word in one access.
- MIPS is **big endian**: the most-significant byte of a word is stored at the lowest of its four addresses. (*Little endian*, used by x86, does the opposite.) For example the 32-bit value `0x0A0B0C0D` stored at address 100 puts `0x0A` at 100, `0x0B` at 101, `0x0C` at 102 and `0x0D` at 103 in a big-endian machine.

Worked Example: Array read: `g = h + A[8];` [MIPS slide 9]

Given: `g` in `$s1`, `h` in `$s2`, base address of `A` in `$s3`. Element `A[8]` is the 9th word; its byte offset is $8 \times 4 = 32$.

```
lw $t0, 32($s3) # t0 = A[8]
add $s1, $s2, $t0 # g = h + A[8]
```

Worked Example: Array read-modify-write: `A[12] = h + A[8];` [MIPS slide 10]

```
lw $t0, 32($s3) # t0 = A[8]
add $t0, $s2, $t0 # t0 = h + A[8]
sw $t0, 48($s3) # A[12] = t0 (12*4 = 48)
```

Common mistake: Index versus offset

The offset is in *bytes*. The C index i must be multiplied by 4 (for `int` arrays) before use. When the index is in a register you cannot write a constant offset; you must compute the address with `sll` and `add` (see the loop example of Chapter 3).

1.4.3 Registers versus memory

Registers are faster than memory, and operating on memory data needs extra load/store instructions. Therefore the compiler keeps frequently-used variables in registers and only **spills** (moves) the less frequently used ones to memory [MIPS slide 11]. If a program has more live variables than registers (for example 40 variables and 32 registers) some variables must be spilled. *Register optimisation is important*.

1.5 Immediate operands and Design Principle 3

Programs often add small constants (`i = i + 1`). Loading the constant from memory first would be wasteful. An **immediate** operand is a constant written directly inside the instruction:

```
addi $s3, $s3, 4 # s3 = s3 + 4
addi $s2, $s1, -1 # s2 = s1 - 1 (no 'subi': use a negative constant)
```

There is no *subtract immediate*: adding a negative constant does the same [MIPS slide 12].

Definition: Design Principle 3: Make the common case fast [MIPS slide 12]

Small constants are very common in real programs, so the architecture provides a special form for them, which avoids a load instruction. Optimise what happens most often.

1.6 The constant zero

Register 0, written `$zero`, always contains the value 0 and cannot be overwritten [MIPS slide 13]. This is useful because it gives a free zero for many operations. For example, to *copy* register `$s1` into `$t2` there is no dedicated `move` instruction: write `add $t2, $s1, $zero`.

1.7 Summary of the chapter

- MIPS arithmetic instructions: `op dest, src1, src2`, always three register operands.
- 32 registers $\times$ 32 bits. Memory is byte-addressed; words are aligned (address multiple of 4); MIPS is big endian.
- `lw`/`sw` are the only instructions that move data between memory and registers.
- Immediates avoid loads for small constants; `$zero` is hard-wired to 0.

- Principles so far: (1) simplicity favours regularity, (2) smaller is faster, (3) make the common case fast.

Lecture source

MIPS slides 1–13 (§2.1–§2.3).

CHAPTER 2

Numbers and Machine Code: How Instructions Are Encoded

Chapter goal

Instructions are stored as 32-bit numbers. To understand how, we first need the number representations used by the machine (unsigned and two's-complement signed integers), then the two most important MIPS instruction layouts: *R-format* and *I-format*. *Lecture source: MIPS slides 14–23.*

2.1 Unsigned binary integers

Bit = binary digit (0 or 1). In an n-bit **unsigned** number, bit i (counting from 0 at the right) is worth 2^i:

$$x = \sum_{i=0}^{n-1} b_i \, 2^i, \qquad 0 \le x \le 2^n - 1.$$

Example from the slide [MIPS slide 14]: $0000\ldots1011_2 = 1\cdot2^3 + 0\cdot2^2 + 1\cdot2^1 + 1\cdot2^0 = 8+2+1 = 11_{10}$. With $n = 32$: 0 to 4,294,967,295.

2.2 Two's-complement signed integers

We also need negative numbers. MIPS uses **two's complement**: the leftmost bit has *negative* weight.

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i \, 2^i, \qquad -2^{n-1} \le x \le 2^{n-1} - 1.$$

Worked Example: [MIPS slide 15]

$1111\ 1111\ \ldots\ 1111\ 1100_2$ (32 bits) $= -2^{31} + 2^{30} + \cdots + 2^2 = -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4$.

Facts to remember [MIPS slide 16]:

- Bit 31 is the **sign bit**: 1 means negative, 0 means non-negative. Non-negative numbers have identical unsigned and signed representations.
- $0 = 00\ldots0$, $-1 = 11\ldots1$, most negative $= 10\ldots0$ (-2^{31}), most positive $= 01\ldots1$ $(2^{31}-1)$.
- The most-negative number has no positive counterpart: $-(-2^{31})$ cannot be represented in 32 bits.

2.2.1 Negation: complement and add 1

To negate a two's-complement number, flip every bit (*complement*: $0 \to 1$, $1 \to 0$) and add 1 [MIPS slide 17]. Why? A number plus its complement is all ones, which equals -1; so $x + \bar{x} = -1$, hence $-x = \bar{x} + 1$.

Worked Example: Negate $+2$

$+2 = 0000\ldots0010$; complement $= 1111\ldots1101$; add $1 \Rightarrow 1111\ldots1110 = -2$.

2.2.2 Sign extension

To represent a number with *more* bits without changing its value, copy the sign bit into the new left-hand bits (**sign extension**) [MIPS slide 18]. For unsigned values pad with zeros instead.

Worked Example: 8-bit to 16-bit

$+2$: 0000 0010 $\to$ 0000 0000 0000 0010. -2: 1111 1110 $\to$ 1111 1111 1111 1110 (still -2).

MIPS uses sign extension in `addi` (immediate), `lb`/`lh` (loaded byte/halfword) and `beq`/`bne` (the branch displacement).

2.3 Representing instructions

A binary instruction is called **machine code**. In MIPS every instruction is exactly one 32-bit word, split into **fields** (groups of bits with a meaning). The first six bits are always the **opcode** (operation code) saying what kind of instruction it is. Having few, regular formats makes decoding fast — Design Principle 1 again [MIPS slide 19].

2.3.1 Register numbers

A register name is just a nickname for a number $0\ldots31$, which needs 5 bits. Important ones [MIPS slide 19]: `$t0`–`$t7` are 8–15; `$t8`,`$t9` are 24, 25; `$s0`–`$s7` are 16–23. The full list appears in Table 2.1.

2.4 R-format (register format)

Used by instructions that compute on registers [MIPS slide 20].

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- `op`: opcode; for all R-format instructions it is 0.
- `rs`: first source register number; `rt`: second source register number; `rd`: destination register number.
- `shamt`: *shift amount* (used by shifts; 0 otherwise).
- `funct`: *function code* that extends the opcode, selecting `add`, `sub`, `and`, … among the instructions with `op`=0.

Total: $6+5+5+5+5+6 = 32$ bits.

Worked Example: Encode `add $t0, $s1, $s2` [MIPS slide 21]

`$s1` = 17, `$s2` = 18, `$t0` = 8; for `add`, `funct`=32.

op	rs	rt	rd	shamt	funct
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

Concatenating: $0000\,0010\,0011\,0010\,0100\,0000\,0010\,0000_2$ = `0x02324020`.

Note the field *order*: the destination is `rd`, which comes *after* the two sources in the binary word although it comes *first* in the assembly text.

2.5 Hexadecimal

Writing 32 bits is tedious, so we group bits in fours, each written as one **hexadecimal** (base 16) digit 0–9, *a*–*f* [MIPS slide 22].

0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Example: `0xeca86420` = 1110 1100 1010 1000 0110 0100 0010 0000.

2.6 I-format (immediate format) and Design Principle 4

Instructions with a constant or a memory offset need a field for it. R-format has no room for a 16-bit constant, so MIPS uses a second layout [MIPS slide 23]:

`op`	`rs`	`rt`	`constant or address`
6 bits	5 bits	5 bits	16 bits

- Immediate arithmetic (`addi`): `rs` = source, `rt` = destination, constant in $-2^{15} \ldots 2^{15}-1$.
- Load/store (`lw`,`sw`): `rs` = base register, `rt` = register loaded or stored, 16-bit field = byte offset.

Common mistake: `rt` plays two roles

In an R-format instruction `rt` is a *source*; in `addi` and `lw` it is the *destination*; in `sw` it is the register being *stored* (a source). Keep this straight when encoding.

Definition: Design Principle 4: Good design demands good compromises [MIPS slide 23]

Different formats complicate decoding, but they allow all instructions to be the same length (32 bits). Keep the formats as similar as possible: the `op`, `rs` and `rt` fields are in the same bit positions in R and I formats.

Worked Example: Encoding a load and a store

Using the array example of Chapter 1:
`lw $t0, 32($s3)`: op=35, rs=19 (`$s3`), rt=8 (`$t0`), const=32 ⇒ `0x8e680020`.
`sw $t0, 48($s3)`: op=43, rs=19, rt=8, const=48 ⇒ `0xae680030`.

`addi $s3,$s3,1`: op=8, rs=19, rt=19, const=1 ⇒ `0x22730001`.

Table 2.1: MIPS register conventions (slides 6, 19, 37).

Number	Name	Purpose	Preserved by callee?
0	`$zero`	constant 0	n/a
1	`$at`	assembler temporary (pseudoinstructions)	no
2–3	`$v0,$v1`	result values	no
4–7	`$a0-$a3`	arguments	no
8–15	`$t0-$t7`	temporaries	no
16–23	`$s0-$s7`	saved	yes
24–25	`$t8,$t9`	temporaries	no
28	`$gp`	global pointer (static data)	yes
29	`$sp`	stack pointer	yes
30	`$fp`	frame pointer	yes
31	`$ra`	return address	yes

2.7 Summary

Unsigned: 0 to $2^n - 1$. Two's complement: -2^{n-1} to $2^{n-1} - 1$; negate = invert + 1; sign-extend by copying the sign bit. R-format: `op rs rt rd shamt funct` (6/5/5/5/5/6). I-format: `op rs rt imm16` (6/5/5/16). J-format (jumps) is in Chapter 5.

Lecture source

MIPS slides 14–23 (§2.4–§2.5).

CHAPTER 3

Logical Operations and Making Decisions

Chapter goal

Programs do more than add: they manipulate individual bits, choose between alternatives (`if`) and repeat work (`while`, `for`). This chapter shows the MIPS instructions for bitwise logic and shifts, and for conditional branching, and how a compiler turns `if` and loops into them. *Lecture source: MIPS slides 24–35.*

3.1 Logical (bitwise) operations

Some tasks need to look at or change individual *bits* of a word: extracting a field from packed data, setting a flag, inverting a pattern. **Bitwise** instructions apply an operation to each bit position independently [MIPS slide 24].

Operation	C	Java	MIPS
Shift left	«	«	`sll`
Shift right	»	»>	`srl`
Bitwise AND	&	&	`and, andi`
Bitwise OR	|	|	`or, ori`
Bitwise NOT	~	~	`nor`

3.1.1 Shifts

A **shift** moves all bits left or right by a number of positions; vacated places are filled with 0 (a *logical* shift). The count is the `shamt` field of the R-format [MIPS slide 25].

- `sll` (shift left logical) by i bits *multiplies by* 2^i. Example: $0000\,0101_2 = 5$; shifted left by 2 gives $0001\,0100_2 = 20 = 5 \times 4$.
- `srl` (shift right logical) by i bits *divides by* 2^i — for *unsigned* numbers only (it fills with zeros, so it would corrupt the sign of a negative number).

For example `sll $t1,$s3,2` has op=0, rs=0, rt=19, rd=9, shamt=2, funct=0 and encodes to `0x00134880`. It is how we multiply an array index by 4 to obtain a byte offset.

3.1.2 AND, OR, NOT

- **AND** gives 1 only where *both* bits are 1. It is used to *mask*: select some bits and clear the rest to 0 [MIPS slide 26].
- **OR** gives 1 where *either* bit is 1. It is used to *include* bits: set some bits to 1 and leave others unchanged [MIPS slide 27].
- **NOT** inverts every bit. MIPS has no NOT instruction; instead it has the 3-operand `nor` (a NOR $b = \neg(a$ OR $b)$). Since a OR $0 = a$, `nor $t0,$t1,$zero` computes ¬`$t1` [MIPS slide 28]. (The same regularity principle: no new instruction needed.)

Worked Example: Masking and setting [MIPS slide 26–28]

```
$t2                0000 0000 0000 0000 0000 1101 1100 0000
$t1                0000 0000 0000 0000 0011 1100 0000 0000
and $t0,$t1,$t2    0000 0000 0000 0000 0000 1100 0000 0000
or  $t0,$t1,$t2    0000 0000 0000 0000 0011 1101 1100 0000
nor $t0,$t1,$zero  1111 1111 1111 1111 1100 0011 1111 1111
```

3.2 Conditional operations: branches

A **branch** is an instruction that may change which instruction runs next. Normally the processor executes instructions in sequence; a **conditional branch** jumps to a **label** (a name for an instruction's position in the program) *if* a condition is true, otherwise execution continues with the next instruction [MIPS slide 29].

- `beq rs, rt, L1`: *branch if equal* — if `rs == rt` go to `L1`.
- `bne rs, rt, L1`: *branch if not equal*.
- `j L1`: *jump* — unconditional.

The assembler converts labels into numeric offsets or addresses (Chapter 5).

Worked Example: Compiling an `if–else` [MIPS slide 30]

`if (i==j) f = g+h; else f = g-h;` with `f,g,h,i,j` in `$s0,...,$s4`.

```
     bne $s3, $s4, Else # if i != j go to Else
     add $s0, $s1, $s2 # f = g + h
     j Exit # skip the else part
Else: sub $s0, $s1, $s2 # f = g - h
Exit: ...
```

Reasoning: we test the *opposite* condition (`bne`) so that the "then" code can follow directly. After the then-part we must jump over the else-part.

Worked Example: Compiling a loop [MIPS slide 31]

`while (save[i] == k) i += 1;` with `i` in `$s3`, `k` in `$s5`, base address of `save` in `$s6`.

```
Loop: sll $t1, $s3, 2 # t1 = 4*i (byte offset)
     add $t1, $t1, $s6 # t1 = &save[i]
     lw $t0, 0($t1) # t0 = save[i]
     bne $t0, $s5, Exit # leave if save[i] != k
     addi $s3, $s3, 1 # i = i + 1
     j Loop # repeat
Exit: ...
```

3.2.1 Basic blocks

A **basic block** is a sequence of instructions with no embedded branches (except possibly at the end) and no branch targets (except possibly at the beginning) [MIPS slide 32]. Once the first instruction of a block starts, all the rest are certain to execute. Compilers find basic blocks to optimise within them, and advanced processors accelerate their execution. In the loop above, `Loop:...bne` is one block, `addi; j Loop` is another.

3.3 Comparison: `slt`

Branches test only equality. For "less than" MIPS uses **set on less than** [MIPS slide 33]:

- `slt rd, rs, rt`: if `rs` < `rt` then `rd` = 1 else `rd` = `0`;
- `slti rt, rs, constant`: same with an immediate.

Combine with `bne`/`beq`:

```
slt $t0, $s1, $s2 # t0 = ($s1 < $s2)
bne $t0, $zero, L # branch to L if $s1 < $s2
```

Definition: Why no `blt`, `bge`? [MIPS slide 34]

The hardware that decides $<$ or $\geq$ is slower than that for $=$ or $\neq$ (the latter needs only to detect whether the bits are all identical). Folding a magnitude compare into the branch would take more work per instruction and force a slower clock for *all* instructions. Since `beq`/`bne` are the common case, MIPS makes them fast and builds the rest from `slt` + branch — "a good design compromise" (Principle 4). The assembler offers `blt` etc. as *pseudoinstructions* (Chapter 6).

3.3.1 Signed versus unsigned comparison

The same bit pattern is different numbers under the two interpretations, so MIPS has `slt`, `slti` (signed) and `sltu`, `sltiu` (unsigned) [MIPS slide 35]. Let `$s0`= `0xffffffff` and `$s1`= `0x00000001`. Signed: $-1 < +1$, so `slt` gives 1. Unsigned: 4,294,967,295 > 1, so `sltu` gives 0.

3.4 Worked examples: translating conditions

Worked Example: Comparing with all relational operators

Let `a` in `$s0`, `b` in `$s1`. Branch to `L` when:

Condition	Code	Idea
$a < b$	`slt $t0,$s0,$s1; bne $t0,$zero,L`	$t0$=1 iff $a < b$
$a \geq b$	`slt $t0,$s0,$s1; beq $t0,$zero,L`	not ($a < b$)
$a > b$	`slt $t0,$s1,$s0; bne $t0,$zero,L`	swap operands
$a \leq b$	`slt $t0,$s1,$s0; beq $t0,$zero,L`	not ($b < a$)

Worked Example: A conditional with a large constant

`if (x > y + 131075) x = 8;` (`x` in `$t1`, `y` in `$t2`). The constant 131075 exceeds the 16-bit immediate range ($-32768 \ldots 32767$), so `addi` cannot hold it. It must be built with `lui`+`ori` (Chapter 5). This is solved in Chapter 7.

3.5 Summary

`sll`/`srl` multiply/divide by powers of two; `and` masks; `or` sets; `nor` (with `$zero`) inverts. `beq`/`bne`/`j` change control flow; `slt`/`slti`/`sltu`/`sltiu` compare; `blt`-style branches are made

from `slt`+`bne`/`beq` because equality compare is cheaper in hardware. A basic block is branch-free straight-line code.

Lecture source

MIPS slides 24–35 (§2.6–§2.7).

CHAPTER 4

Procedures, the Stack and the Memory Layout

Chapter goal

A *procedure* (function) is a reusable piece of code called from many places. Hardware must help with three things: passing arguments, jumping to the code and *returning to the right place*, and giving the procedure working space without destroying the caller's data. This chapter explains the MIPS conventions, the stack, leaf and non-leaf procedures, recursion, and where programs live in memory. *Lecture source: MIPS slides 36–45.*

4.1 Terms

The **caller** is the code that calls a procedure; the **callee** is the procedure being called. A procedure that calls no other procedure is a **leaf procedure**; one that does is a **non-leaf procedure**. The six steps of a call [MIPS slide 36] are:

1. The caller places the **parameters** (arguments) where the callee can find them.
2. Control is transferred to the procedure.
3. The procedure acquires the storage it needs.
4. It performs its operation.
5. It places the result where the caller can find it.
6. It returns control to the point of the call.

4.2 Register usage conventions

To make step 1, step 5 and independent compilation work, programmers agree on which register is used for what [MIPS slide 37] (full table in Table 2.1):

- `$a0`-`$a3`: four *argument* registers (regs 4–7);
- `$v0, $v1`: two *result* (value) registers (regs 2, 3);
- `$t0`-`$t9`: *temporaries* — may be overwritten by the callee (so a caller that needs them after the call must save them);
- `$s0`-`$s7`: *saved* registers — the callee must restore them to their original values before returning (so if it wants to use one, it first saves it on the stack);
- `$gp` global pointer (static data), `$sp` stack pointer, `$fp` frame pointer, `$ra` return address.

4.3 Calling and returning: `jal` and `jr`

- `jal ProcedureLabel` (*jump and link*): saves the address of the *following* instruction in `$ra` and jumps to the procedure [MIPS slide 38]. The saved value is the **return address**. ("Link" means leaving a trail back.)
- `jr $ra` (*jump register*): copies `$ra` into the program counter (the PC, the register that holds the address of the next instruction), thereby returning. `jr` can also jump to any computed address, e.g. for `switch/case` statements (Section 4.7).

4.4 The stack

What if a procedure needs more registers than it may freely use? It *spills* registers to memory, to a region called the **stack**, a last-in-first-out structure. `$sp` points to the *top* (most recently pushed item). In MIPS the stack *grows downward*, toward lower addresses:

- **push**: decrease `$sp` then store: `addi $sp,$sp,-4` ; `sw $s0,0($sp)`;
- **pop**: load then increase `$sp`: `lw $s0,0($sp)` ; `addi $sp,$sp,4`.

Worked Example: Leaf procedure [MIPS slide 39–40]

```
int leaf_example (int g, h, i, j)
{ int f; f = (g + h) - (i + j); return f; }
```

Arguments `g,...,j` arrive in `$a0,...,$a3`; `f` is held in `$s0`, so `$s0` must be saved (a saved register!); the result goes in `$v0`.

```
leaf_example:
    addi $sp, $sp, -4 # make room for 1 item
    sw $s0, 0($sp) # save $s0
    add $t0, $a0, $a1 # g + h
    add $t1, $a2, $a3 # i + j
    sub $s0, $t0, $t1 # f = (g+h) - (i+j)
    add $v0, $s0, $zero # return value f
    lw $s0, 0($sp) # restore $s0
    addi $sp, $sp, 4 # pop the stack
    jr $ra # return
```

(A cleverer compiler would use only temporaries and avoid the save/restore entirely.)

4.5 Non-leaf procedures and recursion

A non-leaf procedure uses `jal`, which overwrites `$ra`. So before calling, it must save on the stack [MIPS slide 41]: its own return address, and any arguments or temporaries it needs after the call; it restores them after the call.

Worked Example: Factorial, a recursive non-leaf procedure [MIPS slide 42–43]

```
int fact (int n) { if (n < 1) return 1; else return n * fact(n - 1); }
```

(The slide's C text says `return f`, which is a typo; the assembly clearly returns 1.)

```
fact:
    addi $sp, $sp, -8 # room for 2 items
```

```
    sw $ra, 4($sp) # save return address
    sw $a0, 0($sp) # save argument n
    slti $t0, $a0, 1 # n < 1 ?
    beq $t0, $zero, L1 # no: go to L1
    addi $v0, $zero, 1 # yes: result = 1
    addi $sp, $sp, 8 # pop 2 items (nothing to restore)
    jr $ra # return
L1: addi $a0, $a0, -1 # n - 1
    jal fact # recursive call
    lw $a0, 0($sp) # restore original n
    lw $ra, 4($sp) # restore return address
    addi $sp, $sp, 8 # pop 2 items
    mul $v0, $a0, $v0 # n * fact(n-1)
    jr $ra
```

Why save `$a0`? Because the recursive call changes `$a0` to $n-1$, but after the call we need the original n to multiply. *Why save `$ra`?* Each `jal` overwrites it; without saving, the first call would return to the wrong place.
Trace of `fact(3)`: calls $\texttt{fact}(3) \to \texttt{fact}(2) \to \texttt{fact}(1) \to \texttt{fact}(0)$, which returns 1 without further calls. Unwinding: $\texttt{fact}(1) = 1 \cdot 1 = 1$, $\texttt{fact}(2) = 2 \cdot 1 = 2$, $\texttt{fact}(3) = 3 \cdot 2 = 6$. The stack grows by 8 bytes per active call (4 levels = 32 bytes) and shrinks back.

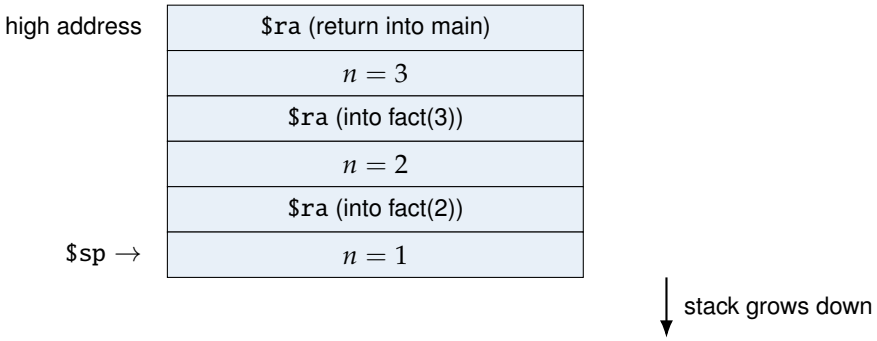


Figure 4.1: Stack contents (conceptually) while `fact(1)` is about to call `fact(0)`.

4.6 Local data, frames and the memory layout

Local variables ("automatic variables" in C) that do not fit in registers (arrays, structures) are also allocated on the stack. The portion of the stack holding a procedure's saved registers and locals is its **procedure frame** or **activation record** [MIPS slide 44]; the **frame pointer** `$fp` may point to the first word of the current frame so that fields can be reached with fixed offsets even while `$sp` moves.

4.6.1 The MIPS memory layout [MIPS slide 45]

A program's memory is divided into four regions (Figure 4.2):

- **Text**: the program code (instructions);
- **Static data**: global and `static` variables, constant arrays and strings — allocated once for the whole life of the program. `$gp` is initialised to point into this segment so that data within ± 32 KB can be reached with a 16-bit offset;
- **Dynamic data** (the *heap*): memory obtained at run time (`malloc` in C, `new` in Java);

- **Stack**: automatic storage, growing downward from the top toward the heap.

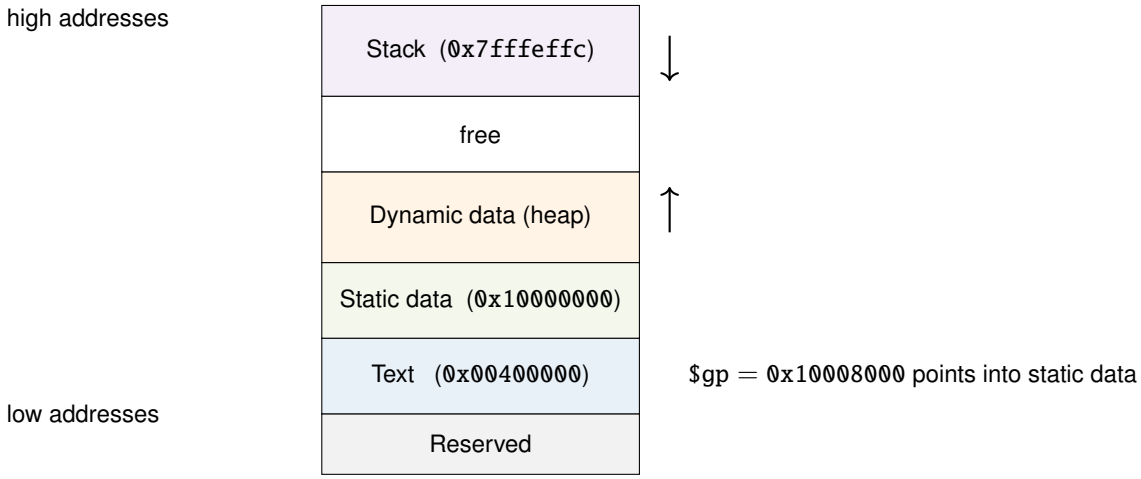


Figure 4.2: Typical MIPS memory layout. The addresses are those of the standard MIPS convention (the exam questions use `0x00400000` for text and `0x10000000` for static data).

4.7 Jump address tables (`switch/case`)

A `switch` with many cases can be compiled as a chain of `if`s, but that is slow. A faster way is a **jump address table**: an array in memory whose k-th entry is the *address* of the code for case k. The program computes the table slot from the case value, loads the address with `lw`, and jumps with `jr`. This is the use of "computed jumps" mentioned on [MIPS slide 38].

Worked Example: Switch with a jump table

```
switch (k) { case 0: f = i + j; break; case 1: f = g + h; break;
             case 2: f = g - h; break; case 3: f = i - j; break; }
```

Let `f,g,h,i,j,k` be in `$s0-$s5`, and `$t4` hold 4, `$t2` hold the base address of the table, which stores the addresses of `L0,L1,L2,L3` in words 0, 1, 2, 3.

```
     slt $t3, $s5, $zero # k < 0 ?
     bne $t3, $zero, Exit # yes: no case matches
     slt $t3, $s5, $t4 # k < 4 ?
     beq $t3, $zero, Exit # no (k >= 4): leave
     sll $t1, $s5, 2 # t1 = 4*k
     add $t1, $t1, $t2 # address of table[k]
     lw $t0, 0($t1) # t0 = address of the case code
     jr $t0 # jump to it
L0: add $s0, $s3, $s4 # f = i + j
     j Exit
L1: add $s0, $s1, $s2 # f = g + h
     j Exit
L2: sub $s0, $s1, $s2 # f = g - h
     j Exit
L3: sub $s0, $s3, $s4 # f = i - j
Exit:
```

Why it is fast: the number of instructions executed is the same (about 8) whether k is 0 or 3, instead of growing with the number of cases. The first two tests (bounds check) protect against jumping to garbage when k is out of range.

4.8 Summary

Arguments in `$a0`–`$a3`, results in `$v0`–`$v1`; `jal` sets `$ra`, `jr $ra` returns. Saved registers (`$s`) must survive a call; temporaries (`$t`) may not. The stack (grows down, `$sp`) holds saved registers, return addresses of non-leaf procedures, and locals. Memory = text, static data, heap, stack.

Lecture source

MIPS slides 36–45 (§2.8).

CHAPTER 5

Characters, Big Constants and Addressing Modes

Chapter goal

Programs also handle text (characters), 32-bit constants that do not fit in a 16-bit field, branch targets and jump targets. This chapter explains how MIPS handles each, and ends with the summary of the five MIPS *addressing modes*, a favourite exam question. *Lecture source: MIPS slides 46–55.*

5.1 Character data

Text is stored as numbers: each character has a code. **ASCII** uses 7 bits for 128 characters (95 printable and 33 control); **Latin-1** extends it to 256 characters; **Unicode** (32-bit code points) covers most alphabets, and its encodings **UTF-8**/**UTF-16** use variable-length sequences [MIPS slide 46]. Because one character in ASCII is one byte, string processing is common enough that MIPS gives direct byte and halfword (16-bit) transfers [MIPS slide 47]:

Load	Store	Behaviour
`lb rt,off(rs)`	`sb rt,off(rs)`	byte; load *sign-extends* to 32 bits
`lbu rt,off(rs)`	—	byte; load *zero-extends*
`lh rt,off(rs)`	`sh rt,off(rs)`	halfword (16 bits); load sign-extends
`lhu rt,off(rs)`	—	halfword; zero-extends

A store writes only the *rightmost* byte/halfword of the register, so it does not need a sign/zero variant. (An exam asks why `shu` is unnecessary: a store transfers bits to memory unchanged; nothing is being extended.)

Worked Example: String copy [MIPS slide 48–49]

A string in C is a sequence of bytes ending in the **null** byte (value 0).

```
void strcpy (char x[], char y[])
{ int i = 0; while ((x[i] = y[i]) != '\0') i += 1; }
```

Addresses of `x`, `y` in `$a0`, `$a1`; `i` in `$s0`.

```
strcpy:
    addi $sp, $sp, -4 # room for 1 item
    sw $s0, 0($sp) # save $s0
    add $s0, $zero, $zero # i = 0
L1: add $t1, $s0, $a1 # address of y[i] (1 byte per char: no *4!)
    lbu $t2, 0($t1) # t2 = y[i]
    add $t3, $s0, $a0 # address of x[i]
    sb $t2, 0($t3) # x[i] = y[i]
    beq $t2, $zero, L2 # exit if the char just copied was 0
    addi $s0, $s0, 1 # i = i + 1
    j L1
```

```
L2: lw $s0, 0($sp) # restore $s0
    addi $sp, $sp, 4 # pop
    jr $ra # return
```

Note that the index is added *directly* to the base (characters are 1 byte), whereas for `int` arrays we would first shift the index left by 2.

5.2 32-bit constants

Most constants are small (the 16-bit immediate suffices [MIPS slide 50]). For the occasional large one, MIPS provides `lui` (*load upper immediate*):

- `lui rt, constant` copies the 16-bit constant into the *left* (upper) 16 bits of `rt` and clears the right 16 bits to zero.
- then `ori rt, rt, constant2` fills the lower 16 bits (OR with zeros leaves the upper half intact).

Worked Example: The slide's constant

To load $0000\,0000\,0111\,1101\,0000\,1001\,0000\,0000_2$ into `$s0`:

```
lui $s0, 61 # s0 = 0000 0000 0111 1101 0000 0000 0000 0000
ori $s0, $s0, 2304 # s0 = 0000 0000 0111 1101 0000 1001 0000 0000
```

since $61 =$ `0x003d` and $2304 =$ `0x0900`; the result is `0x003d0900`. (The slide writes `lhi`, an older name for the same operation; MIPS-32 calls it `lui`.) Use `ori` (a logical OR, which zero-extends) for the lower half, not `addi`, because `addi` sign-extends and would corrupt the upper half whenever bit 15 of the lower part is 1.

5.3 Branch addressing

A conditional branch needs two registers and a target, all inside one 32-bit word:

op (6)	rs (5)	rt (5)	constant or address (16)

With only 16 bits left, the target cannot be a full 32-bit address. But most branch targets are *near* the branch (loops and `if`s are small), forward or backward. So MIPS uses **PC-relative addressing** [MIPS slide 51]:

$$\text{Target address} = \text{PC} + 4 + 4 \times \text{offset}$$

- The offset counts *instructions* (words), not bytes: instructions are word aligned so the low two address bits are always zero and need not be stored; multiplying by 4 gains two extra bits of reach.
- "PC already incremented by 4 by this time": when the branch executes, the PC has already advanced to the next instruction, so the offset is measured from the instruction *after* the branch.
- The 16-bit offset is signed, so the reach is from -2^{15} to $2^{15} - 1$ instructions, i.e. from $4 - 131072$ to $4 + 131068$ bytes relative to the branch's own address (about ± 128 KB).

5.4 Jump addressing

Jumps (`j`, `jal`) can go anywhere in the program, and have no registers to specify, so MIPS gives them 26 bits for the address [MIPS slide 52]:

op (6)	address (26)

(Pseudo)direct addressing:

$$\text{Target address} = \text{PC}_{31\ldots28} \ : \ (\text{address} \times 4)$$

The 26-bit field is multiplied by 4 (shifted left 2) to give 28 bits, and the four missing upper bits are taken from the PC (strictly, from the already-incremented PC, i.e. PC+4; the two differ only if the jump is the last instruction before a 256 MB boundary). The symbol : means concatenation. Consequently a `j` can reach only addresses within the same 256 MB-aligned block as the jump; to go further, load the full address into a register and use `jr`.

Worked Example: Encoding branch and jump targets [MIPS slide 53]

Assume `Loop` is at address 80000 (decimal) in the earlier `while` loop:

Instruction	Addr	op	rs	rt	rd/immediate	shamt	funct
`Loop: sll $t1,$s3,2`	80000	0	0	19	9	2	0
`add $t1,$t1,$s6`	80004	0	9	22	9	0	32
`lw $t0,0($t1)`	80008	35	9	8	0		
`bne $t0,$s5,Exit`	80012	5	8	21	2		
`addi $s3,$s3,1`	80016	8	19	19	1		
`j Loop`	80020	2			20000		
`Exit:`	80024						

bne: the branch is at 80012; the next instruction is at 80016; `Exit` is at 80024, so the offset is $(80024 - 80016)/4 = 2$ words. **j:** `Loop`'s byte address $80000/4 = 20000$ is stored in the 26-bit field.

Correction / clarification to the lecture

The lecture's table prints the `sll` row as "0 0 19 9 4 0", but `sll $t1,$s3,2` shifts by 2, so the correct fields (corrected in the table above) are `op`=0, `rs`=0, `rt`=19, `rd`=9, `shamt`=2, `funct`=0.

5.4.1 Branching far away

If a branch target is too far for the 16-bit offset (Chapter 7), the assembler rewrites the code: it inverts the condition and uses a jump, whose 26-bit address reaches farther [MIPS slide 54]:

```
beq $s0,$s1,L1 ==> bne $s0,$s1,L2
                       j L1
                   L2: ...
```

5.5 Summary of MIPS addressing modes [MIPS slide 55]

Mode	Example	How the operand/target is found
1. **Immediate**	`addi $t0,$t1,5`	the operand *is* the constant (16 bits) in the instruction
2. **Register**	`add $t0,$t1,$t2`	the operand is in the register named by a 5-bit field
3. **Base (displacement)**	`lw $t0,32($s3)`	address = register + sign-extended 16-bit offset; selects a byte, halfword or word
4. **PC-relative**	`beq $t0,$t1,L`	address = (PC+4) + 4× offset (words)
5. **Pseudodirect**	`j L`	address = $PC_{31..28}$ concatenated with 26-bit field ×4

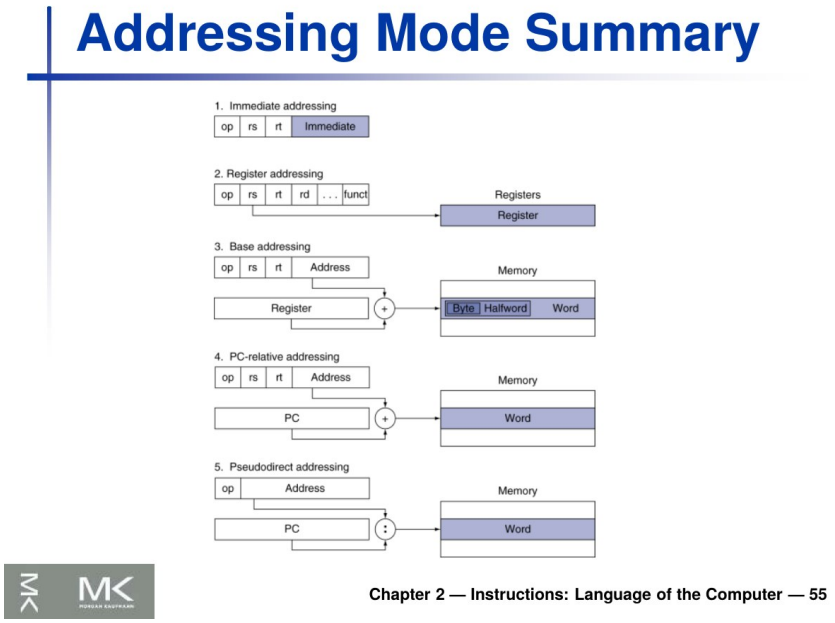


Figure 5.1: Addressing mode summary: the lecture's own picture of the five modes (MIPS slide 55). Shaded boxes show what is used.

Worked Example: Computing an address in each mode

Suppose `$s3`= `0x10001000` and PC= `0x00400020`.

- `lw $t0,32($s3)`: base = `0x10001000` + 32 = `0x10001020`.
- `beq` with offset +5: target = `0x00400020` + 4 + 4 · 5 = `0x00400038`.
- `j` with field `0x0100010` (i.e. 26 bits = `0x0100010`): target = `0x0` : (`0x0100010` × 4) = `0x00400040`.

5.6 Summary

Byte/halfword loads and stores support text; `lui+ori` builds 32-bit constants; branches use PC-relative word offsets (16 bits), jumps use pseudodirect 26-bit word addresses; five addressing modes in all.

Lecture source

MIPS slides 46–55 (§2.9–§2.10).

CHAPTER 6

Synchronization, Translating and Starting a Program

Chapter goal

Two last topics of the ISA: (1) how instructions let two processors safely share data (*synchronisation*), and (2) the chain of software tools — compiler, assembler, linker, loader — that turns a C program into something that runs, including dynamic linking and Java's JIT. We end with the fallacies and pitfalls. *Lecture source: MIPS slides 56–69.*

6.1 Synchronization

Suppose processors P1 and P2 share a memory location. If P1 writes while P2 reads, the result depends on who gets there first. This is a **data race**: two or more accesses to the same location, at least one a write, with no ordering enforced, so the outcome depends on timing [MIPS slide 56]. Programs avoid races by **synchronising**, e.g. using a *lock* variable that a processor must acquire before touching the data.

Acquiring a lock needs a **test-and-set**: read the lock, and if it is free, write "taken" — and nothing else may touch the location between the read and the write, or two processors could both see "free" and both take it. So hardware must offer an **atomic** (indivisible) read-write operation. It could be one instruction (e.g. atomic swap of a register and a memory word) or an *atomic pair* of instructions.

6.1.1 MIPS: `ll` and `sc` [MIPS slide 57]

MIPS uses a pair:

- **load linked**: `ll rt, offset(rs)` — an ordinary load that additionally "remembers" the address;
- **store conditional**: `sc rt, offset(rs)` — stores `rt` to the address *only if* the location has not been changed since the `ll` (by any other processor, or by an interrupt). If the store succeeds, `rt` is set to 1; if it fails, nothing is written and `rt` is set to 0.

Worked Example: Atomic swap of `$s4` with memory at `($s1)`

```
try: add $t0, $zero, $s4 # t0 = value to exchange (copy, since sc overwrites it)
     ll $t1, 0($s1) # t1 = old memory value, link established
     sc $t0, 0($s1) # try to store; t0 becomes 1 (ok) or 0 (failed)
     beq $t0, $zero, try # failed => someone interfered; retry
     add $s4, $zero, $t1 # success: put the old memory value in s4
```

If another processor writes the location between `ll` and `sc`, `sc` fails and the loop restarts, so the swap appears to happen instantaneously. Other primitives (test-and-set, fetch-and-increment) are built the same way: `ll`, modify in a register, `sc`, retry.

6.2 Translation and startup [MIPS slide 58]

A C program becomes a running process in four steps (Figure 6.2):

1. **Compiler**: C → assembly language.
2. **Assembler**: assembly → **object module** (machine code plus bookkeeping). (Many compilers produce object modules directly.)
3. **Linker**: combines object modules and library routines into one **executable**.
4. **Loader**: copies the executable from disk into memory and starts it.

Linking all modules before the program runs is **static linking**.

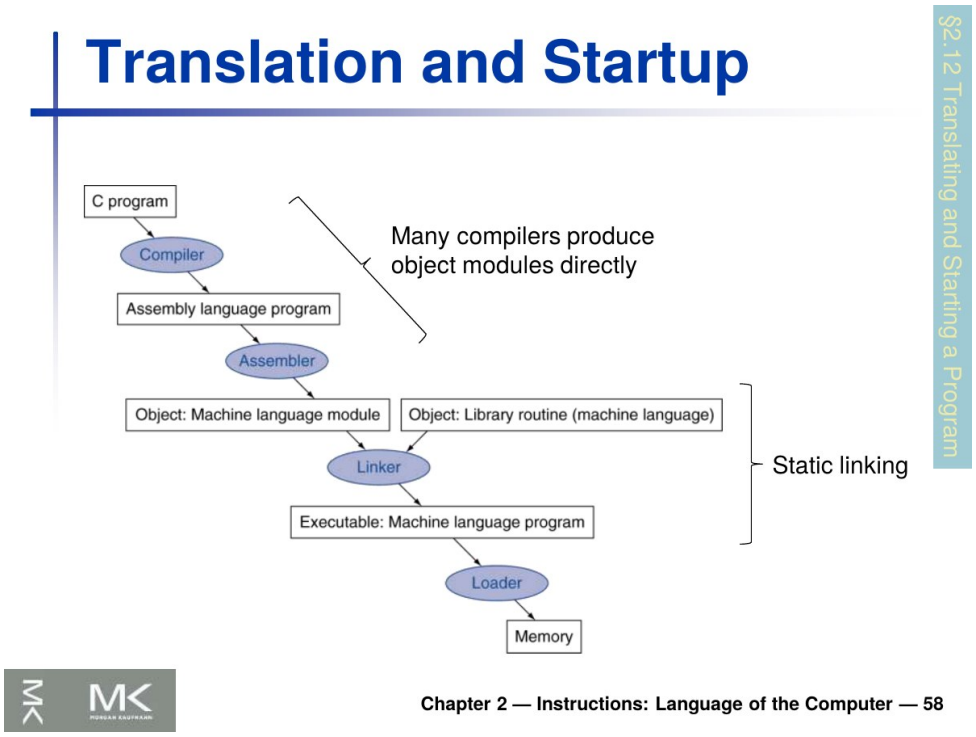


Figure 6.1: Translation and startup hierarchy (MIPS slide 58).

6.2.1 Assembler pseudoinstructions [MIPS slide 59]

Most assembler instructions correspond one-to-one to machine instructions. **Pseudoinstructions** are conveniences that the assembler expands into one or more real instructions:

Pseudoinstruction	Real instructions
`move $t0,$t1`	`add $t0,$zero,$t1`
`blt $t0,$t1,L`	`slt $at,$t0,$t1;bne $at,$zero,L`
`li $t0,100000`	`lui $at,1;ori $t0,$at,34464`

Register 1 (`$at`) is reserved as the *assembler temporary* for such expansions — which is why it must not be used by ordinary programs. Pseudoinstructions make assembly language easier to write without complicating the hardware (the ISA stays small and regular), but hand-written assembly is rarely beneficial: modern compilers schedule code for the processor better than people do (see §6.3).

6.2.2 Producing an object module [MIPS slide 60]

The assembler (or compiler) translates the program to machine instructions and also produces information needed to build the whole program from pieces:

- **Header**: sizes and positions of the other parts;
- **Text segment**: the translated instructions;
- **Static data segment**: data allocated for the life of the program;
- **Relocation information**: which words depend on the absolute location of the loaded program (e.g. `jal` targets and `$gp`-relative data);
- **Symbol table**: global definitions and *external references* (labels defined elsewhere);
- **Debug information**: to map machine code back to source.

6.2.3 Linking object modules [MIPS slide 61]

The **linker** produces an executable image by:

1. *merging* the segments of the modules (all text together, all data together);
2. *resolving labels* — determining their final addresses;
3. *patching* location-dependent and external references, using the relocation information and symbol tables.

It could leave location dependences to a *relocating loader*, but with virtual memory every program can be loaded at the same absolute virtual address, so the linker fixes them once.

Worked Example: Linking two modules — how the patched fields are computed

Module A has text size `0x100`, data size `0x20`; its first text instructions are `lw $a0,0($gp)` (reference to its data item X) and `jal 0` (a call to procedure B, defined in module B). Module B has text `0x200`, data `0x30`.
Choose the standard layout: text starts at `0x00400000`, static data at `0x10000000`, `$gp`= `0x10008000`. The linker places A's text at `0x00400000`, B's text at `0x00400100`, A's data at `0x10000000`, B's data at `0x10000020`.

- `lw $a0,0($gp)` must reach X at `0x10000000`: offset = `0x10000000` − `0x10008000` = −`0x8000` = −32768, encoded as `0x8000`. The instruction becomes `lw $a0,-32768($gp)`.
- `jal 0` must reach B at `0x00400100`: field = `0x00400100`/4 = `0x00100040`.

This is exactly the calculation asked in the 2022–23 CSE 305 exam (Chapter 7).

6.2.4 Loading a program [MIPS slide 62]

The **loader** takes the executable on disk and:

1. reads the header to determine segment sizes;
2. creates a virtual address space;
3. copies text and initialised data into memory (or sets page-table entries so they are brought in on demand by page faults);

4. sets up arguments on the stack;
5. initialises the registers (including `$sp, $fp, $gp`);
6. jumps to a startup routine that copies the arguments into `$a0,...` and calls `main`; when `main` returns, the startup routine performs the *exit* system call.

6.2.5 Dynamic linking [MIPS slide 63–64]

Static linking copies every library routine used (and all the libraries *they* use) into each executable, which bloats the image and freezes old library versions. **Dynamic linking** (as in DLLs — dynamically linked libraries) links and loads a library routine *only when it is first called*. Advantages: the executable is smaller (no bloat); memory is saved because many programs can share one copy of the library; and updated library versions are picked up automatically. The cost: the routine code must be relocatable, and the first call is slower.

Lazy linkage

Calls to a library routine initially go through an *indirection table* entry that points to a small **stub**. On the **first call**, the stub loads the routine's ID and jumps to the dynamic linker/loader; the linker finds and maps the routine and *rewrites the indirection table entry* to point directly at it. On **subsequent calls**, the entry already points to the library routine, so the call goes straight there (Figure 6.2.5).

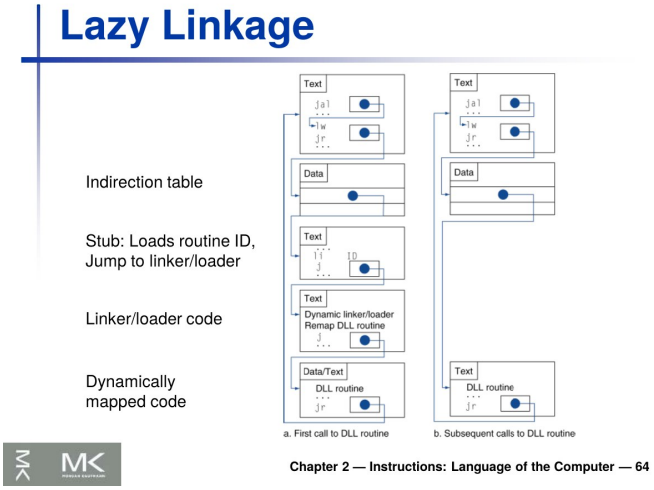


Figure 6.2: Lazy linkage: (a) first call goes through the stub and linker; (b) later calls use the updated table entry (MIPS slide 64).

6.2.6 Starting Java applications [MIPS slide 65]

Java uses a different route to portability. The compiler produces **bytecodes**, a simple portable instruction set of the **Java Virtual Machine** (JVM). The JVM either *interprets* the bytecodes (reads and executes them one at a time — simple but slow) or, for "hot" (frequently executed) methods, uses a **Just-In-Time (JIT) compiler** that translates the bytecodes into native machine code for the host at run time, so later executions run at hardware speed.

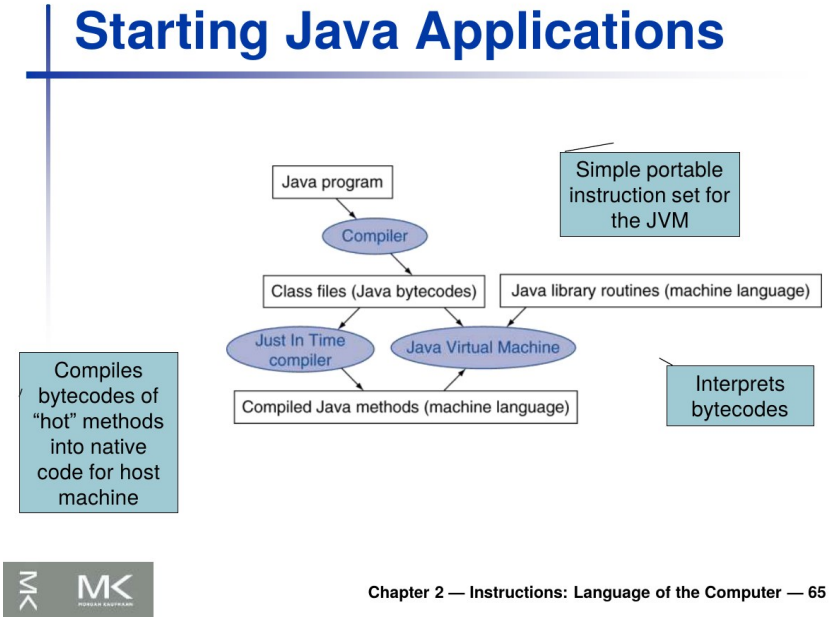


Figure 6.3: Starting a Java application (MIPS slide 65).

6.3 Fallacies and pitfalls [MIPS slide 66–68]

- **Fallacy: a powerful instruction means higher performance.** It means fewer instructions, but complex instructions are hard to implement and can slow down *all* instructions, including the simple ones; compilers do well with simple instructions.
- **Fallacy: use assembly code for high performance.** Modern compilers handle modern processors better, and more lines of code mean more errors and less productivity.
- **Fallacy: backward compatibility means the instruction set does not change.** In fact instruction sets *accrete* new instructions (the x86 instruction set has grown steadily, which is one reason it is complex; examples: variable instruction length of 1–17 bytes, many addressing modes, string and complex instructions).
- **Pitfall: sequential words are not at sequential addresses** — increment by 4, not 1.
- **Pitfall: keeping a pointer to an automatic variable after the procedure returns.** The variable lived on the stack; once the stack is popped the pointer is invalid.

6.4 Concluding remarks [MIPS slide 69]

The four design principles: (1) simplicity favours regularity; (2) smaller is faster; (3) make the common case fast; (4) good design demands good compromises. Layers: compiler, assembler, hardware. MIPS is typical of RISC ISAs; contrast with x86.

Lecture source

MIPS slides 56–69 (§2.11, §2.12, §2.19, §2.20). The 5th-edition book's §2.12 also covers the JVM in more depth; here only the lecture's scope is covered.

CHAPTER 7

MIPS: Solved Past Examination Questions (Topic-wise)

How this chapter is organised

Every question below comes from the supplied CSE 305 / CSE 209 papers (2012–13 to 2023–24) and falls inside the MIPS lecture. Questions are grouped by *topic*, not by year, so you can practise one idea at a time. The paper and question number is printed in each box. Where several years ask almost the same thing, the first box carries the full solution and the later ones point to it, noting any difference. Assembly programs were executed on a small MIPS simulator I wrote to confirm they give correct results; machine-code fields and addresses were re-computed by script.

Topic list

T1. Design principles and ISA-design reasoning

T2. Instruction formats, machine code and instruction design

T3. Addressing modes

T4. Branch and jump reach; big constants; pseudoinstructions

T5. Translating C to MIPS (loops, conditions, arrays)

T6. Procedures, recursion and the stack

T7. Synchronization

T8. Translation, linking, loading, dynamic linking and Java

7.1 T1: Design principles and ISA-design reasoning

Past Exam Question (2016–17 Q6(c)) 15 marks)

The four design principles guiding MIPS are: (i) simplicity favours regularity, (ii) smaller is faster, (iii) good design demands good compromises, (iv) make the common case fast. Identify which principle lies behind each: (a) keeping the instruction immediate in 16 bits; (b) always requiring three register operands in arithmetic instructions; (c) PC-relative addressing for conditional branches; (d) keeping 32 registers rather than many more; (e) keeping register fields in the same place in each instruction format.

Solution.

	Design feature	Principle	Reason
(a)	16-bit immediate	**Make the common case fast** (iv)	most constants in programs are small; a 16-bit field covers them without a load; larger constants are rare and use `lui+ori` (also a compromise, since a 32-bit immediate would not fit a 32-bit instruction)
(b)	always three register operands	**Simplicity favours regularity** (i)	every arithmetic instruction has the same shape, so decoding and the datapath are uniform
(c)	PC-relative branch	**Make the common case fast** (iv)	almost all branch targets are near the branch, so a short offset suffices and fits in the 16-bit field
(d)	32 registers	**Smaller is faster** (ii)	a small register file is faster to access; also 5 bits per register field keeps instructions short
(e)	register fields in the same place	**Good design demands good compromises** (iii) (with regularity)	different formats are needed (R, I) but keeping `op, rs, rt` in the same positions keeps the decoder simple while all instructions stay 32 bits long

Past Exam Question (2013–14 Q6(a) 8 marks; undated paper Q2(a) 8 marks)

What are the four design principles used for the MIPS instruction set? Mention one use of each in the MIPS ISA. (2013–14 adds: What are the steps for starting a Java application?)

Solution.

1. **Simplicity favours regularity** — every arithmetic instruction has three register operands (`add a,b,c`); fixed 32-bit instruction length.

2. **Smaller is faster** — only 32 registers, so the register file is fast.

3. **Make the common case fast** — immediate operands (`addi`) for small constants; PC-relative branches; `beq/bne` as the only conditional branches because equality tests are common and cheap.

4. **Good design demands good compromises** — three instruction formats (R, I, J) of the same length, with fields placed alike; `slt`+`beq/bne` instead of a full set of compare-and-branch instructions.

For the Java steps see Section 7.8 (T8): compile to bytecodes; the JVM interprets them; the JIT compiler turns hot methods into native code.

Past Exam Question (2021–22 Q5(a) 15 marks)

Which design principle(s) are behind: (i) a 16-bit branch offset; (ii) 32 registers; (iii) a consistent format with 3 registers for R-type; (iv) multiple instruction formats for flexibility; (v) pseudoinstructions such as `bgt`, `bge`, `move`?

Solution. (i) *Make the common case fast* (branches are short) — and a compromise, since a 32-bit offset would not fit. (ii) *Smaller is faster*. (iii) *Simplicity favours regularity*. (iv) *Good design demands good compromises*: separate R/I/J formats give flexibility (a 16-bit immediate, a 26-bit jump address) while every instruction is still 32 bits. (v) *Good design demands good compromises* (the hardware stays simple — only `slt`, `beq`, `bne`, `add` exist — while the assembler supplies convenient forms), also *simplicity favours regularity*.

Past Exam Question (2019–20 Q3(c)) 8 marks)

Why does MIPS arithmetic always demand three operands? Why has the number of registers been restricted to 32?

Solution. *Three operands:* **simplicity favours regularity.** With a fixed pattern "destination, source 1, source 2" the hardware always reads two registers and writes one, so the instruction decoder, register file ports and datapath are the same for every arithmetic instruction (an instruction with a variable number of operands would need variable-length encodings and special cases). Three operands also keep the original source values intact (no operand is destroyed, as in a two-address machine), which helps the compiler.
Thirty-two registers: **smaller is faster.** A larger register file is slower to access (longer wires, bigger selection logic) and costs more area; it needs more bits in every register field (each doubling adds 1 bit per field, i.e. 3 bits to an R-type instruction), leaving fewer bits for other fields; and more registers means more state to save on procedure calls and context switches. Compiler studies show that 32 registers give almost all the benefit.

Past Exam Question (2022–23 CSE 305 Q7(b)) 7 marks)

A program has 40 variables but MIPS has only 32 registers. What is the problem and how is it handled?

Solution. Not all 32 registers are available (`$zero, $at, $k0, $k1, $gp, $sp, $fp, $ra` have special roles), so fewer than 32 can hold variables, and a program with 40 live variables cannot keep them all in registers. The compiler **spills**: it keeps the most frequently used (innermost-loop) variables in registers and keeps the others in memory (typically on the stack), loading them with `lw` just before use and writing back with `sw`. This costs extra instructions and memory accesses, so it makes the program slower; "register optimisation is important". (A variable that is not live at the same time as another can share its register, so "40 variables" only needs spilling if more than the available number are *simultaneously* live.)

Past Exam Question (2020–21 Q11(d)) 5 marks)

"Good design demands good compromises." Give an example from MIPS.

Solution. MIPS would like all instructions to be 32 bits (simple fetch and decode) but also wants a 16-bit constant in immediate instructions and a 26-bit jump address; that is impossible with one format. The compromise: three formats (R, I, J), all 32 bits long, with `op`, `rs`, `rt` in the same positions. (Another example: only `beq`/`bne` are branches; `blt` etc. are built from `slt`.)

Past Exam Question (2020–21 Q13(d)) 5 marks; undated paper Q2(e)) 4.5 marks)

Compare x86 (IA-32) with MIPS / give three examples of the complexity of IA-32.

Solution.

	MIPS (RISC)	x86 / IA-32 (CISC)
Instruction length	fixed, 32 bits	variable, 1 to 17 bytes
Operands	load/store: only `lw`/`sw` touch memory; ALU operates on registers	ALU instructions can have a memory operand
Registers	32 general-purpose	8 (special purposes)
Formats	3 regular formats	many formats and addressing modes
History	designed for pipelining	accreted instructions over decades for backward compatibility; internally translated to simple micro-operations

Three examples of IA-32 complexity: variable-length instructions (hard to decode in one step); complex addressing modes (base + scaled index + displacement, with register-update side effects); many special-purpose instructions (string operations, loops) added over time [MIPS slide 66–67].

7.2 T2: Instruction formats, machine code and instruction design

Past Exam Question (2018–19 Q6(c)) 5 marks; 2017–18 Q8(c) 10 marks)

How many instruction formats are there in MIPS? Name each along with the length of each field.

Solution. Three:

R-type	op (6)	rs (5)	rt (5)	rd (5)	shamt (5) funct (6)
I-type	op (6)	rs (5)	rt (5)	immediate / address offset (16)	
J-type	op (6)	target address (26)			

All are 32 bits. R: register arithmetic/logic and shifts (`add, sub, and, or, slt, sll, jr`); I: `addi, andi, ori, slti, lw, sw, lb, sb, beq, bne, lui`; J: `j, jal`.

Past Exam Question (2018–19 Q7(a)) 10 marks)

The following code is loaded at location 60000. Write the MIPS machine code with decimal field values.

```
Loop: sll $t1, $s2, 2
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      beq $t0, $s5, Lp1
      j Exit
Lp1: addi $s2, $s2, 1
      j Loop
Exit:
```

Solution. Addresses: 60000, 60004, 60008, 60012 (`beq`), 60016 (`j Exit`), 60020 (`Lp1`), 60024 (`j Loop`), 60028 (`Exit`). Register numbers: `$t0`=8, `$t1`=9, `$s2`=18, `$s5`=21, `$s6`=22.

Instruction	Addr	op	rs	rt	rd	shamt	funct
`sll $t1,$s2,2`	60000	0	0	18	9	2	0
`add $t1,$t1,$s6`	60004	0	9	22	9	0	32
`lw $t0,0($t1)`	60008	35	9	8		0	
`beq $t0,$s5,Lp1`	60012	4	8	21		1	
`j Exit`	60016	2			15007		
`addi $s2,$s2,1`	60020	8	18	18		1	
`j Loop`	60024	2			15000		

beq offset: $(60020 - (60012 + 4))/4 = 1$. *j Exit:* $60028/4 = 15007$. *j Loop:* $60000/4 = 15000$.

Past Exam Question (2014–15 Q6(c)) 15 marks)

Complete the machine code (code at address 6000, same program shape with `beq $t0,$s5,L1`, `j Exit`, `L1: addi $s2,$s2,1`, `j Loop`). The partly filled fields were: `sll`: 0 0 18 9 _ 0 at 6000; add: at 6004 funct 32; `lw`: 35 at 6008; `beq`: 4 8 21 at 6012; `j Exit` at 6016;

`addi`: 8 18 18 1 at 6020; `j Loop` at 6024; `Exit` at 6028.

Solution. Same method as the previous solution: `sll`: $0, 0, 18, 9, 2, 0$; `add $t1,$t1,$s6`: $0, 9, 22, 9, 0, 32$; `lw $t0,0($t1)`: $35, 9, 8, 0$; `beq`: $4, 8, 21$, $(6020 - 6016)/4 = 1$; `j Exit`: 2, $6028/4 = 1507$; `addi`: $8, 18, 18, 1$; `j Loop`: 2, $6000/4 = 1500$.

Past Exam Question (2016–17 Q6(a)) 10 marks)

The machine code below (addresses 1000–1020) is complete but the assembly is not. Complete the assembly. (As read from the scan.)

	Addr	op	rs	rt	rd	shamt	funct
sll	1000	000000	00000	10000	01111	00010	000000
add	1004	000000	01111	10001	01111	00000	100000
lw	1008	100011	01111	01000	0000000000000000		
bne	1012	000101	01000	10111	0000000000000001		
addi	1016	001000	10000	10000	0000000000000001		
j	1020	000010	00000000000000000011111010				

Solution. Decode field by field (register numbers: 16=`$s0`, 15=`$t7`, 17=`$s1`, 8=`$t0`, 23=`$s7`).

- `sll`: rt=16, rd=15, shamt=2: `sll $t7, $s0, 2`.
- `add`: rs=15, rt=17, rd=15: `add $t7, $t7, $s1`.
- `lw`: op 35, rs=15 (base), rt=8, offset 0: `lw $t0, 0($t7)`.
- `bne`: rs=8, rt=23, offset 1: `bne $t0, $s7, L`, where $L = 1012 + 4 + 4 \cdot 1 = 1020$ (the `j` instruction).
- `addi`: rs=rt=16, constant 1: `addi $s0, $s0, 1`.
- `j`: field $= 0 \ldots 011111010_2 = 250$, address $= 250 \times 4 = 1000$: `j` to the `sll` (label `Loop` at 1000).

Re-encoding this assembly with the field rules reproduces the given bit patterns (checked by script). Note that, as the scan shows an offset of 1, the `bne` target is the `j` at 1020.

Past Exam Question (2012–13 Q1(c)) 8 marks)

The code below is loaded at location 80000. What are the values of the immediate fields of the `bne` and the `j`?

```
Loop: sll $t1,$s3,2
      add $t1,$t2,$s6
      lw $t0,0($t1)
      bne $t0,$s5,Exit
      addi $s3,$s3,1
      j Loop
Exit:
```

Solution. Addresses: `Loop`=80000, `bne`=80012, `addi`=80016, `j`=80020, `Exit`=80024. `bne`: offset $= (80024 - (80012 + 4))/4 = \mathbf{2}$. `j Loop`: field $= 80000/4 = \mathbf{20000}$ (this is the lecture's own example, MIPS slide 53).

Past Exam Question (2013–14 Q6(b)) 12 marks)

Write the MIPS code and the machine code for `A[3] = A[1] - A[2];`. The base address of `A` is in `$s0`. Opcode of `lw` is 35, of `sw` 43; (op, funct) for `add` is (0,32), for `sub` (0,34). `$t0`-`$t7` are registers 8–15 and `$s0`-`$s7` are 16–23.

Solution.

```
lw $t0, 4($s0) # t0 = A[1]
lw $t1, 8($s0) # t1 = A[2]
sub $t2, $t0, $t1 # t2 = A[1] - A[2]
sw $t2, 12($s0) # A[3] = t2
```

	op	rs	rt	rd/imm	shamt	funct
`lw $t0,4($s0)`	35	16	8	4		
`lw $t1,8($s0)`	35	16	9	8		
`sub $t2,$t0,$t1`	0	8	9	10	0	34
`sw $t2,12($s0)`	43	16	10	12		

In hex: `0x8e080004`, `0x8e090008`, `0x01095022`, `0xae0a000c`. (Offsets: 1×4, 2×4, 3×4.)

Past Exam Question (2020–21 Q12(b)) 10 marks)

`L1: bne $t0,$t1,L3` is at address 1000; `L2: j LX` at 1004; `L3: addi $t0,$t1,1` at 1008; `LX` is at address 131072. Opcodes: `j`=2, `bne`=5, `addi`=8; `$t0`=8, `$t1`=9. Give the format and field values.

Solution. **bne** (I-format): op=5, rs=$t0=8, rt=$t1=9, offset $= (1008 - (1000 + 4))/4 = 1$. $\Rightarrow$ 5, 8, 9, 1 = `0x15090001`.
j (J-format): op=2, address field $= 131072/4 = 32768 \Rightarrow$ 2, 32768 = `0x08008000`.
addi $t0,$t1,1 (I-format): op=8, rs=$t1=9, rt=$t0=8, immediate 1 $\Rightarrow$ 8, 9, 8, 1 = `0x21280001`.
(Whether the jump can actually reach `LX` also depends on the upper 4 bits of PC; here address 131072 = `0x00020000` has upper nibble 0, the same as the code at 1004.)

Past Exam Question (2022–23 CSE 305 Q5(a)) 7 marks)

MIPS has `lw/sw` for words and `lb, lbu, sb`. Ross proposes adding `lh, lhu, sh, shu` for halfwords and says you don't need all four. Which are needed?

Solution. A halfword is 16 bits, but registers are 32 bits. On a *load* the missing upper 16 bits must be filled: `lh` sign-extends (for signed `short`), `lhu` zero-extends (for `unsigned short`); both are needed since the same bits mean different numbers. On a *store*, only the low 16 bits of the register are written to memory; there is no extension, so one instruction `sh` suffices — a "`shu`" would do exactly the same and is unnecessary. So Ross is right: three instructions `lh, lhu, sh` (all I-format, with the address halfword-aligned) are enough.

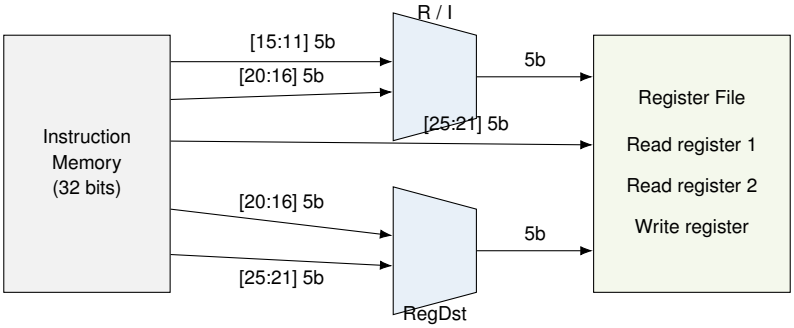
Past Exam Question (2023–24 CSE 209 (22 batch) Q5(a)) 20 marks)

A new ISA, MIPS-v22, has R-format `opcode, rt, rd, rs, shamt, funct` and I-format `opcode, rt, rs, const`. (i) Draw the connections from the Instruction Memory to Register File inputs (Read register 1, Read register 2, Write register) using MUXes; show bit sizes. (ii) With `$t0,$t1,$t2` = 2, 3, 4, shamt=funct=0, encode: `add $t0,$t1,$t2` (opcode 0), `addi $t2,$t0,1` (opcode 6), `lw $t1,4($t0)` (opcode 5).

Solution. **Assumptions.** Field widths as in MIPS: R = 6,5,5,5,5,6; I = 6,5,5,16. As in MIPS, `rs` = first source, `rt` = second source in R-type, `rd` = destination; in I-type `rt` = destination (`addi`, `lw`) and `rs` = source/base.
(i) Bit positions. R: opcode [31:26], rt [25:21], rd [20:16], rs [15:11], shamt [10:6], funct [5:0]. I: opcode [31:26], rt [25:21], rs [20:16], const [15:0]. Compare where each register role lives:

Role	R-type position	I-type position
second source (Read register 2)	rt: [25:21]	(not needed)
first source / base (Read register 1)	rs: **[15:11]**	rs: **[20:16]**
destination (Write register)	rd: **[20:16]**	rt: **[25:21]**

So *Read register 2* can be wired directly to [25:21] (5 bits). *Read register 1* needs a 2×1 MUX (5-bit wide) choosing [15:11] for R-type or [20:16] for I-type. *Write register* needs a 2×1 MUX (5-bit wide, the usual RegDst) choosing [20:16] for R-type or [25:21] for I-type. Both MUX selects come from the control unit (decoded from opcode [31:26]).



(ii) Encoding.

Instruction	Fields	Binary / hex
`add $t0,$t1,$t2`	op 0, rt=4 ($t2), rd=2 ($t0), rs=3 ($t1), shamt 0, funct 0	000000 00100 00010 00011 00000 000000 = `0x00821800`
`addi $t2,$t0,1`	op 6, rt=4 ($t2), rs=2 ($t0), const 1	000110 00100 00010 0000000000000001 = `0x18820001`
`lw $t1,4($t0)`	op 5, rt=3 ($t1), rs=2 ($t0), const 4	000101 00011 00010 0000000000000100 = `0x14620004`

Past Exam Question (2022–23 CSE 209 Q6 — 15 marks)

MIPS hardware does not support memory operands for R-type instructions. Add a new **M-type** instruction that performs only addition and subtraction where one operand is in memory. Design the format (any bit assignment you wish), the conversion protocol from MIPS, any new control signals, and the hardware changes. Your design must not hamper the three original formats.

Solution. **Semantics.** `madd rt, off(rs)`: `rt` ← `rt` + Mem[`rs` + off]; `msub rt, off(rs)`: `rt` ← `rt` − Mem[`rs` + off]. The register operand is also the destination (as in two-operand ISAs); three explicit registers plus a base register and an offset cannot fit in 32 bits.
Format. Reuse the I-format field layout exactly:

op (6) = new opcode	rs (5) = base register	rt (5) = register operand and destination	offset (16)

Two unused opcodes are assigned: one for `madd`, one for `msub` (the opcode itself selects add or subtract, so no extra bits are needed). *Justification:* the new instructions need a base register, a second register and an offset — exactly the I-format fields — so the field positions are the same as `lw`: the register-file read/write addresses, the sign-extender and the decoder are unchanged,

and R, I, J encodings are untouched.
Hardware changes (single-cycle datapath):

1. The existing ALU computes the address `rs` + sign-extended offset (ALUSrc=1, ALUOp=add), exactly as for `lw`, and data memory is read (MemRead=1).
2. **A second adder/subtractor (ALU2)** is added after the data memory: its inputs are *Read data 2* (the value of `rt`, which must therefore be read in the same cycle) and the *memory read data*; a new control signal `M-Sub` (0=add, 1=sub, derived from the opcode) selects the operation. (A second ALU is needed because the first one is busy with address calculation.)
3. The write-data MUX (MemtoReg) gets a third input, the ALU2 result, so MemtoReg becomes 2 bits; RegDst selects `rt` (as for I-type) and RegWrite=1; MemWrite=0.

Effect: the clock cycle of a single-cycle design lengthens because the M-type path is IF → register read → ALU → memory → ALU2 → register write. (In a multi-cycle or pipelined implementation the extra step would instead be an additional cycle/stage, e.g. using the main ALU again.)

Past Exam Question (2022–23 CSE 209 Q11(b) 8 marks)

How does `jal` differ from an unconditional `j`? How is the NOT operation performed in MIPS?

Solution. `j` only replaces the PC with the target address. `jal` (jump and link) additionally writes the address of the next instruction (PC + 4) into register `$ra` (31), so the callee can return with `jr $ra`; it is therefore the procedure-call instruction. MIPS has no NOT instruction; use `nor $t0,$t1,$zero`, since $\neg(x \text{ OR } 0) = \neg x$ [MIPS slide 28].

7.3 T3: Addressing modes

Past Exam Question (2018–19 Q6(a) 15; 2015–16 Q6(b) 15; 2014–15 Q7(a) 15; 2013–14 Q7(b) 4; 2016–17 Q7(a) 10; undated Q2(d) 10; 2021–22 Q8(c) 10; 2022–23 CSE 209 Q11(c) 10; 2023–24 Q6(c) 9; 2020–21 Q13(b) 7)

What are the different addressing modes of MIPS? Describe and illustrate the calculation process for each. (Variants: draw the illustrations and give an example of each; explain PC-relative, immediate and base addressing with examples; calculate a PC-relative target.)

Solution. MIPS has five addressing modes: how an instruction finds its operand or target.

1. **Immediate addressing.** The operand is a 16-bit constant inside the instruction (I-format), sign-extended to 32 bits. Example: `addi $t0,$t1,5` gives `$t0` = `$t1` + 5 — no memory access. Used for small constants.
2. **Register addressing.** The operand is in a register named by a 5-bit field (`rs, rt, rd`). Example: `add $t0,$t1,$t2`.
3. **Base (displacement) addressing.** Memory address = (register named by `rs`) + sign-extended 16-bit offset. Example: `lw $t0,32($s3)`: if `$s3=0x10001000` then the address is `0x10001020`; it selects a byte, halfword or word depending on the instruction (`lb, lh, lw`).
4. **PC-relative addressing.** Used by conditional branches: target = (PC+4) + 4 × (sign-extended 16-bit offset). Example: `beq $t0,$t1,L` at address 1000 with offset 3 branches to

$1000 + 4 + 12 = 1016$. Reach about ± 128 KB.

5. **Pseudodirect addressing.** Used by `j, jal`: target = $\text{PC}_{31..28}$: (26-bit field $\times 4$). Example: field = 20000 gives address 80000 if the PC is in the same 256 MB block.

(And `jr $ra` is a register jump: the target is the full contents of a register.) *Illustration:* see Figure 7.1 and the lecture picture (slide 55) reproduced in Chapter 5. *Why different modes?* Each

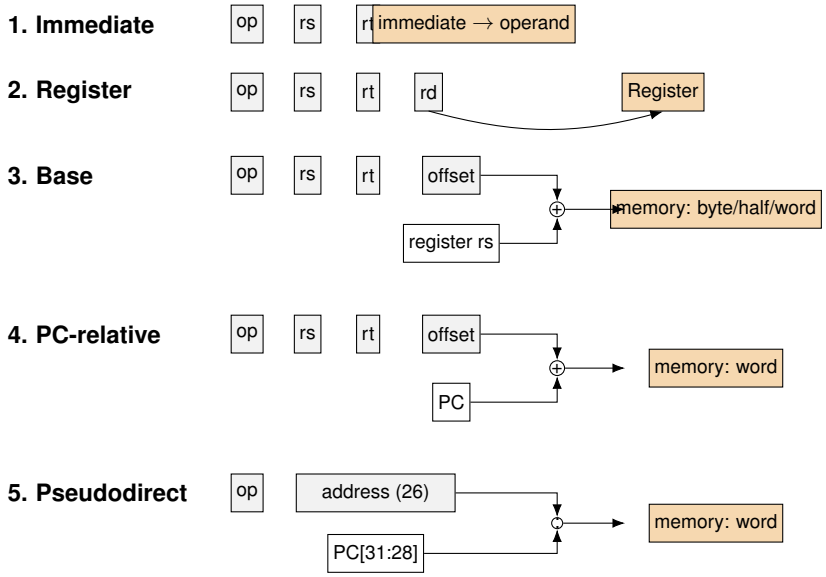


Figure 7.1: The five MIPS addressing modes. Shaded orange = the operand or target finally used. In PC-relative mode the offset is multiplied by 4 and added to PC+4; in pseudodirect mode the 26-bit field is shifted left by 2 and prefixed by the top 4 bits of PC.

fits a purpose: immediate for constants (avoid a load), register for speed, base for arrays/structures/stack, PC-relative because branch targets are near, pseudodirect because jump targets are anywhere in the code.

Past Exam Question (practice, modelled on the PC-relative calculation of 2020–21 Q13(b))

A `beq` at address 4000 branches to a label at address 4040. Find the offset stored in the instruction.

Solution. Offset $= (\text{target} - (\text{PC} + 4))/4 = (4040 - 4004)/4 =$ **9** words. Conversely, offset 9 at address 4000 gives target $4000 + 4 + 36 = 4040$.

7.4 T4: Branch and jump reach; big constants; pseudoinstructions

Past Exam Question (2012–13 Q1(b) 5; 2015–16 Q5(b) 6)

How can you create a 32-bit constant in MIPS? Give the code to load the constant `0001 0000 0011 1101 0000 1001 0000 0000` into `$s0`.

Solution. The 16-bit immediate field cannot hold 32 bits, so build the constant in two steps: `lui` loads the *upper* 16 bits (and clears the lower 16), then `ori` supplies the lower 16 bits. Split the constant: upper half 0001 0000 0011 1101 = `0x103d` = 4157; lower half 0000 1001 0000 0000 = `0x0900` = 2304.

```
lui $s0, 4157 # s0 = 0x103D0000 (4157 = 0x103D)
ori $s0, $s0, 2304 # s0 = 0x103D0900 (2304 = 0x0900)
```

Past Exam Question (2022–23 CSE 209 Q12(c)) 10)

(i) Write the MIPS code that creates the 32-bit constant $0010\,0000\,0000\,0001\,0100\,1001\,0010\,0100_2$ in `$t1`. (ii) If the current PC is `0x00000000`, can you use a single jump instruction to go to the address equal to that constant? (iii) If the current PC is `0x1FFFF000`, can you use a single branch instruction to go to that address?

Solution. **(i)** The constant is `0x20014924`: upper half `0x2001`, lower half `0x4924`.

```
lui $t1, 0x2001
ori $t1, $t1, 0x4924
```

(ii) No. A `j` supplies only 28 bits of the address (26 bits $\times 4$); the top 4 bits come from the PC (PC+4): 0 here. The target's top 4 bits are 2 (`0x2...`), so it lies outside the 256 MB block `0x0000 0000`–`0x0FFF FFFF` that a jump at this PC can reach. (A register jump, `jr $t1`, would work.)
(iii) Yes. Branches are PC-relative: offset $= (\texttt{0x20014924} - (\texttt{0x1FFFF000} + 4))/4 = \texttt{0x15920}/4 = \texttt{0x5648} = 22088$ words, which lies in the signed 16-bit range (≤ 32767), and the target is word-aligned. (Note that a `j` from this PC would *still* fail: PC+4 $=$ `0x1FFFF004` has top nibble 1, not 2.) A branch must have a condition, e.g. `beq $zero,$zero,offset` for an unconditional branch.

Past Exam Question (2021–22 Q6(b)) 10)

Assume PC=`0x20000000`. Can you jump to the address `0x40000000` with a single jump instruction? Can you reach it with a single branch?

Solution. Jump: no — the upper 4 bits of PC+4 are 2, the target's are 4 (different 256 MB regions). Branch: no — the offset would be $(\texttt{0x40000000} - \texttt{0x20000004})/4 = 134{,}217{,}727$ words, far beyond $2^{15} - 1 = 32767$. Use `lui $t0,0x4000` followed by `jr $t0`.

Past Exam Question (2017–18 Q7(b)) 10)

Write the MIPS assembly code to jump to the instruction at address `0x40000001`.

Solution. A `j` cannot do it (the target is not a multiple of 4; the field holds only address/4). Load the full address into a register and use `jr`:

```
lui $t0, 0x4000 # t0 = 0x40000000
ori $t0, $t0, 0x0001 # t0 = 0x40000001
jr $t0 # PC = 0x40000001
```

Caution: MIPS instructions are 4-byte aligned, so fetching an instruction from `0x40000001` is an illegal (misaligned) address and raises an address-error exception. The code shows how a jump to an arbitrary 32-bit address is written; the address in the question is probably a trap or a typo for `0x40000000`.

Past Exam Question (2023–24 CSE 209 Q8(c) 5; 2012–13 undated Q2(c) 3.5; 2013–14 Q6(d) 3; 2012–13 Q1(d) 7)

`beq $s0, $s1, L1` where `L1` is too far for the 16-bit offset. Replace the `beq` by a pair of instructions that reaches farther. (2012–13: why might the assembler have problems with a `beq` to a distant label, and how is it fixed?)

Solution. The 16-bit offset reaches only about $\pm 2^{15}$ instructions (128 KB). If the distance is

greater, the assembler cannot encode the `beq`. It *inverts the condition* and adds a jump (which has 26 address bits):

```
     bne $s0, $s1, L2 # opposite condition: skip the jump
     j L1 # reaches farther (any address in the same 256 MB block)
L2: ...
```

If the two registers are equal the `bne` falls through, the `j` executes and control reaches `L1`; if they are unequal the `bne` jumps over it to `L2`. (If `L1` is beyond even the jump's 256 MB block, load the address with `lui/ori` and use `jr`.)

Past Exam Question (2018–19 Q8(c)) 5; 2014–15 Q7(c) 5+5)

What is the problem in the following code and how can you solve it? `Address 005000: beq $s0, $s1, 30500`

Solution. The problem is the branch range. Reading both numbers as *hexadecimal* (the usual convention for addresses in these papers), the branch is at `0x5000` and the target is `0x30500`. The required offset is $(\texttt{0x30500} - (\texttt{0x5000} + 4))/4 = \texttt{0x2b4fc}/4 = 44351$ words, which exceeds the largest 16-bit signed offset 32767; the instruction cannot be encoded. Fix: invert the condition and use a jump:

```
005000: bne $s0, $s1, 005008 # skip next instruction if different
005004: j 30500 # jump has a 26-bit address field
005008: ...
```

(Remark: if both numbers were read as decimal, the offset would be $(30500 - 5004)/4 = 6374$, which fits; the question is only a "problem" under the hexadecimal reading.)

Past Exam Question (2014–15 Q7(b)) 5+5)

MIPS supports `beq` and `bne` but not `blt, bge`, etc. What is the reason, and how can you program them?

Solution. *Reason:* hardware to evaluate $<, \geq$ is slower than for $=, \neq$; combining it with the branch would make every instruction pay (a slower clock). `beq/bne` are the common case, so MIPS makes them fast (Principle 3/4) [MIPS slide 34]. *How to program:* use `slt` and `beq/bne` with `$zero`:

`blt $s0,$s1,L`	`slt $t0,$s0,$s1 ; bne $t0,$zero,L`
`bge $s0,$s1,L`	`slt $t0,$s0,$s1 ; beq $t0,$zero,L`
`bgt $s0,$s1,L`	`slt $t0,$s1,$s0 ; bne $t0,$zero,L`
`ble $s0,$s1,L`	`slt $t0,$s1,$s0 ; beq $t0,$zero,L`

(The assembler provides these as pseudoinstructions using `$at`.)

Past Exam Question (2015–16 Q5(c)) 4; 2019–20 Q3(b)) 6+2+4)

What are pseudoinstructions? Give two examples; describe how they work with an example. If there is a performance-critical section, is it always a good idea to write the assembly yourself? Justify.

Solution. Pseudoinstructions are instructions that exist in assembly language but not in the hardware: the assembler replaces each with one or more real instructions [MIPS slide 59]. `move $t0,$t1` → `add $t0,$zero,$t1`; `blt $t0,$t1,L` → `slt $at,$t0,$t1 ; bne $at,$zero,L` (register `$at` is reserved as the assembler temporary); `li $t0,100000` → `lui $at,1 ; ori $t0,$at,34464`. They add convenience without adding hardware.

Hand-written assembly? Not necessarily. Modern compilers know the pipeline and allocate registers and schedule instructions better than most programmers; hand-written code takes more lines (more bugs, less productivity), is not portable and hard to maintain. Assembly is worthwhile only for a measured hot spot where the compiler is demonstrably weak (the lecture lists "use assembly code for high performance" as a fallacy) [MIPS slide 66].

Past Exam Question (2012–13 Q1(a)) 15)

What is a jump address table? Discuss with an example how case/switch statements can be implemented with it.

Solution. A **jump address table** is an array in memory whose entries are the addresses of the code for each case. The program uses the case value as an index, loads the address, and jumps to it with `jr`, so reaching any case costs about the same number of instructions instead of a chain of compares. The full worked example (with a bounds check) is in Chapter 4, Section on jump address tables: `sll` (index $\times 4$), `add` (table base), `lw` (load address), `jr`.

7.5 T5: Translating C into MIPS

Past Exam Question (2019–20 Q3(a)) 15)

Write the MIPS code for: (i) `sum = 0; x = 100; while (x >= -5) { sum = sum + x; x = x - 1; }` (ii) `if (x > y+131075) x = 1;` (iii) `x = A[131075] + 5;`

Solution. **(i)** `sum` in `$s0`, `x` in `$s1`. The loop continues while $x \geq -5$, i.e. it exits when $x < -5$ (`slti`):

```
     add $s0, $zero, $zero # sum = 0
     addi $s1, $zero, 100 # x = 100
Loop: slti $t0, $s1, -5 # t0 = (x < -5)
     bne $t0, $zero, Exit # if x < -5, the loop condition x >= -5 is false
     add $s0, $s0, $s1 # sum = sum + x
     addi $s1, $s1, -1 # x = x - 1
     j Loop
Exit:
```

(Checked: the loop adds $100 + 99 + \cdots + (-5) = 5035$.)
(ii) `x` in `$t1`, `y` in `$t2`. The constant 131075 does not fit in 16 bits (> 32767), so `addi` cannot add it; build it with `lui`/`ori`: $131075 = 2 \cdot 65536 + 3$.

```
     lui $t3, 2 # t3 = 2 * 65536 = 131072
     ori $t3, $t3, 3 # t3 = 131075
     add $t3, $t2, $t3 # t3 = y + 131075
     slt $t4, $t3, $t1 # t4 = (y + 131075 < x) i.e. x > y + 131075
     beq $t4, $zero, Skip # condition false: skip
     addi $t1, $zero, 1 # x = 1 (use 8 in the 2022-23 version)
Skip:
```

(iii) Address of `A[131075]` $= A + 131075 \times 4 = A + 524300$, and $524300 = 8 \cdot 65536 + 12$. The offset does not fit in 16 bits, but we can split it: add the large part 524288 to the base using `lui` and keep the small part 12 as the offset. `A` in `$s6`, `x` in `$s0`:

```
     lui $t0, 8 # t0 = 8 * 65536 = 524288
     add $t0, $t0, $s6 # t0 = A + 524288
     lw $t1, 12($t0) # t1 = Mem[A + 524288 + 12] = A[131075]
     addi $s0, $t1, 5 # x = A[131075] + 5
```

Past Exam Question (2022–23 CSE 305 Q6(c)) 8)

Write MIPS code for `if (x > y + 131075) x = 8;` with `x` in `$t1`, `y` in `$t2`.

Solution. Identical to the previous solution (ii); only the last instruction changes to `addi $t1, $zero, 8`.

Past Exam Question (2023–24 CSE 209 Q6(b)) 13)

Write the MIPS assembly code for the program below (a=$t0, b=$t1, c=$t2, N=$t3).

```
int main() { int a = 5, b = 10, c = 2; int N = 30;
  do { if ((a << 2) > b) { a += 2; b += a; }
       else { a += c; b += c; }
  } while (a < N);
  return 0; }
```

Solution. Strategy: test the *negation* of the `if` condition to jump to the `else`; put the loop test at the bottom (a do-while executes the body once before testing). `(a«2) > b` is the same as `b < (a«2)`, i.e. `slt` with swapped operands.

```
main: addi $t0, $zero, 5 # a = 5
      addi $t1, $zero, 10 # b = 10
      addi $t2, $zero, 2 # c = 2
      addi $t3, $zero, 30 # N = 30
Loop: sll $t4, $t0, 2 # t4 = a << 2
      slt $t5, $t1, $t4 # t5 = (b < (a<<2)), i.e. (a<<2) > b
      beq $t5, $zero, Else # condition false -> else part
      addi $t0, $t0, 2 # a += 2
      add $t1, $t1, $t0 # b += a
      j Cond # skip the else part
Else: add $t0, $t0, $t2 # a += c
      add $t1, $t1, $t2 # b += c
Cond: slt $t6, $t0, $t3 # t6 = (a < N)
      bne $t6, $zero, Loop # repeat while a < N
      add $v0, $zero, $zero # return 0
      jr $ra
```

(Simulation: the loop ends with $a = 31$, $b = 121$, matching a direct C computation.)

Past Exam Question (2020–21 Q11(c)) 12)

Write MIPS code for `int testFunc(int a, int b, int c)` with `i = 0, result = 0; do { if (i > b) result += c; i++; } while (i < a); return result;`. Use `$s1` for `i` and `$s2` for `result`.

Solution. Arguments `a,b,c` arrive in `$a0,$a1,$a2`. Because `$s1` and `$s2` are *saved* registers, the procedure must save and restore them.

```
testFunc:
      addi $sp, $sp, -8 # save the two saved registers we use
      sw $s1, 4($sp)
      sw $s2, 0($sp)
      add $s1, $zero, $zero # i = 0
      add $s2, $zero, $zero # result = 0
Loop: slt $t0, $a1, $s1 # t0 = (b < i) i.e. i > b
```

```
     beq $t0, $zero, Skip # not (i > b): skip the addition
     add $s2, $s2, $a2 # result += c
Skip: addi $s1, $s1, 1 # i++
     slt $t0, $s1, $a0 # t0 = (i < a)
     bne $t0, $zero, Loop # while (i < a)
     add $v0, $s2, $zero # return result
     lw $s2, 0($sp) # restore
     lw $s1, 4($sp)
     addi $sp, $sp, 8
     jr $ra
```

Past Exam Question (2017–18 Q7(a)) 25)

Write the MIPS code for the `swap` and `sort` functions (insertion-style exchange sort over `v[0..n-1]`).

Solution. Registers: `v` in `$a0`, `k/n` in `$a1`; in `sort`, `i` in `$s0`, `j` in `$s1`, and the saved copies of the arguments in `$s2` (v) and `$s3` (n), because `$a0,$a1` are overwritten when calling `swap`. `sort` is a non-leaf procedure, so it saves `$ra` and the four `$s` registers.

```
swap: sll $t1, $a1, 2 # t1 = k * 4
     add $t1, $a0, $t1 # t1 = address of v[k]
     lw $t0, 0($t1) # t0 = v[k] (temp)
     lw $t2, 4($t1) # t2 = v[k+1]
     sw $t2, 0($t1) # v[k] = v[k+1]
     sw $t0, 4($t1) # v[k+1] = temp
     jr $ra

sort: addi $sp, $sp, -20 # room for 5 registers
     sw $ra, 16($sp)
     sw $s3, 12($sp)
     sw $s2, 8($sp)
     sw $s1, 4($sp)
     sw $s0, 0($sp)
     add $s2, $a0, $zero # s2 = v (a0 will be reused for swap)
     add $s3, $a1, $zero # s3 = n
     add $s0, $zero, $zero # i = 0
for1: slt $t0, $s0, $s3 # i < n ?
     beq $t0, $zero, exit1
     addi $s1, $s0, -1 # j = i - 1
for2: slti $t0, $s1, 0 # j < 0 ?
     bne $t0, $zero, exit2 # yes: the test j >= 0 fails
     sll $t1, $s1, 2
     add $t2, $s2, $t1 # t2 = address of v[j]
     lw $t3, 0($t2) # t3 = v[j]
     lw $t4, 4($t2) # t4 = v[j+1]
     slt $t0, $t4, $t3 # v[j+1] < v[j] ? (v[j] > v[j+1])
     beq $t0, $zero, exit2
     add $a0, $s2, $zero # argument 1: v
     add $a1, $s1, $zero # argument 2: j
     jal swap
     addi $s1, $s1, -1 # j = j - 1
     j for2
exit2: addi $s0, $s0, 1 # i = i + 1
     j for1
exit1: lw $s0, 0($sp)
     lw $s1, 4($sp)
     lw $s2, 8($sp)
```

```
    lw $s3, 12($sp)
    lw $ra, 16($sp)
    addi $sp, $sp, 20
    jr $ra
```

(Tested on random arrays of length 1–9; the result was always correctly sorted.) The inner loop's condition `j >= 0 && v[j] > v[j+1]` is implemented as two separate exits (`exit2`), using "short-circuit" evaluation: the array is not read when $j < 0$.

7.6 T6: Procedures, recursion and the stack

Past Exam Question (undated paper Q2(b)) 7.5)

Which instructions and registers are used in procedure calling in MIPS hardware? What is the purpose of each?

Solution. Instructions: `jal Label` (jump and link: saves PC+4 in `$ra` and jumps to the procedure); `jr $ra` (return by copying `$ra` to the PC). Registers: `$a0-$a3` (pass up to four arguments), `$v0-$v1` (return values), `$ra` (return address), `$sp` (stack pointer, for saved registers and locals), `$fp` (frame pointer), `$s0-$s7` (saved registers: preserved across calls; callee saves/restores if it uses them), `$t0-$t9` (temporaries: may be destroyed by the callee), `$gp` (global pointer to static data). See Chapter 4.

Past Exam Question (2013–14 Q6(c) 12; 2014–15 Q6(a) 15)

Write the MIPS code for `int sum(int n) { if (n <= 0) return 0; else return n + sum(n-1); }` (`n` in `$a0`, result in `$v0`). (2014–15 gives the same program as a fill-in-the-blanks template `recurseSum` with `if (n<1) return 0`; the completed template is below.)

Solution. Since $n \leq 0 \iff n < 1$, the same program serves both questions. It is a non-leaf procedure: it must save `$ra` (overwritten by the recursive `jal`) and `$a0` (changed to $n-1$ but needed afterwards for $n + \ldots$). The blanks of the 2014–15 template are the `slti $t0,$a0,1`, the two `jr $ra`, the `addi $sp,$sp,-8`, `sw $a0,0($sp)`, `addi $a0,$a0,-1`, `lw $a0,0($sp)`, `addi $sp,$sp,8`, `add $v0,$a0,$v0`.

```
recurseSum:
    slti $t0, $a0, 1 # n < 1 ?
    beq $t0, $zero, Recurse
    add $v0, $zero, $zero # return 0
    jr $ra
Recurse:
    addi $sp, $sp, -8
    sw $ra, 4($sp)
    sw $a0, 0($sp)
    addi $a0, $a0, -1 # n - 1
    jal recurseSum
    lw $a0, 0($sp) # restore n
    lw $ra, 4($sp)
    addi $sp, $sp, 8
    add $v0, $a0, $v0 # n + recurseSum(n-1)
    jr $ra
```

(Tested for $n = 0..7$: result $n(n+1)/2$.)

Past Exam Question (2021–22 Q6(a)) 15)

Write MIPS code for the recursive Fibonacci function `fib(n)`.

Solution. Definition used: $\text{fib}(0) = 0$, $\text{fib}(1) = 1$, $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$. The result of the first recursive call must be kept while the second call runs, so we save it on the stack along with `$ra` and `n`.

```
fib: slti $t0, $a0, 2 # n < 2 ?
     beq $t0, $zero, Rec
     add $v0, $a0, $zero # fib(0)=0, fib(1)=1: return n
     jr $ra
Rec: addi $sp, $sp, -12
     sw $ra, 8($sp)
     sw $a0, 4($sp) # save n
     addi $a0, $a0, -1
     jal fib # v0 = fib(n-1)
     sw $v0, 0($sp) # save fib(n-1)
     lw $a0, 4($sp) # restore n
     addi $a0, $a0, -2
     jal fib # v0 = fib(n-2)
     lw $t0, 0($sp) # t0 = fib(n-1)
     add $v0, $v0, $t0 # fib(n-1) + fib(n-2)
     lw $ra, 8($sp)
     addi $sp, $sp, 12
     jr $ra
```

(Tested for $n = 0..9$.)

Past Exam Question (2022–23 CSE 209 Q9(c)) 10)

Convert to MIPS: `int fun(int a, int b) { if (b == 0) return 0; else if (b / 2 == 0) return fun(a*4, b/8); return fun(a+a, b/4) + a; }`

Solution. `a` in `$a0`, `b` in `$a1`, result in `$v0`. Multiplications and divisions by powers of two are done with shifts (`b` assumed non-negative, so `srl` = divide): $a \times 4 = a \ll 2$, $b/2 = b \gg 1$, $b/8 = b \gg 3$, $b/4 = b \gg 2$. The first recursive return is a plain call whose result is returned unchanged (so `$a0` need not be restored); the second needs the original `a` afterwards.

```
fun: addi $sp, $sp, -8 # frame: 2 words
     sw $ra, 4($sp) # save return address
     sw $a0, 0($sp) # save a (needed after the second kind of call)
     bne $a1, $zero, L1 # if (b == 0) ...
     add $v0, $zero, $zero # return 0
     addi $sp, $sp, 8
     jr $ra
L1: srl $t0, $a1, 1 # t0 = b / 2 (b >= 0 assumed)
     bne $t0, $zero, L2 # if (b/2 != 0) go to the last return
     sll $a0, $a0, 2 # a * 4
     srl $a1, $a1, 3 # b / 8
     jal fun # return fun(a*4, b/8)
     lw $ra, 4($sp) # the result is already in $v0
     addi $sp, $sp, 8
     jr $ra
L2: add $a0, $a0, $a0 # a + a
     srl $a1, $a1, 2 # b / 4
     jal fun # fun(a+a, b/4)
     lw $a0, 0($sp) # restore the original a
     lw $ra, 4($sp)
```

```
      addi $sp, $sp, 8
      add $v0, $v0, $a0 # result + a
      jr $ra
```

(Compared with a direct C evaluation for six (a, b) pairs; all matched.)

Past Exam Question (2018–19 Q8(a)) 20)

Write MIPS code for the function that calculates a^b: `int fun(int a, int b) { if (b == 0) return 0; else if (b % 2 == 0) return fun(a*a, b/2); return fun(a*a, b/2) * a; }`. Use a minimal number of instructions.

Solution. Parity of `b` is decided *before* the call with `andi`, since `$a1` is modified by the call. Only the odd branch needs the original `a` afterwards (to multiply), so only there is it restored.

```
fun: addi $sp, $sp, -8
      sw $ra, 4($sp)
      sw $a0, 0($sp) # save a
      bne $a1, $zero, L1
      add $v0, $zero, $zero # b == 0: return 0 (as printed in the question)
      addi $sp, $sp, 8
      jr $ra
L1: andi $t0, $a1, 1 # t0 = b % 2
      bne $t0, $zero, Odd
      mul $a0, $a0, $a0 # even: fun(a*a, b/2)
      srl $a1, $a1, 1
      jal fun
      lw $ra, 4($sp)
      addi $sp, $sp, 8
      jr $ra # result already in $v0
Odd: mul $a0, $a0, $a0 # odd: fun(a*a, b/2) * a
      srl $a1, $a1, 1
      jal fun
      lw $a0, 0($sp) # original a
      lw $ra, 4($sp)
      addi $sp, $sp, 8
      mul $v0, $v0, $a0
      jr $ra
```

Remark on the question: as printed, the base case returns `0`, which makes every result 0 (all products contain the factor 0). To compute a^b the base case must return 1: replace the instruction after `bne` by `addi $v0,$zero,1`. With that change the code returned $2^5 = 32$, $3^4 = 81$ and $2^{10} = 1024$ on the simulator. (An exam answer should follow the printed code, and may note this.)

Past Exam Question (2015–16 Q5(a)) 25)

Write the MIPS code for `void rstrcopy(char *dest, char *source) { if (*source) { *dest++ = *source++; rstrcopy(dest, source); } else *dest = '\0'; }` with `dest, source` in `$a0, $a1`. Use a minimal number of instructions.

Solution. The recursive call is the *last* thing done (a *tail call*), so it can be turned into a jump to the beginning, and then no stack frame or `$ra` save is needed — the minimal solution:

```
rstrcopy:
      lbu $t0, 0($a1) # t0 = *source
      beq $t0, $zero, Else # if (*source == 0) go to else
      sb $t0, 0($a0) # *dest = *source
```

```
     addi $a0, $a0, 1 # dest++
     addi $a1, $a1, 1 # source++
     j rstrcopy # tail call: nothing is left to do after it
Else: sb $zero, 0($a0) # *dest = '\0'
     jr $ra
```

If a straightforward (non-optimised) recursion is required, keep `$ra` on the stack:

```
rstrcopy:
     lbu $t0, 0($a1)
     beq $t0, $zero, Else
     addi $sp, $sp, -4 # non-leaf version: keep $ra
     sw $ra, 0($sp)
     sb $t0, 0($a0)
     addi $a0, $a0, 1
     addi $a1, $a1, 1
     jal rstrcopy
     lw $ra, 0($sp)
     addi $sp, $sp, 4
     jr $ra
Else: sb $zero, 0($a0)
     jr $ra
```

Both copied "hello world" correctly in the simulator. Characters are bytes, hence `lbu`/`sb`, and the pointers advance by 1, not 4.

Past Exam Question (2022–23 CSE 305 Q7(c)) 8)

Write MIPS code for `int f(int a, int b, int c, int d) { return func(func(a,b), c+d); }`. Use `$t0, $t1, ...` (lowest numbers first) for temporaries.

Solution. `a,b,c,d` arrive in `$a0`-`$a3`. The first call `func(a,b)` needs `a,b` already in `$a0,$a1`. After it returns, its result must become the *first* argument and $c + d$ the second; but a call may destroy temporaries, so $c + d$ (computed first, in `$t0`) is saved on the stack across the first call, together with `$ra`.

```
func: add $v0, $a0, $a1 # stand-in for func (for testing only)
     jr $ra
f: addi $sp, $sp, -8
     sw $ra, 4($sp) # save return address
     add $t0, $a2, $a3 # t0 = c + d (compute before the call)
     sw $t0, 0($sp) # save c+d across the call
     jal func # v0 = func(a, b); a,b are already in $a0,$a1
     add $a0, $v0, $zero # first argument = func(a,b)
     lw $a1, 0($sp) # second argument = c + d
     lw $ra, 4($sp)
     addi $sp, $sp, 8
     j func # tail call: return func(...) directly
```

The last call is a tail call: after restoring `$ra` and `$sp`, `j func` makes `func` return directly to `f`'s caller. (The first two lines of the listing define a stand-in `func` used only to test the code: $f(1,2,3,4) = \text{func}(\text{func}(1,2),7) = 10$.)

7.7 T7: Synchronization

Past Exam Question (2016–17 Q6(b) 10; 2015–16 Q6(c) 8; 2014–15 Q6(b) 5; 2022–23 CSE 305 Q6(b) 7; 2013–14 Q7(a) 8)

What is a data race? What support does MIPS provide for synchronization? Explain with an example. (2013–14: write MIPS code to do `A[0] = X` atomically; base address of `A` in `$s1`, `X` in `$s2`.)

Solution. **Data race:** two processors access the same memory location, at least one writing, with no ordering between them, so the result depends on the timing. **Support:** hardware must provide an atomic read–modify–write; MIPS provides the pair `ll` (load linked) and `sc` (store conditional). `sc` writes the register to memory only if the location has not been modified since the `ll`; it sets the register to 1 on success and 0 on failure (Chapter 6).

Example — atomic swap / lock acquire (exchange `$s4` with `0($s1)`):

```
try: add $t0, $zero, $s4 # copy exchange value
     ll $t1, 0($s1) # load linked
     sc $t0, 0($s1) # store conditional
     beq $t0, $zero, try # store failed: retry
     add $s4, $zero, $t1 # old value to s4
```

2013–14 (atomic `A[0] = X`):

```
try: add $t0, $zero, $s2 # t0 = X
     ll $t1, 0($s1) # link A[0] (old value in t1, discarded)
     sc $t0, 0($s1) # A[0] = X if nobody wrote A[0] meanwhile
     beq $t0, $zero, try # sc returned 0: retry
```

7.8 T8: Translation, linking, loading, dynamic linking and Java

Past Exam Question (2019–20 Q1(c) 8; 2022–23 CSE 209 Q12(a),(b) 5+10; 2022–23 CSE 305 Q7(d) 15)

What are the advantages of dynamically linked libraries (DLLs)? What is the role of JIT in Java execution? What is the difference between static and dynamic linking? Explain lazy linkage with a diagram.

Solution. **Static vs dynamic linking.** Static: the linker copies all the library routines a program (transitively) uses into its executable image before it runs. Dynamic: a library routine is linked and loaded only when it is first called.

Advantages of dynamic linking (DLL): (1) the executable is smaller (no image bloat from linking every library in advance); (2) only routines actually called are loaded; (3) one copy of a library in memory can be shared by many programs; (4) programs automatically pick up new library versions (bug fixes) without relinking. Cost: slower first call, and the library code must be relocatable.

Lazy linkage (Figure 7.2): the call site calls through an indirection-table entry that initially points to a *stub*. First call: the stub loads the routine's ID and jumps to the dynamic linker/loader, which locates and maps the routine and *patches the table entry* to point at it. Later calls: the entry points directly to the routine, so there is no further linker overhead. **Role of JIT:** Java source is compiled to portable bytecodes executed by the JVM. Pure interpretation is slow; the Just-In-Time compiler translates "hot" (frequently executed) methods into native machine code

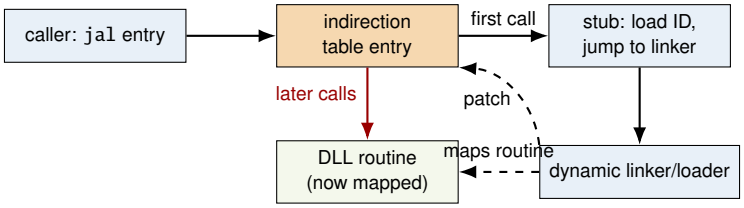


Figure 7.2: Lazy linkage: solid black path = first call; red = subsequent calls after the entry has been patched.

at run time, so repeated executions run at compiled speed, giving portability with near-native performance.

Past Exam Question (2013–14 Q6(a), 2nd part)

What are the steps for starting a Java application?

Solution. (1) The Java compiler translates the source into *class files* of bytecodes (the simple portable instruction set of the JVM). (2) The Java Virtual Machine loads the class files (and the library routines, which are in machine language). (3) The JVM *interprets* the bytecodes. (4) For frequently executed ("hot") methods the JIT compiler compiles the bytecodes into native code for the host, which is then executed directly [MIPS slide 65].

Past Exam Question (2022–23 CSE 305 Q5(c) 14)

Object file A has text size `0x150` and data size `0x40`; its text contains `lw $a0, 0($gp)` (referring to data item X) and `jal 0` (a call to B). Object file B has text size `0x350`, data size `0x50`; it contains `sw $a1, 0($gp)` (data item Y) and `jal 0` (a call to A). The linked text starts at `0x00400000` (A first, then B), data at `0x10000000`, `$gp=0x10008000`. Find the final values of the patched instruction fields.

Solution. **Layout.** Text: A at `0x00400000`, B at `0x00400000+0x150=0x00400150`. Data: A at `0x10000000`, B at `0x10000000+0x40=0x10000040`. **Assumption:** X is the first item of A's data and Y the first item of B's data (offset 0 in their data segments) — the question gives no other offsets.

- A: `lw $a0,0($gp)`: X at `0x10000000`; offset = `0x10000000` − `0x10008000` = $-$`0x8000` ⇒ `lw $a0, -32768($gp)` (16-bit field `0x8000`).
- A: `jal 0`: B is at `0x00400150`; field = `0x00400150`/4 = `0x00100054` ⇒ `jal 0x0c100054`.
- B: `sw $a1,0($gp)`: Y at `0x10000040`; offset = `0x10000040` − `0x10008000` = $-$`0x7fc0` ⇒ field `0x8040` (i.e. `sw $a1, -32704($gp)`).
- B: `jal 0`: A at `0x00400000`; field = `0x00100000` ⇒ `jal 0x0c100000`.

The linker found these using the symbol tables (where X, Y, A, B are defined) and the relocation information (which instructions refer to them). Note that data references use `$gp`-relative addressing, so only the offset changes, while procedure calls use the absolute word address of the callee.

PART II

The Processor, Part 2: Pipelining

CHAPTER 8

Pipelining: The Big Idea

Chapter goal

This chapter explains *why* high-performance processors overlap the execution of instructions (*pipelining*), what the five stages of the MIPS pipeline are, how much speed-up pipelining really gives (and why it is not exactly 5×), and why the MIPS instruction set suits pipelining. *Lecture source: Processor slides 1–32 (§4.5).*

8.1 What you need from the single-cycle processor

The lecture assumes you have seen the *single-cycle* MIPS datapath (Processor Part 1). Here is only what is needed to follow this book.

A **datapath** is the collection of hardware units that process data and addresses inside the CPU. The main units are:

- the **program counter (PC)**: a register holding the address of the current instruction;
- the **instruction memory**: given the PC, outputs the 32-bit instruction;
- the **register file**: the 32 registers, with two read ports and one write port;
- the **ALU** (arithmetic logic unit): adds, subtracts, ANDs, ORs, compares;
- the **data memory**: read by `lw`, written by `sw`;
- small adders (for PC+4 and branch targets), **multiplexers** (MUXes, which choose one of several inputs), and a **sign-extension** unit.

Executing any instruction involves up to five jobs, in this order [Processor slide 3]:

1. **IF** — *instruction fetch*: read the instruction from memory;
2. **ID** — *instruction decode and register read*: work out what the instruction is and read its source registers;
3. **EX** — *execute*: perform the operation, or compute a memory address (or a branch target/-condition);
4. **MEM** — *memory access*: read or write data memory (only `lw`/`sw`);
5. **WB** — *write back*: write the result into a register.

In a **single-cycle** implementation, one *clock cycle* (the fixed time between two ticks of the processor's clock) is long enough to do all five jobs of any instruction. The clock period must fit the *slowest* instruction, and everything else waits.

8.1.1 How slow is it? The lecture's timing assumptions [Processor slide 5]

Suppose register read or write takes 100 ps and the other units take 200 ps (1 ps $= 10^{-12}$ s). Then the time each instruction really needs is:

Instruction	Fetch	Reg read	ALU	Memory	Reg write	Total
`lw`	200	100	200	200	100	**800 ps**
`sw`	200	100	200	200	—	700 ps
R-format	200	100	200	—	100	600 ps
`beq`	200	100	200	—	—	500 ps

The single-cycle clock period must be $T_c = 800$ ps (the `lw` time), even though a `beq` needs only 500 ps. This is the first reason single-cycle designs are inefficient: *every instruction takes as long as the slowest one*. Also each hardware unit sits idle most of the time (the ALU is idle while the instruction is fetched, the memory is idle while the ALU works, …) and units like the adders cannot be shared.

8.2 The pipelining analogy [Processor slide 2]

Imagine a laundry with four steps — wash, dry, fold, put away — each taking 30 minutes, and four loads of washing, A–D. If you finish one load completely before starting the next, each load takes 2 hours and the four loads take $4 \times 2 = 8$ hours. But the washer is free as soon as load A moves to the dryer! Start washing load B immediately. After the first 2 hours, a finished load appears every 30 minutes, and four loads take $2 + 3 \times 0.5 = 3.5$ hours. The speed-up is $8/3.5 \approx 2.3$, approaching 4 if there were many loads.

Two things to notice:

- **Parallelism improves performance:** different loads are in different steps at the same time.
- **Latency does not improve.** One load still takes 2 hours from start to finish; the benefit is *throughput* — the number of loads finished per hour.

8.3 The five-stage MIPS pipeline [Processor slide 3–4]

A **pipelined** processor does the same with instructions: each of the five jobs above is a **pipeline stage**, with its own hardware. While one instruction is in EX, the next is in ID and the one after that in IF. Each stage takes exactly one clock cycle, and *all* instructions go through *all five* stages in the same order (even if a stage does nothing for them) — this keeps the control simple.

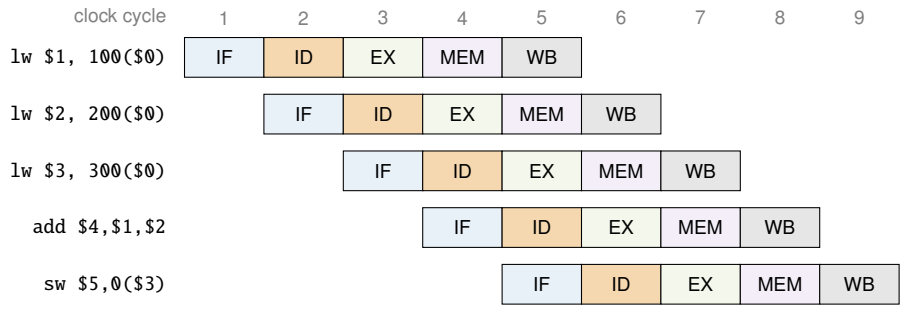


Figure 8.1: A pipeline diagram: time runs left to right, instructions top to bottom. Once the pipeline is full (cycle 5) five instructions are in progress simultaneously, one in each stage, and one instruction completes every cycle.

8.3.1 Pipeline performance [Processor slide 5–7]

In the pipelined processor the clock period must be as long as the *slowest stage*, because all stages advance together at each clock tick. With our stage times (200, 100, 200, 200, 100 ps) the slowest is 200 ps, so $T_c = 200$ ps: even the 100 ps register stages get a 200 ps slot. A new instruction *starts* every 200 ps, a quarter of the single-cycle time per instruction.

Worked Example: Three `lw` instructions [Processor slide 6–8]

Single cycle: each instruction takes 800 ps, so 3 instructions take $3 \times 800 = 2400$ ps.
Pipelined: the first instruction needs 5 cycles to get through; each following instruction finishes one cycle later, so $5 + (3 - 1) = 7$ cycles $= 7 \times 200 = 1400$ ps.
Speed-up $= 2400/1400 = 1.714$.

In general, with k stages, n instructions and cycle time T_c:

$$T_{\text{pipelined}} = (k + n - 1)\, T_c, \qquad T_{\text{single-cycle}} = n\, T_{\text{sc}}.$$

Worked Example: One million more instructions

$T_{\text{pipelined}} = 1{,}000{,}000 \times 200 + 1400 = 200{,}001{,}400$ ps and $T_{\text{single}} = 1{,}000{,}000 \times 800 + 2400 = 800{,}002{,}400$ ps, so the speed-up $= 800{,}002{,}400/200{,}001{,}400 = 3.99998 \approx \mathbf{4}$ (I verified the arithmetic with a script).

The speed-up with 3 instructions is small because the cost of *filling* the pipeline (the first four cycles in which fewer than five stages are busy) matters; for long programs it vanishes and the speed-up approaches the ratio $800/200 = 4$.

Definition: Ideal pipeline speed-up [Processor slide 7]

If all stages are *balanced* (take the same time),

$$\text{time between instructions}_{\text{pipelined}} = \frac{\text{time between instructions}_{\text{non-pipelined}}}{\text{number of stages}}.$$

In our example this would be $800/5 = 160$ ps. If the stages are *not* balanced the speed-up is less: here the slowest stage is 200 ps instead of 160, giving 4 instead of 5.

Common mistake: Throughput versus latency

Pipelining *does not shorten the time of one instruction*. In fact a single `lw` takes $5 \times 200 = 1000$ ps in the pipeline, longer than 800 ps in the single-cycle machine. The gain is that instructions finish more often: it is a speed-up in **throughput**.

8.4 Why MIPS suits pipelining [Processor slide 9]

The MIPS ISA was designed with pipelining in mind:

- **All instructions are 32 bits.** Easier to fetch and decode in one cycle (compare x86 with 1 to 17 byte instructions: hardware must first find out how long the instruction is).
- **Few, regular instruction formats.** The registers can be read in the decode stage at the same time as the instruction is decoded, since register fields are always in the same place.

- **Load/store addressing.** Only `lw/sw` access memory, so we can calculate the address in the third stage and access memory in the fourth. (If ALU instructions could take a memory operand, we would need an address stage *and* an execute stage after memory.)
- **Alignment of memory operands.** A memory access completes in one cycle (an unaligned word would straddle two memory words and require two accesses).

8.5 Hazards: what can go wrong [Processor slide 10–11]

If instructions were independent, the pipeline would complete one every cycle. A **hazard** is a situation that prevents starting the next instruction in the next clock cycle [Processor slide 10]. There are three kinds; the following chapters treat each in detail.

Structural hazard a required hardware resource is busy (two instructions need the same unit in the same cycle);

Data hazard an instruction needs data that a previous, unfinished instruction has not yet produced;

Control hazard the decision about which instruction to fetch next depends on a previous instruction (a branch) whose outcome is not yet known.

Hazard	Example	Typical remedy
Structural	one memory used for both instruction fetch and `lw` data	separate instruction and data memories (or caches)
Data	`add $s0,..` followed by `sub $t2,$s0,..`	forwarding; stall for load-use; code scheduling
Control	`beq` followed by the next instruction	early branch decision, prediction, flush

Chapter 9 introduces them with examples.

8.6 Summary: the big picture [Processor slide 32]

- Pipelining improves performance by increasing instruction **throughput**: it executes multiple instructions in parallel, each in a different stage.
- Each instruction has the same latency (and in practice slightly more than in single-cycle).
- The clock period equals the slowest stage; the ideal speed-up equals the number of stages, but unbalanced stages, filling the pipeline and hazards reduce it.
- The pipeline is subject to hazards: structural, data and control.
- Instruction-set design affects the complexity of the pipeline implementation.

Lecture source

Processor slides 1–11 and 32 (§4.5 An Overview of Pipelining). The five stages are drawn on slide 3–4; the performance example on slides 5–8.

CHAPTER 9

Structural and Data Hazards, Forwarding and Code Scheduling

Chapter goal

A pipeline works beautifully only if instructions do not interfere with each other. This chapter studies two of the three kinds of hazard in detail: *structural* hazards (two instructions want the same hardware) and *data* hazards (one instruction needs a value another has not yet produced). You will learn the two fixes — *forwarding* and *stalling* — and how the compiler can reduce stalls by *reordering code*. The hardware that implements them is in Chapter 11. *Lecture source: Processor slides 10–24.*

9.1 Reading a pipeline diagram

Throughout, we use diagrams like Figure 9.2. Each row is an instruction in program order, each column a clock cycle; a box shows which stage the instruction occupies in that cycle. *Stall* boxes (dashed, red) are cycles in which the instruction waits and does no useful work; they are also called **bubbles**. Red arrows show a value being *forwarded* (copied directly) from one instruction to a later one.

9.2 Structural hazards [Processor slide 11]

Definition: Structural hazard

A structural hazard occurs when two instructions need the same hardware resource in the same clock cycle, so one of them must wait.

In the MIPS pipeline, suppose there were only *one* memory used for both instructions and data. A `lw` or `sw` needs the memory in its MEM stage (cycle 4 for the first instruction), but at the same moment the pipeline wants to *fetch* the fourth instruction (its IF stage) from the same memory. One of them must wait, which creates a pipeline bubble. **Remedy.** Provide separate

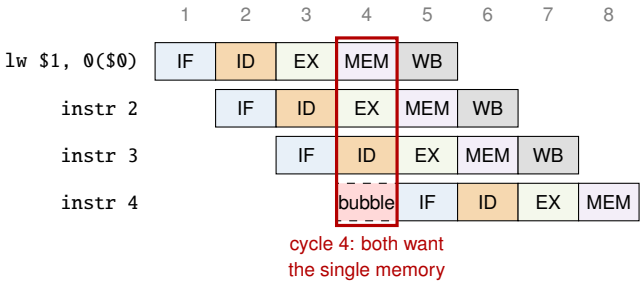


Figure 9.1: A structural hazard with a single memory: in cycle 4 the `lw` reads data (MEM) while instruction 4 wants to be fetched (IF). Instruction 4's fetch is delayed one cycle.

instruction and data memories (in a real machine, separate instruction and data *caches*) so that the IF and MEM stages never compete. The register file is a second shared resource: it is read in ID and written in WB in the same cycle. This is resolved by writing in the *first half* of the clock cycle and reading in the *second half*, so a read in the same cycle returns the newly written value [Processor slide 55]. The datapath also needs more than one adder (PC+4, branch target) and ALU, so that all stages can work together.

9.3 Data hazards [Processor slide 12–16]

Definition: Data hazard

A *data hazard* occurs when an instruction depends on the completion of a data access (reading or writing a register) by a previous instruction that is still in the pipeline.

The most common kind is a **read-after-write** (RAW) dependence: an instruction *reads* a register that an earlier instruction *writes*. Consider [Processor slide 12]

```
add $s0, $t0, $t1 # writes $s0 in its WB stage
sub $t2, $s0, $t3 # needs $s0 in its ID stage (register read)
```

`$s0` is *written* at the end of `add`'s WB stage but *needed* by `sub` at its ID stage. In the pipeline that is too early: if `sub` proceeds, it reads the old value.

9.3.1 Solution 1: stall

The simple but slow fix is to *stall*: hold `sub` in ID until `add` has written the register. Because the register file writes in the first half of a cycle and reads in the second, `sub` can read in the same cycle as `add`'s WB. This costs 2 stall cycles (Figure 9.2, upper).

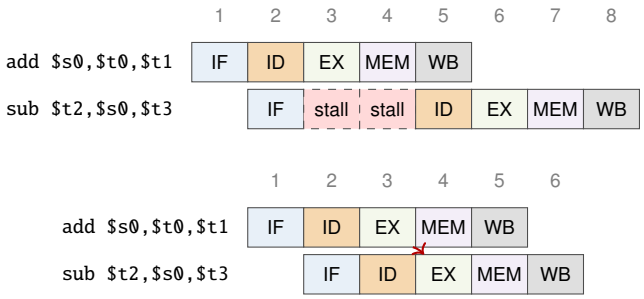


Figure 9.2: Upper: without forwarding `sub` waits 2 cycles (8 cycles in total). Lower: with forwarding the value is passed from the end of `add`'s EX stage directly to `sub`'s EX stage; no stall (6 cycles).

9.3.2 Solution 2: forwarding (bypassing) [Processor slide 16]

Definition: Forwarding

Forwarding (also *bypassing*) means using a result *as soon as it is computed*, without waiting for it to be written to the register file. It requires extra connections in the datapath that carry results from later pipeline stages back to the inputs of the ALU.

In the example the sum is available at the end of `add`'s EX stage (the ALU output), and `sub` needs it at the start of its own EX stage, one cycle later. So we simply route the ALU result through the *pipeline register* (Chapter 10) back to the ALU input. No cycle is lost.

9.3.3 Load-use data hazard [Processor slide 17–21]

Forwarding is not always enough. A result can only be forwarded if it exists by the time it is needed.

- For an **R-type** instruction the value is ready at the *end of EX*.
- For a **load** (`lw`) the value comes out of the data memory, so it is ready only at the *end of MEM*, one stage later.

If the instruction immediately after a `lw` uses the loaded register in its EX stage, it needs the value *before* the load has produced it: "you can't forward backward in time".

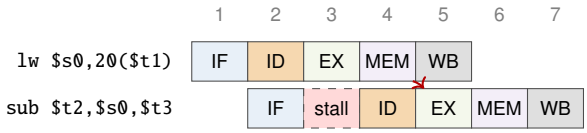


Figure 9.3: Load-use hazard: `lw` produces `$s0` at the end of cycle 4 (MEM); `sub` would need it at the start of cycle 4. One stall cycle is unavoidable even with forwarding.

Definition: Load-use data hazard

A hazard in which an instruction uses the result of the *immediately preceding load*. With forwarding, it still needs **exactly one stall cycle**.

If there is at least one other instruction between the load and its user, no stall is needed: the value is then ready (and forwarded from the MEM/WB pipeline register) by the time the user needs it — this is exactly the gap that code scheduling tries to fill.

9.4 Code scheduling to avoid stalls [Processor slide 22–24]

Since the hazard depends on instruction *order*, the compiler can often **schedule** (reorder) the code, moving an independent instruction into the position that would otherwise be a bubble. The reordering must not change the program's meaning.

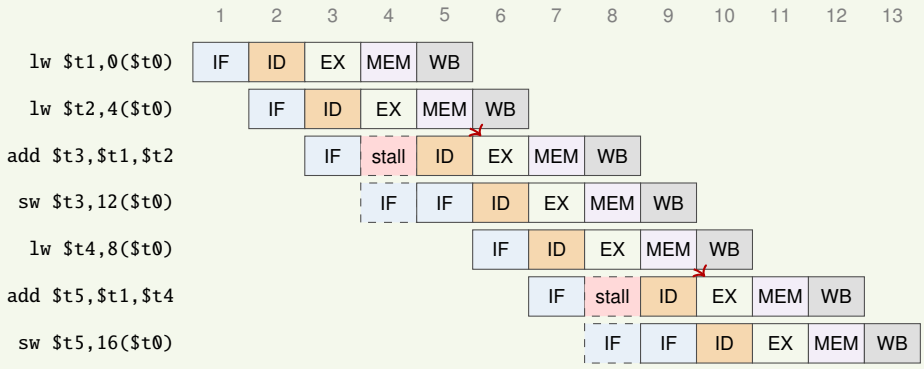
Worked Example: Slide example: `A = B + E; C = B + F;`

Memory addresses relative to `$t0`: `B` at 0, `E` at 4, `F` at 8, `A` at 12, `C` at 16.

```
lw $t1, 0($t0) # B
lw $t2, 4($t0) # E
add $t3, $t1, $t2 # A = B + E (uses $t2 loaded just before: load-use stall)
sw $t3, 12($t0)
lw $t4, 8($t0) # F
add $t5, $t1, $t4 # C = B + F (uses $t4 loaded just before: load-use stall)
sw $t5, 16($t0)
```

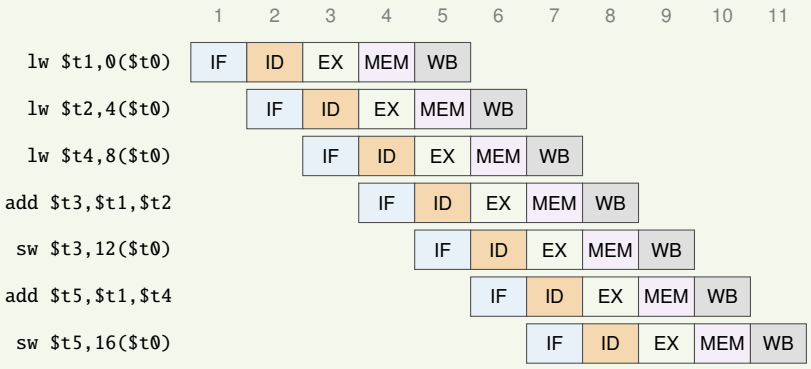
Counting cycles. 7 instructions fill 5 stages: the last one finishes in cycle $7 + 4 = 11$ if there are no stalls. Each load-use hazard adds 1 cycle; there are two (`lw $t2`→`add $t3`,

`lw $t4`→`add $t5`), so the code needs $11 + 2 = \mathbf{13}$ cycles. I confirmed this count and the diagram with a pipeline model.



After scheduling. Move `lw $t4,8($t0)` up, before the first add. Now between each load and its user there is at least one other instruction:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
lw $t4, 8($t0) # moved up: fills the gap
add $t3, $t1, $t2 # $t2 loaded 1 instruction ago: no stall
sw $t3, 12($t0)
add $t5, $t1, $t4 # $t4 loaded 2 instructions ago: no stall
sw $t5, 16($t0)
```



Total: $7 + 4 = \mathbf{11}$ cycles, with no stalls.

Common mistake: Legal reordering

You may move an instruction earlier only if it does not depend on any instruction it jumps over, and nothing it jumps over depends on it. Here `lw $t4` writes a register nobody else uses, so moving it is safe. The compiler must know the pipeline structure to do this, which is why the lecture says "stalls reduce performance but are required to get correct results; the compiler can arrange code to avoid hazards and stalls" [Processor slide 76].

9.5 Summary

- Structural hazard: shared resource; fixed by duplicating resources (separate instruction and data memories).

- Data hazard (RAW): without forwarding a dependent instruction waits until the producer's WB (2 stalls for adjacent instructions); with forwarding no stall for ALU results; one stall for load-use.
- Compiler scheduling can fill load delay slots and reach the no-stall count: $n + 4$ cycles for n instructions.

Lecture source

Processor slides 10–24 (§4.5 Hazards, Forwarding, Load-Use Hazard, Code Scheduling to Avoid Stalls).

CHAPTER 10

The Pipelined Datapath and Its Control

Chapter goal

Chapter 8 described pipelining as an idea. Here we see how the single-cycle datapath is *cut into five stages* by inserting registers, how an instruction's information travels from stage to stage, and how the control signals are generated once and carried along with the instruction. *Lecture source: Processor slides 33–52 (§4.6).*

10.1 From one long cycle to five short ones

Take the single-cycle datapath and draw it left to right in the order that an instruction uses it: instruction memory (IF), register file (ID), ALU (EX), data memory (MEM), then back to the register file (WB). The picture on slide 33 (Figure 10.1) shows this. Two flows go *right to left*, against the direction of the pipeline, and they are the source of the two kinds of trouble that need special treatment [Processor slide 33]:

- the **write-back** flow (from WB back to the register file) — source of *data hazards*: a value is written after later instructions have already read;
- the **PC-update** flow (from the MEM stage's branch decision back to the PC multiplexer) — source of *control hazards*: the next instruction was already fetched before we knew whether to branch.

10.2 Pipeline registers

In the single-cycle machine a signal such as the fetched instruction is held on its wires for the whole (long) cycle. In a pipeline, every stage works on a *different* instruction in each cycle, so at each clock tick the results of a stage must be *saved* and handed to the next stage while the stage starts on a new instruction. For this we insert registers between the stages: **pipeline registers**, named after the stages they separate: **IF/ID**, **ID/EX**, **EX/MEM**, **MEM/WB** [Processor slide 34]. They are written at the end of each clock cycle (on the clock edge) and hold everything the later stages will need for *that* instruction.

Worked Example: Size of the pipeline registers, slide 34

Register	Contents	Size
IF/ID	32-bit instruction + 32-bit PC+4	$32 + 32 = 64$
ID/EX	PC+4, Read data 1, Read data 2, sign-extended immediate: four 32-bit values	$32 \times 4 = 128$
EX/MEM	branch target address, ALU result, Read data 2 (data to store): three 32-bit values, plus the 1-bit Zero flag	$32 \times 3 + 1 = 97$
MEM/WB	data read from memory + ALU result	$32 + 32 = 64$

The register-number fields (Rs, Rt, Rd, 5 bits each) and the control bits must also be carried; the lecture's counts leave them out ("we may need to update the sizes as we proceed"

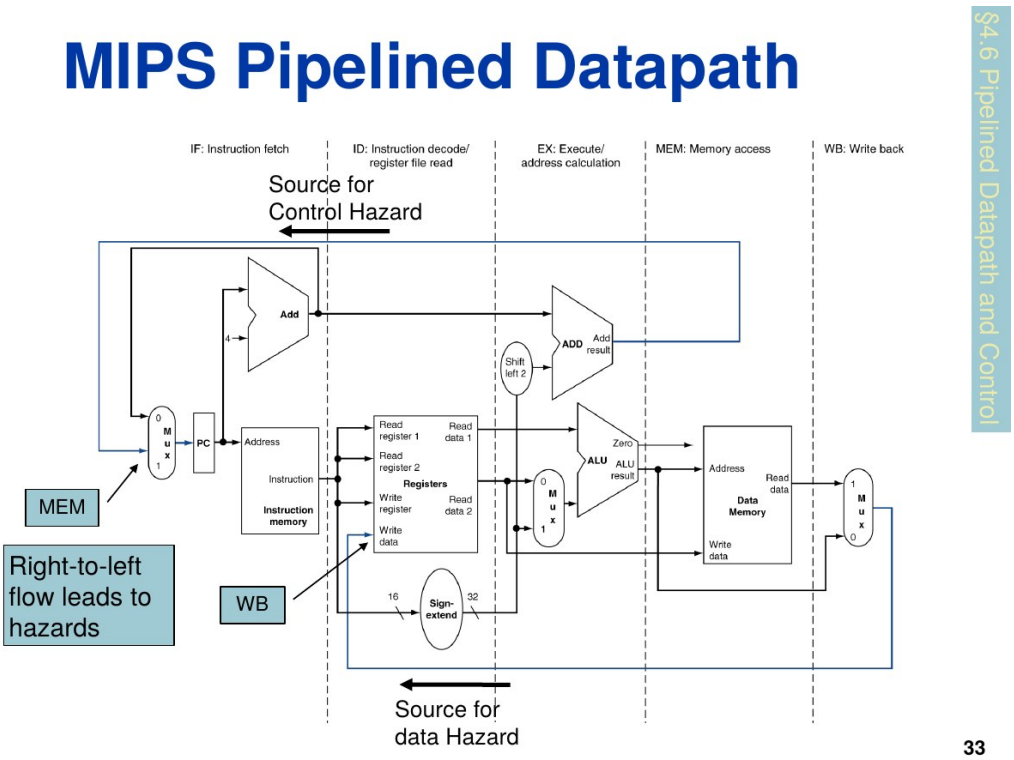


Figure 10.1: The MIPS pipelined datapath (slide 33). The two right-to-left flows (write-back and branch/PC selection) cause data and control hazards respectively.

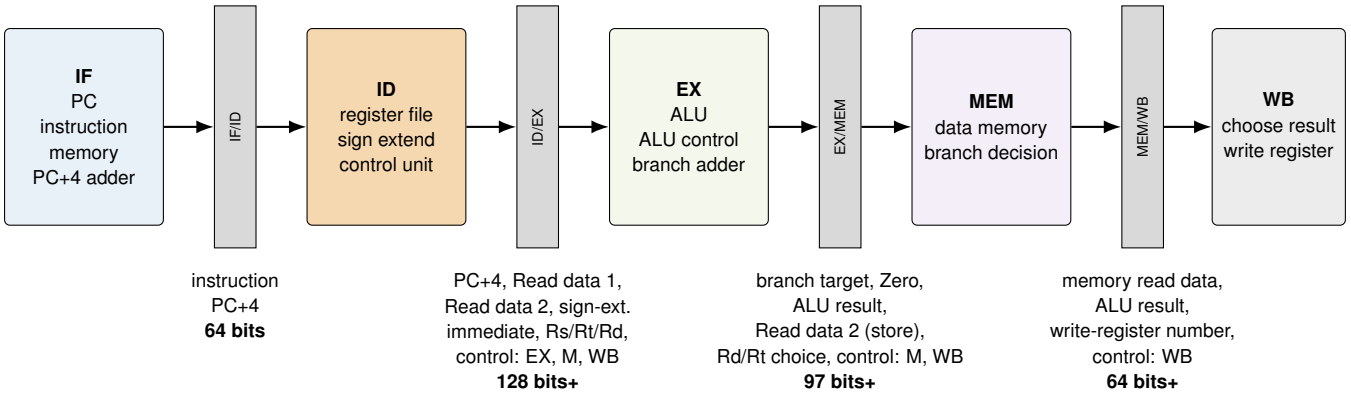


Figure 10.2: Five stages separated by four pipeline registers, and what each register holds. The bit counts are those of the lecture (slide 34); "+" means: plus the 5-bit register numbers and the control bits.

[Processor slide 34]).

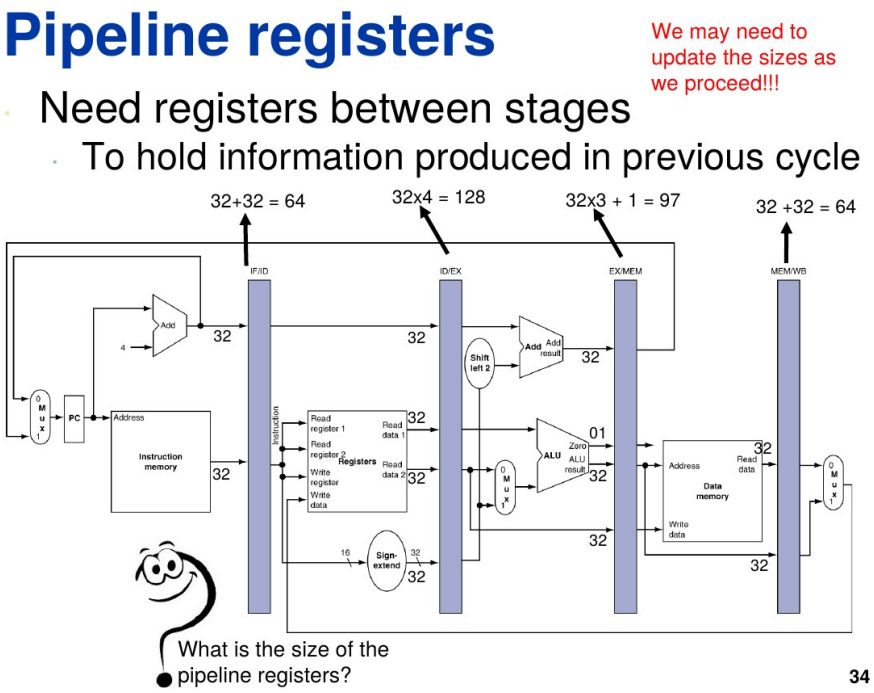


Figure 10.3: Pipeline registers and their sizes (slide 34).

10.3 How an instruction moves through the pipeline

We follow the load `lw $t0, 32($s3)` stage by stage (slides 36–45).

1. **IF.** The PC addresses the instruction memory; the instruction and PC+4 are written into IF/ID at the end of the cycle.
2. **ID.** The instruction in IF/ID is decoded; the register file reads `$s3` (Read data 1); the 16-bit offset 32 is sign-extended to 32 bits; the control unit generates the control signals. Everything is written into ID/EX.
3. **EX.** The ALU adds Read data 1 and the sign-extended offset to form the memory address; the result goes into EX/MEM.
4. **MEM.** The data memory is read at that address; the word goes into MEM/WB.
5. **WB.** A MUX chooses the memory data (rather than the ALU result) and the register file writes it to `$t0`.

A `sw` is identical up to EX, except that Read data 2 (the value to store) is also carried to EX/MEM; in MEM the data memory is *written*; and in WB *nothing happens* (a store writes no register) [Processor slide 41–43].

Common mistake: The "wrong register number" bug of the first-draft datapath [Processor slide 44–45]

In the first version of the datapath the register file's *write register* number simply came from the instruction currently in the *ID* stage. But by the time a `lw` reaches WB (four cycles later),

a different instruction is in ID, so the load's result would be written to the register named by the *wrong* instruction! The fix: the destination register number (Rt for `lw`, or Rd for R-type, chosen by the RegDst MUX in EX) is passed along through ID/EX, EX/MEM and MEM/WB, and it is that stored number which arrives back at the register file in WB. This is why every pipeline register has a field for the destination register number.

10.4 Two ways of drawing pipeline behaviour [Processor slide 35, 46–49]

- **Multi-clock-cycle diagram**: the graph over time (Figure 9.2 style), one row per instruction. The "resource usage" form draws icons for IM, Reg, ALU, DM, Reg; the "traditional" form writes the stage names. Used to show hazards and stalls.
- **Single-clock-cycle diagram**: the datapath drawn once, with the instruction that is in each stage *in that particular cycle* written under it. Used to show what the hardware holds at a given instant, e.g. the contents of the pipeline registers.

In cycle 5 of the five-instruction sequence of Figure 10.4, the instruction in WB is the first one, in MEM the second, . . . , in IF the fifth.

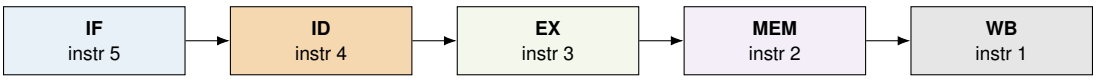


Figure 10.4: A single-clock-cycle view of the pipeline in cycle 5 (compare the pipeline diagram in Chapter 8).

10.5 Pipelined control [Processor slide 50–52]

In the single-cycle machine the control unit looks at the opcode and sets the control signals for the whole instruction at once. In the pipeline, the signals are *still derived from the instruction*, in the ID stage, but different signals are used *later* in different stages, in different clock cycles. So the control unit's outputs are carried along in the pipeline registers, grouped by the stage that uses them:

Stage that uses them	Signals
IF, ID	none special (the instruction is only being fetched/decoded)
EX	`RegDst` (choose Rd or Rt as destination), `ALUOp` (two bits; with the funct field tells the ALU control what operation), `ALUSrc` (second ALU input: Read data 2 or sign-extended immediate)
MEM	`Branch`, `MemRead`, `MemWrite`
WB	`MemtoReg` (write ALU result or memory data), `RegWrite` (write the register file)

After the control unit produces all nine signals in ID, they are stored in ID/EX. The EX stage uses its four and passes the M and WB groups on to EX/MEM; the MEM stage uses its three and passes the two WB signals to MEM/WB; the WB stage uses them. (The slide notes that this is confusing: `RegWrite`, which controls the register file in ID, is actually generated in ID but *applied* in the WB stage, four cycles later [Processor slide 50].)

Here the ALUOp column is two bits (ALUOp1, ALUOp0): 00 = add (used by `lw/sw` for the address), 01 = subtract (`beq` compares by subtracting and checking Zero), 10 = look at the funct field (R-type). Reading the table: for a `lw` we need the destination to be Rt (RegDst=0),

Table 10.1: Control signal values (the same as in the single-cycle design; X = don't care).

Instruction	EX			MEM			WB	
	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R-format (add...)	1	1 0	0	0	0	0	1	0
`lw`	0	0 0	1	0	1	0	1	1
`sw`	X	0 0	1	0	0	1	0	X
`beq`	X	0 1	0	1	0	0	0	X

the ALU's second input to be the offset (ALUSrc=1), to read memory (MemRead=1), to write the register file (RegWrite=1) with the memory data (MemtoReg=1); for a `beq` the registers are *compared* (ALUSrc=0, ALUOp=01) and the Branch signal (together with Zero) decides whether to change the PC.

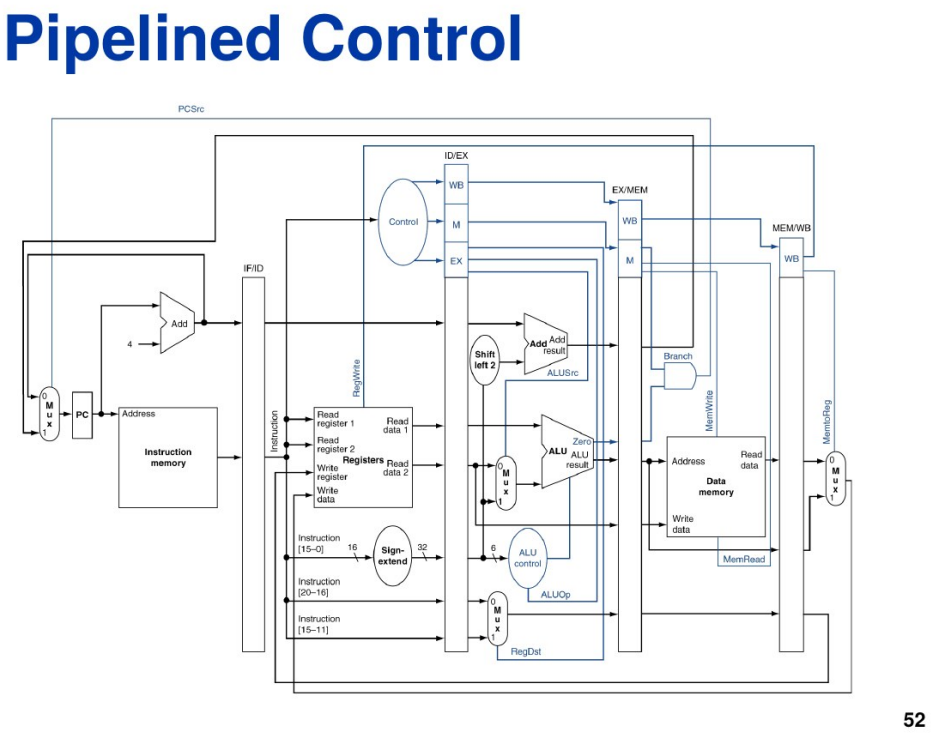


Figure 10.5: The pipelined datapath with control (slide 52): the control signals are generated in ID, stored in ID/EX, and passed down the pipeline registers.

Worked Example: What is in the ID/EX register for `lw $t0,32($s3)`?

Control: EX group (RegDst=0, ALUOp=00, ALUSrc=1), M group (Branch=0, MemRead=1, MemWrite=0), WB group (RegWrite=1, MemtoReg=1). Data: Read data 1 = contents of `$s3`; Read data 2 = contents of `$t0` (read but not used); sign-extended immediate = 32; PC+4; and the register numbers Rs=19, Rt=8, Rd=bits 15–11 of the instruction (the high bits of the offset, here 0).

10.6 Summary

- Pipelining = insert registers between the five stages: IF/ID, ID/EX, EX/MEM, MEM/WB. The slide's sizes are 64, 128, 97 and 64 bits for the data (plus register numbers and control).

- The destination register number is carried to WB so the result is written to the right register.
- Control signals are computed once in ID and travel along in the pipeline registers; each stage consumes its own group (EX: RegDst, ALUOp, ALUSrc; MEM: Branch, MemRead, MemWrite; WB: MemtoReg, RegWrite).

Lecture source

Processor slides 33–52 (§4.6 Pipelined Datapath and Control).

CHAPTER 11

Data Hazards: Forwarding Unit and Hazard Detection Unit

Chapter goal

Chapter 9 introduced forwarding and load-use stalls as ideas. Now we build the hardware that implements them: a *forwarding unit* that detects when an ALU input should be taken from a later pipeline register, and a *hazard detection unit* that stalls the pipeline for a load-use hazard. You must be able to write the exact *conditions* for both. *Lecture source: Processor slides 53–76 (§4.7).*

11.1 Which dependences are hazards?

Consider the sequence of the lecture [Processor slide 53]:

```
sub $2, $1, $3 # writes $2
and $12, $2, $5 # reads $2 1st instruction after sub
or $13, $6, $2 # reads $2 2nd instruction after sub
add $14, $2, $2 # reads $2 3rd instruction after sub
sw $15, 100($2) # reads $2 4th instruction after sub
```

All four later instructions read `$2`, which `sub` writes in its WB stage (cycle 5). Which of them would get the *wrong* (old) value without any help? Stage by stage (the register file is read in the second half of ID and written in the first half of WB):

- `and` reads in cycle 3, before cycle 5: **hazard**.
- `or` reads in cycle 4, before cycle 5: **hazard**.
- `add` reads in cycle 5: the write happens in the first half of cycle 5, the read in the second half, so it gets the *new* value: **no hazard**.
- `sw` reads in cycle 6: no hazard.

So only the instructions within *two* places after the writer are potential hazards. Both are solved by forwarding: the value of `$2` exists at the end of `sub`'s EX stage (cycle 3), i.e. in the EX/MEM pipeline register during cycle 4, and in the MEM/WB register during cycle 5.

11.2 Detecting the need to forward [Processor slide 56–57]

The hardware must *notice* when a source register of the instruction in EX equals the destination register of an instruction ahead of it. To do that, the register numbers are passed along the pipeline. We write `ID/EX.RegisterRs` for the 5-bit number of register `Rs` stored in the ID/EX pipeline register, and similarly for the others. The ALU operands in the EX stage come from registers named `ID/EX.RegisterRs` and `ID/EX.RegisterRt`. A data hazard that needs forwarding exists when

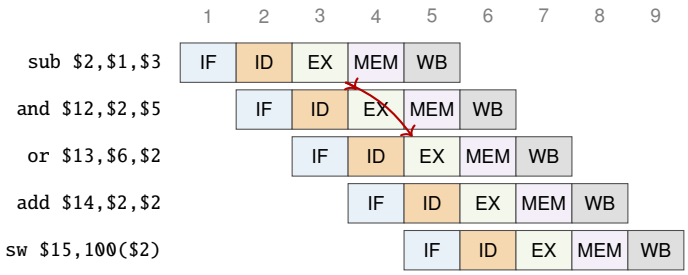


Figure 11.1: With forwarding the five-instruction sequence runs without stalls (9 cycles). `and` gets `$2` from the EX/MEM register, `or` gets it from the MEM/WB register. (Without forwarding, `and` would stall 2 cycles: 11 cycles.)

1a. `EX/MEM.RegisterRd = ID/EX.RegisterRs`	forward from the **EX/MEM** pipeline register
1b. `EX/MEM.RegisterRd = ID/EX.RegisterRt`	
2a. `MEM/WB.RegisterRd = ID/EX.RegisterRs`	forward from the **MEM/WB** pipeline register
2b. `MEM/WB.RegisterRd = ID/EX.RegisterRt`	

The number "1" = the producer is one instruction ahead (it is now in the MEM stage, so its result is in EX/MEM); "2" = two instructions ahead (result in MEM/WB); "a" = first ALU operand (Rs); "b" = second (Rt). Notice that in the *sub/and/or* example, `and` has condition 1a (`EX/MEM.RegisterRd` = 2 = `ID/EX.RegisterRs`) and `or` has condition 2b (`MEM/WB.RegisterRd` = 2 = `ID/EX.RegisterRt`).

Two extra checks. Forwarding should only happen

- if the forwarding instruction *will actually write* a register: `EX/MEM.RegWrite` (or `MEM/WB.RegWrite`) must be 1 — otherwise a `sw` or `beq` whose Rd field happens to match would wrongly forward garbage;
- and only if its destination is not register `$zero`: `EX/MEM.RegisterRd`$\neq$ 0 (`$zero` is always 0 and can never really be written, so forwarding a "result" for it would be wrong, e.g. `sll $0,$1,2` is a `nop`) [Processor slide 57].

11.3 The forwarding unit and the forwarding paths [Processor slide 58–61]

The ALU's two inputs are each fed through a 3-input multiplexer (Figure 11.2). The **forwarding unit** is a small block of logic that looks at the register numbers in the pipeline registers and sets the MUX selects `ForwardA` (for the first ALU input) and `ForwardB` (second input):

MUX control	Source of the ALU operand	Meaning
`00`	ID/EX	the operand read from the register file in ID (no forwarding)
`10`	EX/MEM	forwarded from the prior ALU result
`01`	MEM/WB	forwarded from data memory or an earlier ALU result

11.3.1 The conditions, written out [Processor slide 61]

EX hazard (producer is the previous instruction):

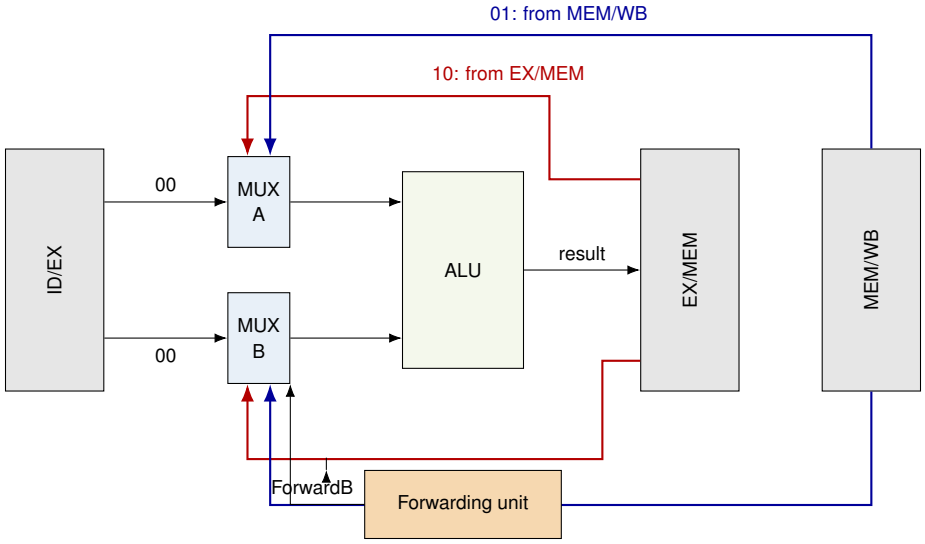


Figure 11.2: Schematic of the forwarding hardware (simplified). Each ALU input passes through a 3-way multiplexer (inputs 00, 10, 01) whose select, `ForwardA` or `ForwardB`, comes from the forwarding unit. The unit compares register numbers held in ID/EX, EX/MEM and MEM/WB (ForwardA wiring omitted for clarity).

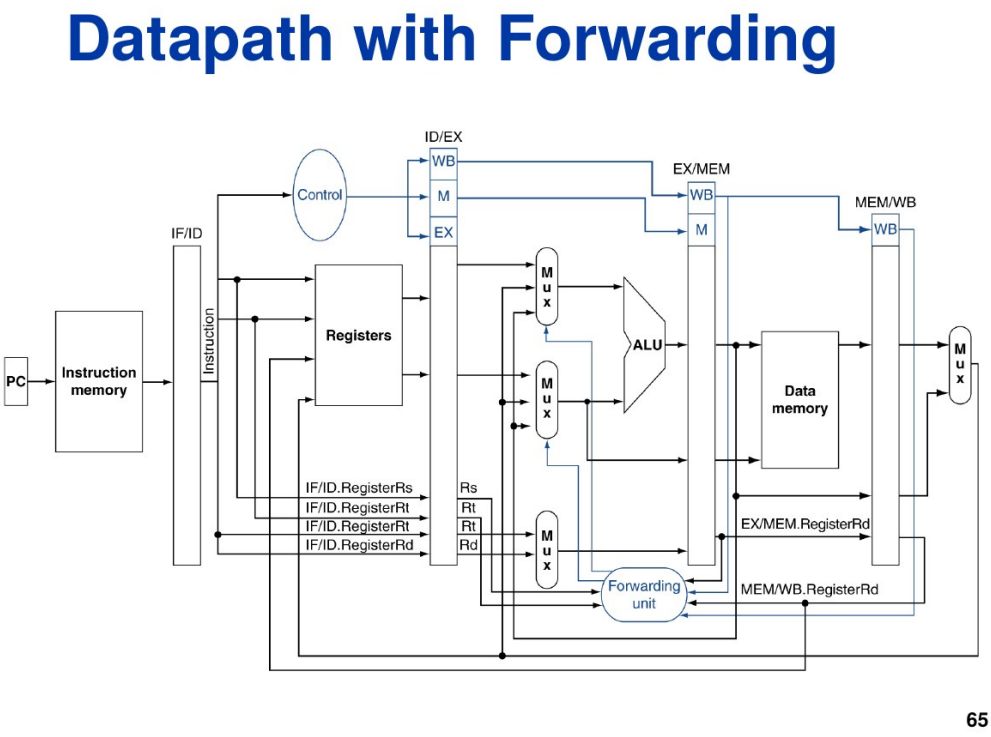


Figure 11.3: The datapath with forwarding (slide 65): two multiplexers in front of the ALU and the forwarding unit below it.

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10
```

MEM hazard (producer is two instructions earlier) — see the corrected version below.

11.4 The double data hazard [Processor slide 62–64]

Definition: Double data hazard

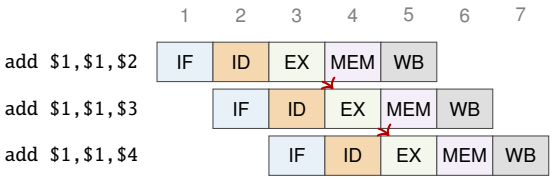
A *double data hazard* occurs when *both* conditions hold for the same operand: the register is written by the instruction one place ahead (EX hazard) *and* also by the instruction two places ahead (MEM hazard). Both pipeline registers then hold a value for the same register, and the hardware must choose the *most recent* one: the EX/MEM value.

Worked Example: Slide example

```
add $1, $1, $2
add $1, $1, $3
add $1, $1, $4
```

Let $1 = 10, $2 = 30, $3 = 20, $4 = 4. The programmer expects $10 + 30 = 40$, then $40 + 20 = 60$, then $60 + 4 = 64$. When the third add is in EX, the first add (value 40) is in MEM/WB, and the second (value 60) is in EX/MEM. Both match `ID/EX.RegisterRs`= 1. If the MEM hazard rule fired, the *older* value 40 would be forwarded — wrong! The correct value is 60, from EX/MEM.

Clock cycle	1	2	3	4	5	6	7
`$2,$3,$4`	30,20,4	same	same	same	same	same	same
$1 (register file)	10	10	10	10	10/40	40/60	60/64



Revised MEM-hazard condition. Forward from MEM/WB **only if the EX-hazard condition is not true**:

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
```

In words: *the nearest earlier writer wins.* A common exam answer is: ForwardA = 10 if the EX hazard holds; else ForwardA = 01 if the (original) MEM hazard holds; else ForwardA = 00.

11.5 Load-use hazards: detection and stall [Processor slide 66–72]

11.5.1 When is the data ready?

For `lw $2,20($1)` the loaded value is ready at the *end of MEM*. For an instruction immediately behind it that needs the value as an ALU input in EX, the data is needed *one cycle too early*; forwarding cannot help and the pipeline must **stall for one cycle**. (If the instruction behind the load is a `sw` storing that same register, the data is needed only in the MEM stage, so it can be forwarded without a stall: slide 66 "Load → Store (mem-to-mem copy)". The hazard detection condition below does not distinguish this case and will stall anyway; a refined design adds a MEM-to-MEM forwarding path.)

11.5.2 The detection condition [Processor slide 68–69]

The check is made when the *using* instruction is decoded, i.e. it is in the ID stage while the load is one stage ahead (in EX). The load's destination register is its `Rt`; the user's source registers are in IF/ID:

```
if (ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))
    stall the pipeline for one cycle
```

`ID/EX.MemRead` is 1 only for a load; its destination is Rt, hence "`ID/EX.RegisterRt`".

11.5.3 How to stall [Processor slide 70–74]

Stalling means: the instruction in ID must be *held* for another cycle, and the instruction in IF must also be held, while the EX stage is given a *bubble* (a `nop`). The hazard detection unit does it with three actions:

1. **Force the control fields in ID/EX to 0**: all of EX, MEM and WB signals become 0, so the instruction that moves into EX is a `nop` (a bubble) that changes nothing (no register write, no memory write).
2. **Do not update the PC** (`PCWrite`=0): the same instruction is fetched again.
3. **Do not update the IF/ID register** (`IF/IDWrite`=0): the using instruction is decoded again in the next cycle.

After the one-cycle stall the load is in MEM/WB and the value can be forwarded by the forwarding unit to the instruction now in EX.

11.5.4 The hazard detection unit in the datapath [Processor slide 75]

11.6 Summary

- Forward ALU results from EX/MEM (one ahead) or MEM/WB (two ahead) into the ALU inputs; conditions 1a, 1b, 2a, 2b, qualified by `RegWrite` and `Rd`$\neq 0$; the *EX hazard has priority* (double data hazard).
- A load followed immediately by a user of the loaded register is a load-use hazard: one stall, detected in ID by `ID/EX.MemRead` and a register-number match with the user's Rs/Rt; the stall zeroes the ID/EX control signals and freezes PC and IF/ID.

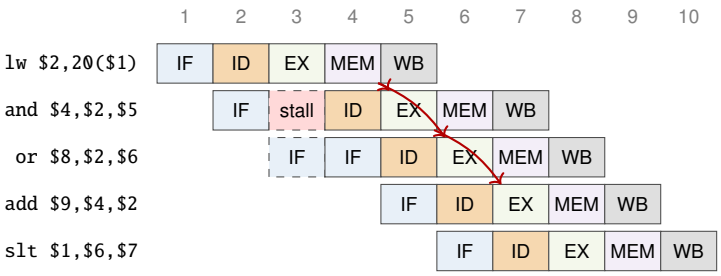


Figure 11.4: Slide 73's example: `and $4,$2,$5` is stalled one cycle because it uses `$2` loaded by the preceding `lw`. During the stall cycle (cycle 4) `and` repeats its decode and `or` is fetched again; `or` gets `$2` from MEM/WB, and `add` gets `$4` from `and`. 10 cycles in total.

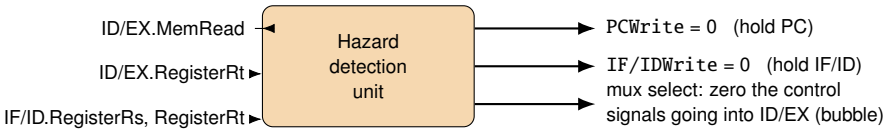


Figure 11.5: Inputs and outputs of the hazard detection unit (the version drawn on slide 75).

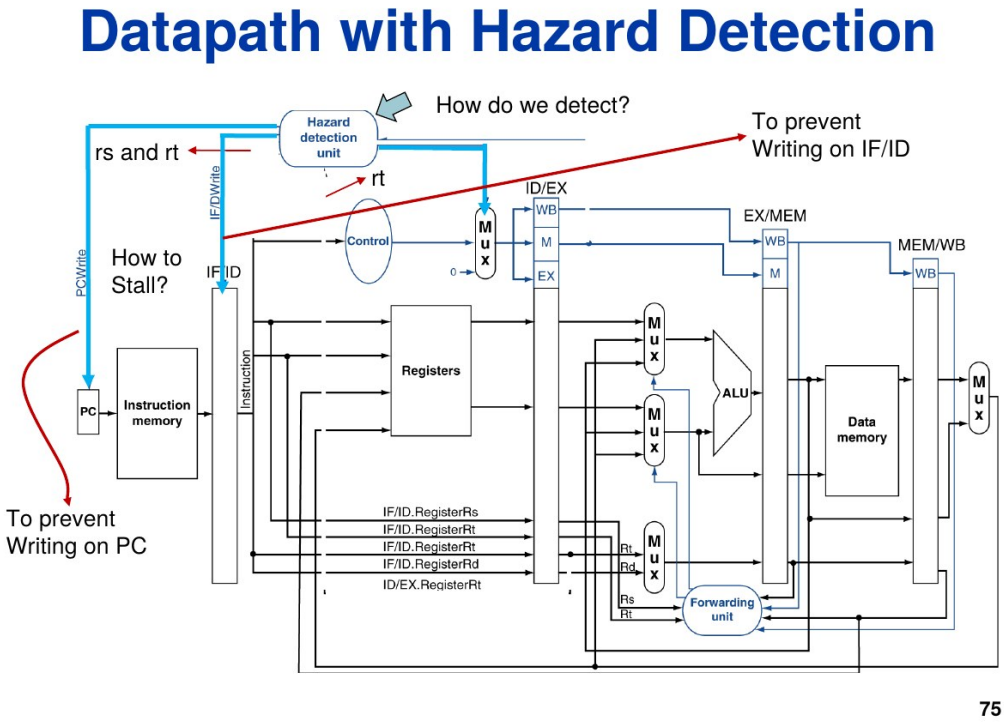


Figure 11.6: The pipelined datapath with the hazard detection unit (slide 75): it reads the register numbers, and controls PC writing, IF/ID writing and the control multiplexer in front of ID/EX.

- Stalls reduce performance but are needed for correctness; compilers can schedule code to avoid them.

Lecture source

Processor slides 53–76 (§4.7 Data Hazards: Forwarding vs. Stalling).

CHAPTER 12

Control Hazards and Branch Prediction

Chapter goal

A branch changes which instruction should come next, but the pipeline has already started fetching the instructions that follow it. This chapter explains the resulting *control hazard*, how much it costs, and the techniques that reduce the cost: moving the decision earlier, predicting, and delaying. *Lecture source: Processor slides 77–94 (§4.8).*

12.1 Why branches are a problem [Processor slide 77]

Definition: Control hazard (branch hazard)

A *control hazard* occurs when the pipeline must decide which instruction to fetch next, but the outcome of a branch (taken or not taken) and its target address are not yet known. Instructions fetched in the meantime may be the wrong ones.

A branch is **taken** if the condition is true and the PC is set to the target address; it is **not taken** if the PC simply continues to the next sequential instruction (PC+4). In the basic pipeline the branch decision (the ALU comparison, plus the target adder) is finished in EX and the PC is only updated in MEM. Until then the fetch stage keeps fetching sequentially.

12.2 Branch resolved in MEM: three lost cycles [Processor slide 78–79]

Suppose `beq` is in the MEM stage and the comparison says "taken". By now three younger instructions (in EX, ID and IF) have already entered the pipeline; they came from the fall-through path and must be **flushed** (turned into bubbles by zeroing their control signals) before the correct target is fetched. The penalty is therefore **3 cycles** per taken branch.

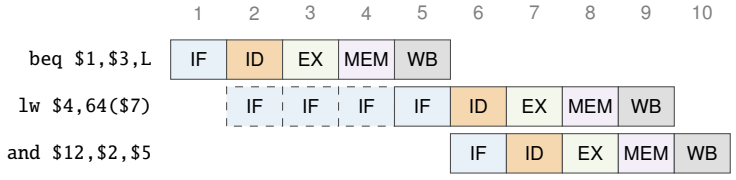


Figure 12.1: Branch decided at the end of the MEM stage: the instruction at the target address (`lw`) can only be fetched in cycle 5, three cycles later than if no branch existed (it would have been fetched in cycle 2).

Worked Example: Cost of the 3-cycle penalty

Suppose 20% of instructions are branches and half are taken, and everything else has CPI = 1. The extra cycles per instruction are $0.2 \times 0.5 \times 3 = 0.3$, so CPI = 1.3, a 30% slowdown. Even though the numbers are made up for illustration, they show why branches matter: the

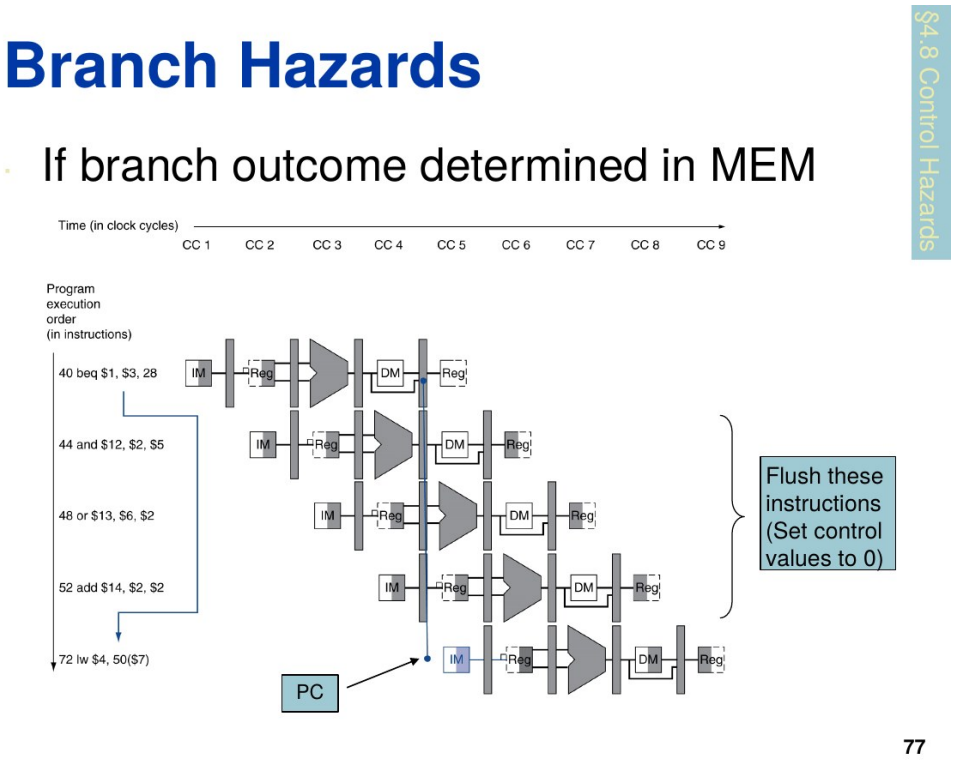


Figure 12.2: Slide 78–79: the branch decision happens in MEM; the three instructions behind it are flushed (IF.Flush/ID.Flush/EX.Flush) and the PC is loaded with the target.

deeper the pipeline, the larger the penalty.

12.2.1 A subtle interaction

A taken branch in MEM and a load-use stall can occur in the same cycle (a stall holds PC and IF/ID, but the branch wants to overwrite them). The *flush must win*: the instructions being held belong to the wrong path anyway, so the branch's PC update takes priority over the hazard unit's hold.

12.3 Reducing the delay: decide the branch in ID [Processor slide 80–83]

The branch penalty is reduced by moving the decision earlier. In the ID stage we can

- add the sign-extended, shifted offset to PC+4 using a *dedicated adder* in ID to get the target, and
- compare the two register values with a simple equality comparator (XOR each bit pair and OR the results; no full ALU subtract is needed for `beq`/`bne`).

Then only the single instruction in IF is wrong: it is flushed (`IF.Flush` turns it into a `nop`), and the penalty falls to **1 cycle**.

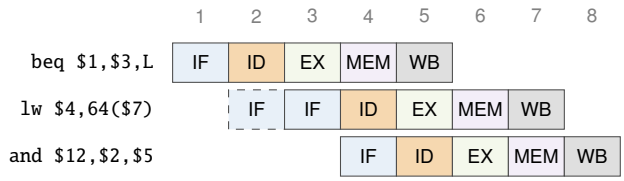


Figure 12.3: Branch decided in ID: the target is fetched in cycle 3, only one cycle later than normal.

12.3.1 New data hazards for branches [Processor slide 84–86]

If the branch decision is made in ID, its operands are needed earlier than for an ordinary instruction, and a producer that is still in the pipeline causes a hazard.

Producer of a compared register	Cost for the `beq` in ID
ALU instruction, 2nd or 3rd before the branch	**No stall**: forward the result.
ALU instruction *immediately* before the branch, or a `lw` 2nd before the branch	**1 stall cycle**.
`lw` *immediately* before the branch	**2 stall cycles** (the data exists only at the end of MEM).

The reasoning is always the same: the branch needs the value at the start of its ID stage, and a value produced by an ALU instruction exists only at the end of that instruction's EX, while a loaded value exists only at the end of its MEM.

Worked Example: Slide 80

```
lw $1, addr($0)
add $4, $5, $6
beq $1, $4, target
```

`$4` comes from the `add` immediately before the branch (1 stall), and `$1` from a `lw` two instructions before it (also 1 stall). Both waits happen in the same cycle, so the `beq` stalls **1** cycle in total, then proceeds with forwarded values.

Common mistake

The exact count depends on which forwarding paths into ID the design provides. In an exam, state your assumption and then count: "value ready at end of stage X; branch needs it at the start of ID."

12.4 Branch prediction [Processor slide 87–93]

Even a 1-cycle penalty hurts in long pipelines. Instead of waiting, the processor can *guess* the branch outcome and start fetching along the guessed path. If the guess is right, nothing is lost; if wrong, the wrongly fetched instructions are flushed (the same penalty as before).

12.4.1 Static prediction

A **static** prediction is fixed at compile or design time.

- **Predict not taken**: always keep fetching PC+4. Simple. Right for forward `if` branches that fall through, wrong at the end of every loop iteration but one.
- **Predict taken** for *backward* branches (loops) and not taken for forward branches: because a loop branch jumps back many times and falls through once.

For a loop executed 10 times, a backward branch is taken 9 times and not taken once; predicting "taken" is right 90% of the time.

12.4.2 Dynamic prediction [Processor slide 88]

A **dynamic** predictor uses the *history* of each branch. A small table, the **branch prediction buffer** (also called branch history table), is indexed by the lower bits of the branch instruction's address. Each entry records the recent outcome of the branch. The processor looks up the entry in IF, fetches along the predicted path, and updates the entry after the real outcome is known. If the guess was wrong, it flushes the pipeline, fetches the other path, and corrects the entry.

1-bit predictor [Processor slide 89]

Each entry stores a single bit: the *last* outcome (1 = taken, 0 = not taken). The prediction is "same as last time".

Worked Example: Nested loop, 1-bit predictor

Consider the inner-loop branch of a nested loop that is taken 9 times and then not taken once on exit, and then the whole inner loop is entered again. The mispredictions are: (i) at the exit (predicted taken, actually not taken), and (ii) on the first iteration of the next execution (the bit now says "not taken" but the branch is taken). Two mispredictions per 10 executions: accuracy $= 8/10 = 80\%$, although the branch is taken 90% of the time. The predictor is wrong twice per loop because it changes its mind after every single deviation.

2-bit predictor [Processor slide 90–91]

To tolerate one-off exceptions, use a 2-bit counter with four states: *strongly taken*, *weakly taken*, *weakly not taken*, *strongly not taken*. The prediction changes only after *two* consecutive mispredictions.

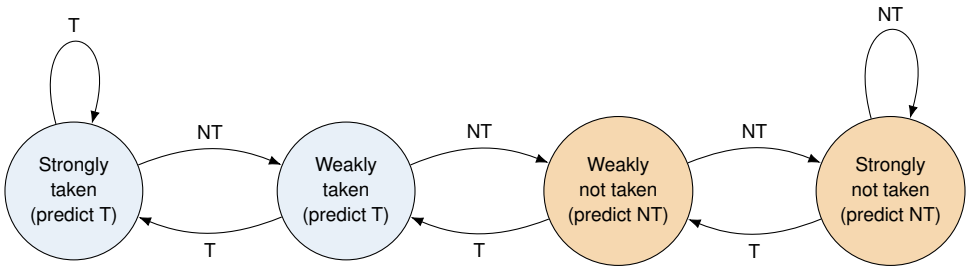


Figure 12.4: The 2-bit saturating-counter predictor. T = branch actually taken, NT = not taken. A prediction flips only when the counter crosses from a "weak" to the opposite "weak" state.

Worked Example: Same loop, 2-bit predictor

Start in *strongly taken*. In the 9 taken iterations it stays there. At the exit (NT) it moves to *weakly taken* but still predicts taken (wrong once). When the loop is entered again the branch is taken, so it returns to strongly taken: correct. Only **one** misprediction per 10: accuracy 90%.

Worked Example: Prediction accuracy for the outcome sequence T, T, NT, T

Start in the strongly-taken state: predictions T (right), T (right), T (wrong; state → weakly taken), T (right). Accuracy = 3/4. With a 1-bit predictor starting at T: T right, T right, T wrong (bit becomes NT), NT wrong (actual T): 2/4. (The starting state matters, so state it in an exam answer.)

12.4.3 Branch target buffer [Processor slide 92]

Predicting "taken" is useless unless we also know the *target address* right away in the IF stage. A **branch target buffer (BTB)** is a small cache indexed by the PC of the fetched instruction; it stores, for recently executed taken branches, the predicted target address (and often prediction bits). In IF the PC is compared with the BTB entries:

- **hit**: the instruction is a known branch, so fetch the stored target next cycle (no bubble if predicted correctly);
- **miss**: fetch PC+4 as usual; if the instruction turns out to be a taken branch, insert its entry afterwards.

Jumps (`j`, `jal`, `jr`) can also be handled by the BTB; without it, a jump is recognised only in ID and costs 1 bubble.

12.4.4 Even a perfect predictor needs a cycle?

A frequently asked question: with perfect prediction is the penalty zero? With a BTB, yes, for branches found in the buffer. Without a BTB, the target is computed in ID (or EX), so **at least 1 cycle** is still lost even if the direction is predicted correctly. Always state which hardware you assume.

12.5 Delayed branch [Processor slide 94]

Another approach, used by early MIPS, is to remove the problem in the instruction set: the instruction immediately *after* a branch (in the **branch delay slot**) is always executed, whichever way the branch goes. The assembler/compiler tries to place there an instruction that is useful in both cases, e.g. an instruction from before the branch that the branch does not depend on.

Worked Example: Filling the delay slot

```
# original # with delayed branch
add $4, $5, $6 beq $1, $2, L
beq $1, $2, L ==> add $4, $5, $6 # delay slot, always runs
lw $7, 0($8) lw $7, 0($8)
```

The `add` does not influence the branch, so moving it into the slot gives the same result with no wasted cycle. If nothing suitable exists, a `nop` is inserted. With deeper pipelines one slot

is not enough, which is why modern processors use dynamic prediction instead.

12.6 Summary

- Branch in MEM: 3-cycle penalty; branch in ID with a dedicated adder and comparator: 1-cycle penalty (+ data-hazard stalls for operands).
- Predictors: static (not-taken / backward-taken), 1-bit, 2-bit, BTB. 2-bit is better for loops (1 miss per loop execution instead of 2).
- Delayed branch fills the slot after the branch with an always-useful instruction.

Lecture source

Processor slides 77–94 (§4.8 Control Hazards). The branch-in-ID data-hazard counts, the accuracy examples and the BTB description are expanded in this book using the standard treatment; stall counts follow slides 79–81.

CHAPTER 13

Instruction-Level Parallelism and Real Processors

Chapter goal

A single pipeline can finish at most one instruction per clock cycle. To go faster, designers start *several* instructions each cycle (multiple issue) and let hardware or the compiler rearrange them. This chapter explains these ideas, the dual-issue MIPS, loop unrolling, dynamic scheduling, why the approach hits a power limit, and what real processors (ARM Cortex-A8, Intel Core i7) do. *Lecture source: Processor slides 86–111 (§4.10, 4.11, 4.14).*

13.1 Instruction-level parallelism [Processor slide 86]

Definition: Instruction-level parallelism (ILP)

ILP is the parallelism available *among the individual instructions of one program*: independent instructions can be executed at the same time. Pipelining already exploits ILP by overlapping instructions. Two ways to get more:

1. **Deeper pipelines**: less work per stage, so a shorter clock cycle (but larger hazard penalties).
2. **Multiple issue**: replicate pipeline stages to form several pipelines and start several instructions in each clock cycle.

With multiple issue the CPI can be below 1, so we prefer its reciprocal, **IPC** (instructions per cycle) $= 1/\text{CPI}$.

Worked Example: Peak performance

A 4 GHz processor that issues 4 instructions per cycle has peak IPC $= 4$, peak CPI $= 1/4 = 0.25$, and a peak rate of $4 \times 4\,\text{GHz} = 16$ billion instructions per second (16 BIPS). "Peak" means the maximum that can be achieved if nothing ever stalls. Real programs have dependences, so the achieved IPC is much smaller.

13.2 Two ways to issue several instructions [Processor slide 87]

	Static multiple issue	**Dynamic multiple issue**
Who decides?	The *compiler* groups instructions to be issued together into "issue slots" and avoids hazards.	The *CPU* examines the instruction stream at run time and chooses what to issue each cycle.
Hazards	Compiler detects and avoids them.	CPU resolves them with advanced hardware.
Name	VLIW-style (very long instruction word).	**Superscalar** processor.

13.3 Speculation [Processor slide 88–89]

Definition: Speculation

Speculation means to *guess* what to do with an instruction, start the operation as early as possible, check later whether the guess was right, and if so complete the operation; if not, *roll back* (undo) and do the right thing.

Examples: guess a branch's outcome and execute the predicted path; guess that a load does not depend on an earlier store and execute it early (roll back if the location is in fact updated). The compiler can speculate by moving a load above a branch, possibly adding "fix-up" instructions to recover. Hardware can look ahead, *buffer* the results of speculated instructions, and only make them permanent when it knows they were needed, flushing the buffers after a wrong guess.

13.4 Static multiple issue [Processor slide 90–94]

In static multiple issue the compiler forms **issue packets**: groups of instructions that can be issued in one cycle, determined by the pipeline resources they need. An issue packet can be thought of as one *very long instruction* specifying several concurrent operations, hence **VLIW**. The compiler must remove hazards inside a packet (no dependences within a packet; dependences between packets may exist, depending on the ISA) and pad unused slots with `nop`.

13.4.1 MIPS with static dual issue [Processor slide 92–93]

The lecture's example machine issues **two-instruction packets**, 64-bit aligned:

- one **ALU/branch** instruction, and
- one **load/store** instruction (in this order within the packet);
- an unused slot is filled with a `nop`.

Two instructions are fetched together (64 bits); both go through IF, ID, EX, MEM, WB side by side. Extra hardware: a second ALU (for address calculation of the load/store), two extra register read ports and one extra write port.

13.4.2 New hazards in the dual-issue machine [Processor slide 94]

- **EX data hazard.** In a single-issue pipeline forwarding avoids stalls. Now the ALU result cannot be used by a load/store *in the same packet*: for `add $t0,$s0,$s1` followed by `lw $s2,0($t0)` the two must be in different packets, effectively a stall.
- **Load-use hazard.** Still a one-cycle use latency, but now it costs *two instruction slots*.

So more aggressive scheduling is needed to keep both slots busy.

13.4.3 Scheduling example [Processor slide 95]

```
Loop: lw $t0, 0($s1) # $t0 = array element
      addu $t0, $t0, $s2 # add scalar in $s2
      sw $t0, 0($s1) # store result
      addi $s1, $s1, -4 # decrement pointer
```

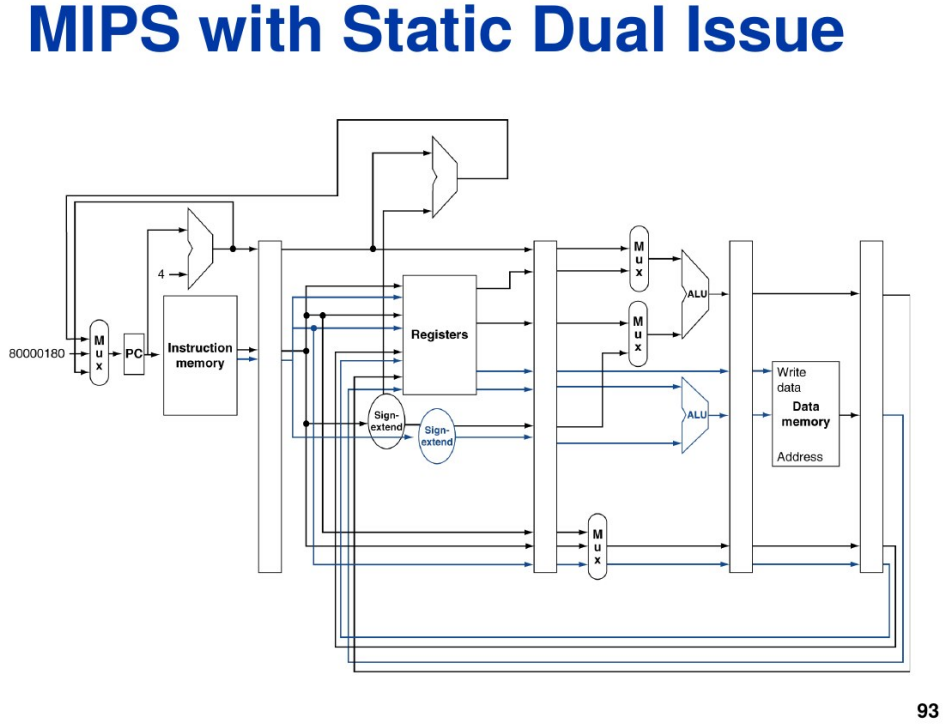


Figure 13.1: Datapath of the dual-issue MIPS (slide 93): an extra adder/ALU, more register-file ports and 64-bit instruction fetch.

```
    bne $s1, $zero, Loop # branch if $s1 != 0
```

We place instructions into packets obeying the data dependences: `addu` needs the loaded value (load-use, one cycle after the load); `sw` needs the `addu` result; `bne` needs the new `$s1` from `addi`. Moving `addi` up (it only depends on `$s1`, which the `lw` has already read in the same or an earlier packet) and adjusting the `sw` offset from 0 to 4 (because `$s1` has already been decremented by 4) gives:

	ALU/branch	Load/store	Cycle
`Loop:`	`nop`	`lw $t0, 0($s1)`	1
	`addi $s1, $s1, -4`	`nop`	2
	`addu $t0, $t0, $s2`	`nop`	3
	`bne $s1, $zero, Loop`	`sw $t0, 4($s1)`	4

Five instructions in four cycles: IPC $= 5/4 = 1.25$, compared with the peak of 2. Four slots are wasted (`nop`s).

13.4.4 Loop unrolling and register renaming [Processor slide 96–97]

Definition: Loop unrolling

Loop unrolling replicates the loop body several times inside one loop iteration so that there are more independent instructions to fill the issue slots, and less loop-control overhead (fewer `addi`/`bne` per element of work).

When copies of the body are interleaved, they must not interfere through reusing the same register. Each copy therefore uses *different registers* (**register renaming** done by the compiler):

`$t0`, `$t1`, `$t2`, `$t3`. The reuse of a register name, in a store followed by a load that overwrites the same register, creates a **name dependence** or *anti-dependence*; it is not a true flow of data, and renaming removes it.

Unrolling the loop four times (each element is 4 bytes, so `$s1` moves by 16 per iteration):

	ALU/branch	Load/store	Cycle
`Loop:`	`addi $s1, $s1, -16`	`lw $t0, 0($s1)`	1
	`nop`	`lw $t1, 12($s1)`	2
	`addu $t0, $t0, $s2`	`lw $t2, 8($s1)`	3
	`addu $t1, $t1, $s2`	`lw $t3, 4($s1)`	4
	`addu $t2, $t2, $s2`	`sw $t0, 16($s1)`	5
	`addu $t3, $t3, $s2`	`sw $t1, 12($s1)`	6
	`nop`	`sw $t2, 8($s1)`	7
	`bne $s1, $zero, Loop`	`sw $t3, 4($s1)`	8

Correction / clarification to the lecture

Slide 97 prints `addu $t3, $t4, $s2` in cycle 6. `$t4` was never loaded; the instruction must add the scalar to `$t3` (the value loaded in cycle 4), so it is `addu $t3, $t3, $s2`, as in the table above.

Count: 1 `addi` + 4 `lw` + 4 `addu` + 4 `sw` + 1 `bne` = 14 instructions in 8 cycles: IPC $= 14/8 = 1.75$, much closer to 2, *at the cost of more registers and larger code*. In terms of time: the first schedule handles 1 array element per 4 cycles; the unrolled one handles 4 elements per 8 cycles, i.e. 2 cycles per element.

13.5 Dynamic multiple issue: superscalar processors [Processor slide 98–101]

In a **superscalar** processor the CPU decides in each cycle whether to issue 0, 1, 2, ... instructions, avoiding structural and data hazards itself. The programmer and compiler need not schedule code for the machine (though good scheduling still helps), and the CPU guarantees the program's meaning.

13.5.1 Dynamic pipeline scheduling [Processor slide 99]

Out-of-order execution: the CPU may execute instructions in a different order from the program to avoid stalls, but results are committed (written to registers and memory) *in program order*.

Worked Example: Slide 99

```
lw $t0, 20($s2)
addu $t1, $t0, $t2 # must wait for the load
sub $s4, $s4, $t3 # independent of the above
slti $t5, $s4, 20 # depends on sub only
```

While `addu` waits for the load to finish, `sub` (and then `slti`) can already start. A simple in-order pipeline would be idle.

The main parts: *reservation stations* hold instructions with their pending operands and wait until all operands are available; results are also sent directly to any reservation station waiting for them; the *reorder buffer* holds results until they can be written to the registers in order, and can supply operands for newly issued instructions. If speculation was wrong, the buffered results are simply discarded.

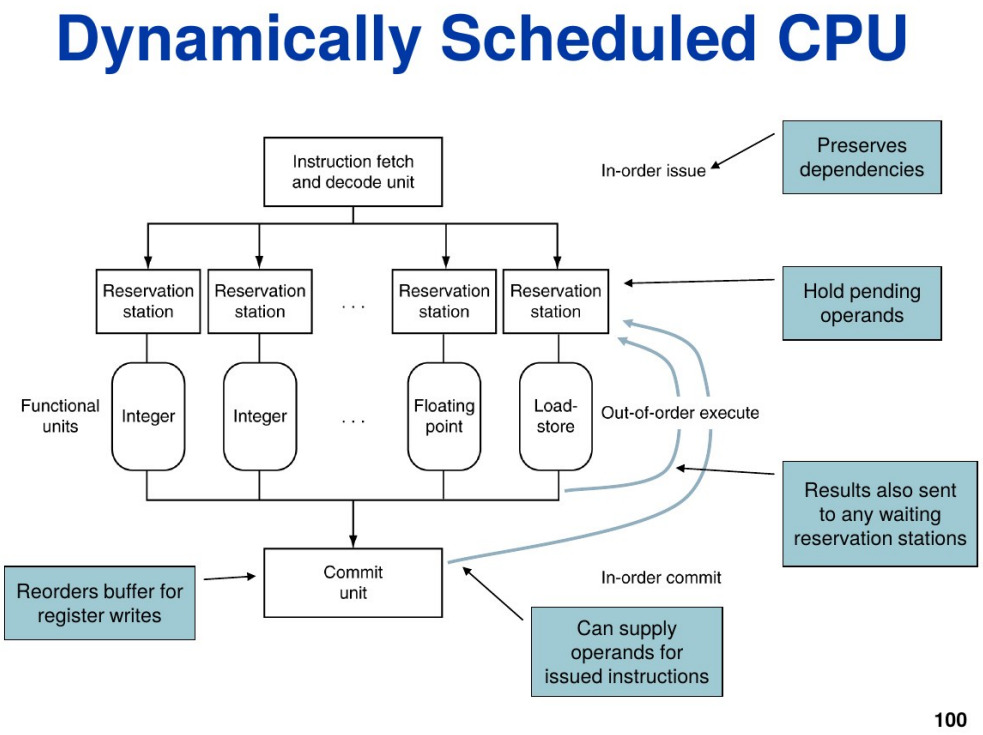


Figure 13.2: A dynamically scheduled CPU (slide 100): an instruction fetch/decode unit feeds *reservation stations* in front of the functional units; results go to a *commit unit with a reorder buffer* that updates registers in program order.

13.5.2 Why not just let the compiler do it? [Processor slide 101]

- Not all stalls are predictable at compile time, e.g. **cache misses**.
- The compiler cannot always schedule around branches because the outcome is determined dynamically.
- Different implementations of the same ISA have different latencies and hazards, so code scheduled for one is not ideal for another.

13.6 Does multiple issue work? [Processor slide 102]

Yes, but not as much as we would like.

- Programs have *real* dependences that limit ILP; some are hard to eliminate (e.g. **pointer aliasing**: the compiler cannot tell whether two pointers refer to the same memory).
- Some parallelism is hard to expose because of the limited *window size* (number of instructions the hardware can examine) at issue.
- Memory delays and limited memory bandwidth make it hard to keep the pipelines full.
- Speculation can help if done well.

13.7 Power efficiency [Processor slide 103]

Dynamic scheduling and speculation need complex hardware, which consumes power. Multiple simpler cores can be better than one very complex core.

Microprocessor	Year	Clock (MHz)	Pipeline stages	Issue width	Out-of-order / speculation	Cores	Power (W)
i486	1989	25	5	1	No	1	5
Pentium	1993	66	5	2	No	1	10
Pentium Pro	1997	200	10	3	Yes	1	29
P4 Willamette	2001	2000	22	3	Yes	1	75
P4 Prescott	2004	3600	31	3	Yes	1	103
Core	2006	2930	14	4	Yes	2	75
UltraSparc III	2003	1950	14	4	No	1	90
UltraSparc T1	2005	1200	6	1	No	8	70

Reading the table: from the i486 to the Prescott, clocks rose 144-fold and power rose twenty-fold; the long pipeline (31 stages) and aggressive speculation burned power. Afterwards the industry moved to several moderate cores (Core, T1) rather than one faster core. This is the **power wall**.

13.8 Real processors: ARM Cortex-A8 and Intel Core i7 [Processor slide 104–108]

	ARM A8	Intel Core i7 920
Market	personal mobile devices	server, cloud
Thermal design power	2 W	130 W
Clock rate	1 GHz	2.66 GHz
Cores per chip	1	4
Floating point?	No	Yes
Multiple issue?	Dynamic	Dynamic
Peak instructions per cycle	2	4
Pipeline stages	14	14
Pipeline schedule	static in-order	dynamic out-of-order with speculation
Branch prediction	2-level	2-level
1st-level caches/core	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
2nd-level caches/core	128–1024 KiB	256 KiB
3rd-level cache (shared)	–	2–8 MB

The A8 is designed for *low power*: it issues two instructions per cycle in program order (statically scheduled order, with simple dynamic checking). The i7 is designed for *high performance*: it issues up to four instructions per cycle out of order with speculation, with micro-operations (the IA-32 instructions are translated into simpler internal *micro-ops*). Both rely on a two-level branch predictor. The slides also show measured CPI charts for both processors: in both cases the measured CPI is well above the ideal CPI (1/peak IPC, i.e. 0.5 for the A8 and 0.25 for the i7) because of cache misses and branch mispredictions. (The individual benchmark values are in the slide figures and are not reproduced here.)

13.9 Fallacies, pitfalls and conclusion [Processor slide 109–111]

Fallacies (statements that sound true but are false).

- *Pipelining is easy.* The basic idea is easy, the devil is in the details, e.g. detecting data hazards correctly.

- *Pipelining is independent of technology.* More transistors make more advanced techniques feasible; pipeline-related ISA design must account for technology trends (e.g. predicated instructions).

Pitfalls (mistakes that are easy to make).

- Poor ISA design makes pipelining harder: complex instruction sets (VAX, IA-32) need significant overhead (the IA-32 micro-op approach), complex addressing modes, register update side effects, memory indirection.
- Delayed branches: advanced pipelines have long delay slots, so the one-slot solution no longer hides the delay.

Concluding remarks. The ISA influences the design of datapath and control, and vice versa. Pipelining improves instruction *throughput*, but the *latency* of each instruction is not reduced. The hazards are structural, data and control. Multiple issue and dynamic scheduling exploit ILP, but dependences limit the achievable parallelism and the complexity leads to the power wall.

13.10 Summary

- Multiple issue: static (compiler, VLIW) vs dynamic (superscalar); IPC $= 1/\text{CPI}$; peak vs achieved.
- Dual-issue example: IPC 1.25 for the simple schedule, 1.75 after $4\times$ unrolling with register renaming.
- Out-of-order execution + in-order commit; reservation stations and reorder buffer; speculation with rollback.
- Power wall $\Rightarrow$ multicore. A8 (low power, in-order) vs i7 (out-of-order, 4-wide).

Lecture source

Processor slides 86–111. The A8/i7 performance charts (slides 106, 108) are described only qualitatively.

CHAPTER 14

Pipelining: Solved Past Examination Questions (Part 1)

How this chapter is organised

Questions are grouped by topic, from the supplied CSE 305 / CSE 209 papers (2012–13 to 2023–24). This part covers **P1** pipeline speed-up, **P2** cycle counts and code scheduling, and **P3** data hazards and forwarding. The next chapter covers hazard detection, control hazards, branch prediction, datapath, multiple issue and CISC/RISC. Every cycle count was computed with a small pipeline timing model (checked against the lecture's own examples) and every diagram below is generated from that model. Where a scan was unreadable, the assumption made is stated in the solution.

Conventions used in all solutions. Five stages IF, ID, EX, MEM, WB. The register file is written in the first half of a cycle and read in the second half. Forwarding paths: EX/MEM → ALU and MEM/WB → ALU (Chapter 11); a load followed immediately by a user costs one stall. Total cycles = (number of instructions +4) + stalls (counted from the first IF to the last WB).

14.1 P1: Pipeline speed-up and why single-cycle is inefficient

Past Exam Question (2015–16 Q2(b), 5 marks; 2013–14 Q7(d), 9 marks)

Times for a load word: instruction fetch 200 ps, register read 100 ps, ALU 200 ps, memory 200 ps, register write 100 ps. Program: `lw $1,100($0); lw $2,200($0); lw $3,300($0)`. What is the speed-up of the pipelined datapath over the single-cycle datapath? What will it be if 2 000 000 more `lw` are added? (2013–14: what are the stages of the MIPS pipeline; 1 000 000 more `lw`.)

Solution. *Stages:* IF, ID (decode and register read), EX (ALU / address calculation), MEM (data memory), WB (register write).
Single-cycle: the clock must fit the whole instruction: $200 + 100 + 200 + 200 + 100 = 800$ ps, and every instruction takes one such cycle. For n instructions: $800n$.
Pipelined: the clock equals the slowest stage, 200 ps. The first instruction needs 5 cycles, each further one finishes one cycle later: $(n + 4) \times 200$ ps.
For $n = 3$: single = 2400 ps; pipelined $= 7 \times 200 = 1400$ ps; speed-up $= 2400/1400 = \mathbf{1.71}$.
For $n = 3 + 2\,000\,000$: speed-up $= \dfrac{800n}{200(n+4)} = \dfrac{4n}{n+4} = \dfrac{4 \times 2\,000\,003}{2\,000\,007} \approx \mathbf{4.0}$ (3.99999). With 1 000 000 more: $3.99998 \approx 4.0$.
Interpretation: the speed-up approaches 4, not 5 (the number of stages), because the stages are unequal (200 vs. 100 ps) and the clock must fit the slowest stage. For three instructions the fill time of the pipeline dominates.

Past Exam Question (2017–18 Q1(c), 5 marks)

Same question with times IF 200, register read 100, ALU 100, memory 200, write 100 ps; 3 000 000 more `lw`.

Solution. Single-cycle clock $= 200 + 100 + 100 + 200 + 100 = 700$ ps; pipeline clock $= 200$ ps (the ALU is now only 100). $n = 3$: 2100 vs. $7 \times 200 = 1400$: speed-up $=$ **1.5**. $n = 3\,000\,003$: $\frac{700n}{200(n+4)} = 3.5\,\frac{n}{n+4} \approx$ **3.5** (limit 700/200).

Past Exam Question (2018–19 Q1(b), 5 marks; 2016–17 Q5(a)(i), idea)

Why are single-cycle implementations not very efficient?

Solution. In a single-cycle design the clock period must be long enough for the *slowest* instruction (the load, which uses all five functional steps: instruction memory, register read, ALU, data memory, register write), yet every instruction, even a simple `add` or `j`, takes this full period. Hardware units are idle most of the time (the ALU waits while memory is accessed, etc.), and the CPI is 1 with a long cycle time. Pipelining overlaps instructions: the clock only has to fit the slowest *stage*, and in steady state one instruction completes each cycle, so the throughput is roughly 5 times better for equal stages. The "great idea" behind this is *performance via pipelining*.

14.2 P2: Cycle counts and code scheduling

Past Exam Question (2015–16 Q2(d), 10 marks; 2017–18 Q1(a), 10; 2022–23 CSE 209 Q1(c), 10; 2021–22 Q1(b), 10; 2019–20 Q5(a–c), 15+5+15)

For the C code `A = B + C; D = A + E;` with base address in `$t0` and offsets A(0), B(4), C(8), D(12), E(16), all variables in memory: write the code, count the cycles (with forwarding), and reschedule to reduce the cycles.
(2019–20 uses `D = A + D` with the code shown below; 2021–22 gives a slightly different sequence with the same structure: see below.)

Solution. **Step 1: the code** (A = B + C needs two loads, one add and one store; D = A + E reuses `$t3` for A, so no reload is needed):

```
lw $t1, 4($t0) # B
lw $t2, 8($t0) # C
add $t3, $t1, $t2 # A = B + C
sw $t3, 0($t0) # store A
lw $t4, 16($t0) # E
add $t5, $t3, $t4 # D = A + E
sw $t5, 12($t0) # store D
```

Step 2: find the hazards. `add $t3` uses `$t2` loaded by the instruction right before it: a load-use hazard, **1 stall**. `sw $t3` uses `$t3` from the add right before it: forwarding, no stall (the store needs the value only in MEM). `add $t5` uses `$t4` loaded right before it: **1 stall**; `$t3` is two instructions earlier (forwarded from MEM/WB or already in the register file). Total: 2 stalls.
Step 3: cycles $= 7 + 4 + 2 =$ **13** with forwarding (without forwarding every dependence waits for WB and the count rises to 19). **Step 4: reschedule.** Load `$t4` earlier so that it fills the delay between the `lw $t2` and the add:

```
lw $t1, 4($t0)
lw $t2, 8($t0)
lw $t4, 16($t0) # moved up: fills the load-use gap
```

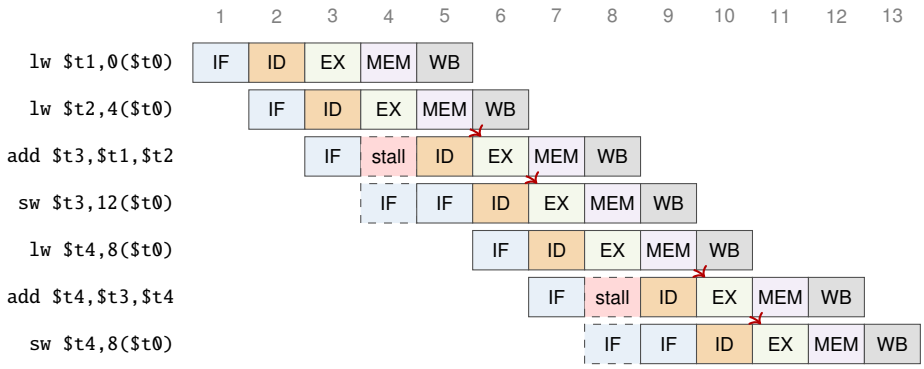


Figure 14.1: Original order, with forwarding (red arrows), 13 cycles. The dashed boxes in the ID row are stall cycles.

```
add $t3, $t1, $t2
sw $t3, 0($t0)
add $t5, $t3, $t4
sw $t5, 12($t0)
```

Now each load is followed by an instruction that does not use it: **0 stalls**, $7 + 4 = \mathbf{11}$ cycles. Correctness check: the moved load does not depend on anything before it and nothing between its old and new positions writes `$t4` or the memory location it reads (a store to A at offset 0 does not alias E at offset 16). **2019–20 version.** The code is `lw $t1,0($t0); lw $t2,4($t0); add`

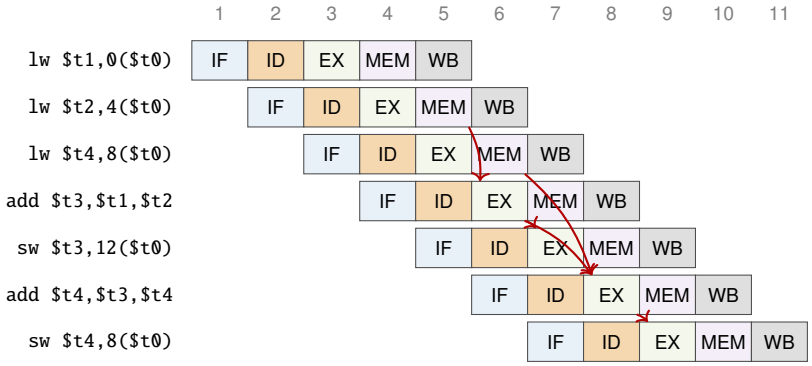


Figure 14.2: Reordered code (this is the 2019–20 variant where the last add is `add $t4,$t3,$t4`): 0 stalls, 11 cycles.

`$t3,$t1,$t2; sw $t3,12($t0); lw $t4,8($t0); add $t4,$t3,$t4; sw $t4,8($t0)` (C code `A=B+C; D=A+D`). Same analysis: stalls after `lw $t2` and after `lw $t4` (2 stalls, **13 cycles**, 5(b)); moving `lw $t4,8($t0)` to just after `lw $t2` gives 0 stalls and **11 cycles** (5(c)). The `sw $t4,8($t0)` stays after the reload, so the old D is read before it is overwritten.
2021–22 Q1(b) and 2017–18 Q1(a): identical in structure to the first listing (the 2021–22 code stores into offset 12 and 16); the answer is again 13 cycles, 2 stalls, rescheduled 11 cycles.

Past Exam Question (2020–21 Q4, 15 marks)

```
lw $t0, 0($t1)
add $t0, $t0, $t1
add $t0, $t2, $t0
add $t0, $t0, $t3
```

```
sub $t1, $t2, $t3
and $t4, $t5, $t6
sw $t0, 0($t6)
```

Find the hazards, the forwarding conditions that apply, and say whether forwarding alone solves all hazards. Give the total number of cycles.

Solution. Dependences (reading I*k* for the *k*-th instruction):

Pair	Register	Action
I1 → I2	`$t0` (load → use)	**1 stall**, then forward from MEM/WB (condition 2a)
I2 → I3	`$t0` (Rt of I3)	forward from EX/MEM: condition 1b
I3 → I4	`$t0` (Rs of I4)	forward from EX/MEM: condition 1a; *I2 also writes `$t0`, now in MEM/WB, so I4 faces a double data hazard: EX/MEM (I3) has priority*
I4 → I7	`$t0` (store data)	three instructions apart: written in the first half of WB, read in the second half of ID: no hazard

The `sub $t1` and `and $t4` do not interact with the others (the new `$t1` is not used later; `$t6` for the store is not written). Forwarding alone cannot remove the load-use stall of I1 → I2, so **no**: one stall is unavoidable (it could be removed by moving an independent instruction such as the `sub` or `and` between the `lw` and the `add`). Cycles: $7 + 4 + 1 = \mathbf{12}$.

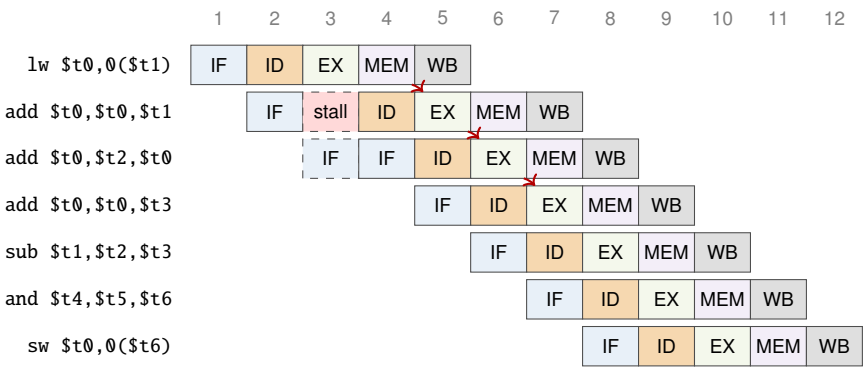


Figure 14.3: 2020–21 Q4: 12 cycles, one load-use stall.

Past Exam Question (2022–23 CSE 305 Q1(a), 12 marks; Q1(b), 16; Q1(c), 7)

```
sub $2, $1, $3
and $2, $2, $5
or $6, $2, $7
```

(a) Draw the pipeline with stalls and give the cycle count if there is no forwarding. (b) Draw it with forwarding (pipeline registers and forwarding lines), give the cycles, and explain the double data hazard. (c) Write the forwarding condition between the first and second instruction.

Solution. **(a) No forwarding.** `sub` writes `$2` in its WB (cycle 5). `and` can read `$2` only in ID in cycle 5, so it stalls 2 cycles (ID in cycles 3, 4 wasted). `or` depends on `and` (WB in cycle 8): ID in cycle 8, 2 more stalls. Total cycles $= 3 + 4 + 4 = \mathbf{11}$. **(b) With forwarding: 7 cycles, no stall.** *Double data hazard:* when `or` is in EX, `and` is in MEM (EX/MEM holds `$2` = result of `and`) and `sub` is in WB

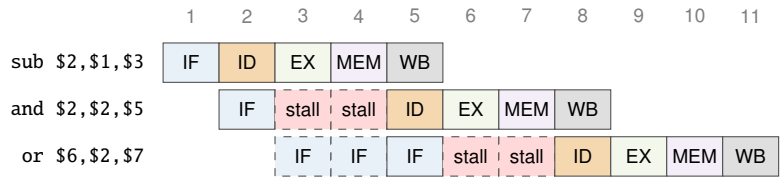


Figure 14.4: No forwarding: 11 cycles.

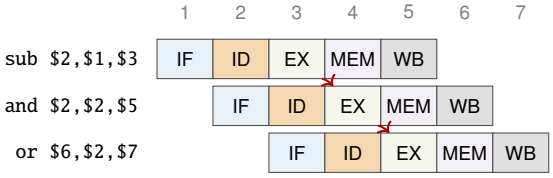


Figure 14.5: With forwarding: and takes $2 from sub (EX/MEM); or takes $2 from and (EX/MEM). 7 cycles.

(MEM/WB holds the older value of $2). Both pipeline registers match ID/EX.RegisterRs= 2, but the correct value is the one from the *later* instruction (and). The hardware gives priority to EX/MEM: forward from MEM/WB only if the EX-hazard condition is false (Chapter 11).
(c) Condition for sub → and: when and is in EX and sub in MEM: EX/MEM.RegWrite ∧ EX/MEM.RegisterRd≠ 0 ∧ EX/MEM.RegisterRd = ID/EX.RegisterRs (here 2 = 2), so ForwardA= 10.

14.3 P3: Data hazards and the forwarding unit

Past Exam Question (2012–13 Q4(a),(b),(c), 17+13+5 marks)

(a) Discuss how data hazards can be detected and solved with a forwarding unit. (b) Present a scenario where a data hazard cannot be solved by forwarding; what to do? (c) What is a branch hazard?

Solution. **(a)** A data hazard occurs when an instruction needs a register that an earlier, still-unfinished instruction will write. Detection is by comparing register numbers carried down the pipeline: if the instruction in EX has a source register (ID/EX.RegisterRs or Rt) equal to the destination register of the instruction in MEM (EX/MEM.RegisterRd) or in WB (MEM/WB.RegisterRd), and that earlier instruction writes a register (RegWrite) other than $zero, the value is not yet in the register file but already exists in a pipeline register. The *forwarding unit* sets the multiplexer selects of the ALU inputs: 00 (register-file value), 10 (from EX/MEM), 01 (from MEM/WB), with priority to the nearer producer. Write the four conditions 1a, 1b, 2a, 2b and the revised MEM condition for the double hazard, and draw Figure 11.2 and the datapath of slide 65 (Chapter 11).
(b) Load-use: lw $2,20($1) followed by and $4,$2,$5. The loaded value exists only at the end of MEM, but the and needs it at the beginning of its EX, one cycle earlier than possible. Solution: detect it in ID with the hazard detection unit (ID/EX.MemRead and ID/EX.RegisterRt equals IF/ID.RegisterRs or Rt), insert one bubble (zero the control signals in ID/EX; hold the PC and IF/ID), then forward from MEM/WB. The compiler can avoid the stall by scheduling an independent instruction after the load.
(c) A branch hazard (control hazard) arises because the branch outcome and target are not known when the next instructions have to be fetched; instructions already fetched may be from

the wrong path and have to be flushed. See Chapter 12.

Past Exam Question (2017–18 Q2(a), 10 marks; 2017–18 Q2(b), 15; 2015–16 Q3(a), 10; 2021–22 Q2(a), 15; 2023–24 Q1(a), 10; undated paper Q4(c), 4)

What is a double data hazard? Write the equations to detect forwarding in MIPS considering the double data hazard. How does MIPS detect the hazard due to a load? Give an example of a double data hazard. Design the forwarding unit.

Solution. **Double data hazard:** an operand is written by *two* instructions that are both still in the pipeline (one in MEM, one in WB), e.g.

```
add $1, $1, $2 # writes $1 (value 40 in the slide's numbers)
add $1, $1, $3 # writes $1 (60)
add $1, $1, $4 # needs the newest $1 (60 -> result 64)
```

The third instruction must take `$1` from EX/MEM (the second add), not MEM/WB (the first).
Equations:

```
EX hazard:
 if (EX/MEM.RegWrite and EX/MEM.RegisterRd != 0 and EX/MEM.RegisterRd = ID/EX.
     RegisterRs) ForwardA = 10
 if (EX/MEM.RegWrite and EX/MEM.RegisterRd != 0 and EX/MEM.RegisterRd = ID/EX.
     RegisterRt) ForwardB = 10
MEM hazard (only if the EX hazard does not apply):
 if (MEM/WB.RegWrite and MEM/WB.RegisterRd != 0
     and not(EX/MEM.RegWrite and EX/MEM.RegisterRd != 0 and EX/MEM.RegisterRd = ID/EX.
         RegisterRs)
     and MEM/WB.RegisterRd = ID/EX.RegisterRs) ForwardA = 01
 (and the same for Rt / ForwardB)
Load-use detection (in ID):
 if (ID/EX.MemRead and (ID/EX.RegisterRt = IF/ID.RegisterRs or ID/EX.RegisterRt = IF/
     ID.RegisterRt))
     stall: zero ID/EX controls, PCWrite = 0, IF/IDWrite = 0
```

The forwarding unit is a combinational block with inputs the four register numbers and the two `RegWrite` bits and outputs `ForwardA`, `ForwardB` driving the two 3-input multiplexers in front of the ALU (Figure 11.2).

Past Exam Question (2019–20 Q8(c), 12 marks)

Figure 8c shows sub, and, or with a forwarding path sub→and (green) and sub→or (yellow). Write the data forwarding condition for each path.

Solution. The figure is not legible in the scan; I assume the standard sequence of slide 53 (`sub $2,$1,$3; and $12,$2,$5; or $13,$6,$2`).
Green (sub → and), producer one ahead, result in EX/MEM: `EX/MEM.RegWrite` $\wedge$ `EX/MEM.RegisterRd` $\neq 0 \wedge$ `EX/MEM.RegisterRd = ID/EX.RegisterRs` $(2 = 2) \Rightarrow$ `ForwardA` $= 10$ (condition 1a).
Yellow (sub → or), producer two ahead, result in MEM/WB: `MEM/WB.RegWrite` $\wedge$ `MEM/WB.RegisterRd` $\neq 0 \wedge$ `MEM/WB.RegisterRd = ID/EX.RegisterRt` $(2 = 2) \wedge$ not an EX hazard on the same operand $\Rightarrow$ `ForwardB` $= 01$ (condition 2b).

CHAPTER 15

Pipelining: Solved Past Examination Questions (Part 2)

Topics in this chapter

P4 hazard detection unit and structural hazards **P5** control hazards **P6** branch prediction **P7** pipelined datapath and control **P8** multiple issue and superscalar **P9** CISC vs. RISC. Same conventions as Chapter 14.

15.1 P4: Hazard detection unit, hazard types

Past Exam Question (2022–23 CSE 209 Q2, 15 marks)

Code: `lw $t0,55($t1); addi $t1,$t0,1; lw $t0,57($t1); sw $t0,32($t4)`. Find the hazards, state the detection conditions, and show the complete hazard detection unit.

Solution. **Dependences.**

Pair	Register	Hazard?
I1 `lw` → I2 `addi`	`$t0`	load-use: **1 stall**, then forward from MEM/WB
I2 `addi` → I3 `lw`	`$t1` (base address)	ALU result needed in EX: forward from EX/MEM (condition 1a), no stall
I3 `lw` → I4 `sw`	`$t0` (data to store)	load → store: see below

(I1 and I3 both write `$t0` (WAW), and I2 overwrites `$t1` read by I1 (WAR); these are *not* hazards here because registers are written in order in WB.)
Detection condition (checked while the user is in ID and the load is in EX):

```
if (ID/EX.MemRead and
    (ID/EX.RegisterRt = IF/ID.RegisterRs or ID/EX.RegisterRt = IF/ID.RegisterRt))
  stall one cycle
```

For I1/I2: `ID/EX.MemRead`=1, `ID/EX.RegisterRt`=$t0= 8, `IF/ID.RegisterRs`=$t0 ⇒ stall. For I3/I4: `ID/EX.MemRead`=1, `Rt`=8, the `sw`'s `Rt`=$t0=8 ⇒ the condition is true and the plain unit stalls again. (A store needs its data only in MEM, so with a MEM-to-MEM forwarding path this stall is unnecessary, as slide 66 shows; the standard hazard unit is conservative.)
Action when the condition is true: zero the EX/MEM/WB control signals going into ID/EX (a bubble), `PCWrite`=0, `IF/IDWrite`=0 (Figure 11.5).
Cycles: one stall (I2) with the MEM-to-MEM path: $4 + 4 + 1 = \mathbf{9}$ cycles; with the plain detection unit two stalls: **10** cycles.

Past Exam Question (2016–17 Q4(a), 10 marks; 2014–15 Q3(a), 10 marks)

What are pipeline hazards? Explain a structural hazard with a time-space diagram. What type of hazard occurs in the code `lw $t1,0($t0); lw $t2,4($t0); add $t3,$t1,$t2; sw $t3,12($t0); lw $t4,8($t0); add $t5,$t1,$t4; sw $t5,16($t0)`? How can it be

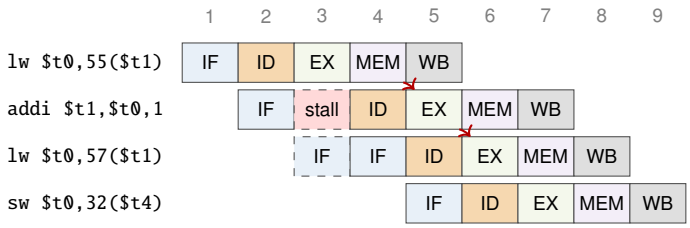


Figure 15.1: Timing with one load-use stall (9 cycles). The store's data is forwarded in the MEM stage.

overcome?

Solution. **Pipeline hazards** are situations in which the next instruction cannot execute in its next clock cycle: *structural* (the hardware resource it needs is busy), *data* (it needs a result not yet produced) and *control* (the instruction to fetch depends on a branch not yet decided).
Structural hazard example. Suppose there is a *single* memory for instructions and data. When the first `lw` is in its MEM stage (cycle 4), the fourth instruction would be in IF (cycle 4): both need the memory.

	1	2	3	4	5	6	7	8	9
`lw $t1`	IF	ID	EX	MEM	WB				
`lw $t2`		IF	ID	EX	MEM	WB			
`add $t3`			IF	ID	EX	MEM	WB		
`sw $t3`				IF→*stall*	IF	ID	EX	MEM	WB

The fourth instruction must wait one cycle (shaded): a bubble. The MIPS pipeline avoids this by having *separate instruction and data memories (or caches)*.
Hazard in the code: data hazards: `$t2` from the second `lw` to the `add` (load-use) and `$t4` from the last `lw` to the second `add` (load-use), plus forwardable ones (`add`→`sw`). *Overcoming:* forwarding for the ALU results; one stall each for the two load-use hazards; or reorder the code by moving `lw $t4,8($t0)` before the first `add`, which removes both stalls (11 cycles instead of 13).

Past Exam Question (undated paper Q3(d), Q4(a), (b), (c))

(a) What values are stored in the ID/EX pipeline register for (i) `lw $6,36($1)` and (ii) `add $5,$5,$5`? What do they do in EX and MEM? (b) What is a pipeline hazard; which types? (c) Give an example of a double data hazard. (d) *The scan of the next part, "Consider the following sequence of instructions", is cut off, so it is not answered here.*

Solution. **ID/EX register contents.** It holds the control signals for EX, MEM and WB, PC+4, the two register values read, the sign-extended 16-bit field, and the register numbers (Rs, Rt, Rd) needed for forwarding and the destination:

	`lw $6,36($1)`	`add $5,$5,$5`
EX controls: RegDst, ALUOp, ALUSrc	0, 00, 1	1, 10, 0
MEM controls: Branch, MemRead, MemWrite	0, 1, 0	0, 0, 0
WB controls: RegWrite, MemtoReg	1, 1	1, 0
Read data 1 / Read data 2	Reg[$1] / Reg[$6] (unused)	Reg[$5] / Reg[$5]
Sign-extended 16 bits	36	`0x2820` = 10272 (*rd* = 5, shamt 0, funct 32)
Rs, Rt, Rd (register numbers)	1, 6, 0	5, 5, 5

EX: `lw` computes the address Reg[$1]+36 with the ALU; `add` computes Reg[$5]+Reg[$5]= 2×Reg[$5]. **MEM:** `lw` reads data memory at that address (MemRead=1); `add` does nothing in

MEM (the ALU result just passes to MEM/WB).
(c) See the three consecutive `add $1,$1,$x` instructions of Chapter 11.

15.2 P5: Control hazards

Past Exam Question (2022–23 CSE 209 Q5, 15 marks; 2018–19 Q2(a), 10 marks)

```
beq $1, $2, TARGET # taken
lw $3, 40($4)
add $2, $3, $4
sw $2, 40($4)
TARGET: or $1, $1, $2
```

The branch decision is made with the ALU zero signal (no early-decision hardware, so it is resolved in MEM). Can a flush and a stall occur simultaneously? Will there be an issue? Explain with an illustration; if there is an issue, give a solution.

Solution. **Timeline.** `beq`: IF 1, ID 2, EX 3, MEM 4 (decision: taken). `lw`: IF 2, ID 3, EX 4. `add`: IF 3, **ID 4**: it uses `$3` loaded by `lw`, which is in EX in cycle 4: the hazard detection unit asks for a stall (load-use). **In cycle 4 the branch in MEM also wants to flush**: the three following instructions are on the wrong path and the PC must be loaded with TARGET.

	1	2	3	4	5	6
`beq`	IF	ID	EX	MEM (taken)	WB	
`lw`		IF	ID	EX	MEM	WB
`add`			IF	ID (load-use stall wanted)		
`sw`				IF (held by stall)		

Is there an issue? Yes. The stall logic says "freeze PC and IF/ID, insert a bubble" while the branch logic says "load PC with the target, turn IF/ID, ID/EX, EX/MEM into bubbles". If the stall wins, the PC is not updated in cycle 4, so TARGET is fetched one cycle late (and the held wrong-path instructions would remain). **Solution:** the flush *overrides* the stall: when the branch is taken (PCSrc=1) the stall is ignored (`PCWrite` and `IF/IDWrite` forced to 1, as the flushed instructions are wrong-path anyway), the PC is loaded with the target, and the wrong-path instructions `lw`, `add`, `sw` become nops. The stalled instruction is on the wrong path, so its stall is pointless. The target `or` is fetched in cycle 5.

Past Exam Question (2018–19 Q2(b), 10 marks)

`lw $1,100($2); add $1,$1,$2; beq $1,$2,TARGET`. Branch decision as early as possible (extra hardware). Discuss the issues and give the cycles.

Solution. With the branch decided in ID, `beq` needs `$1` at the start of its ID stage. `$1` is produced by `add` (immediately before: result at the end of its EX) and by the `lw` two before (but `add` overwrites it, so the `add` value is the one needed). Two issues: (i) the `add` depends on the load: **1 stall**; (ii) the `beq` depends on the `add` immediately before it: **1 stall** (then forward EX/MEM → ID comparator). Timeline: `lw` IF1 ID2 EX3 MEM4 WB5; `add` ID4 (stalled in 3), EX5 MEM6 WB7; `beq` ID6 (stalled in 5), EX7 MEM8 WB9. **9 cycles** in total (plus 1 more if the branch is taken and the target is to be fetched).

Past Exam Question (2021–22 Q1(c), 10; 2013–14 Q8(c), 9; 2017–18 Q2(c), 5)

What is a data hazard for branches? Give MIPS examples that need (i) no stall, (ii) one stall, (iii) two stalls to resolve (branch decided in ID).

Solution. A data hazard for a branch occurs when a register compared by the branch is produced by an instruction still in the pipeline, which is worse than for ordinary instructions because the branch needs it in ID.

Case	Example	Stalls
(i) producer 2nd/3rd before: forward	`add $1,$2,$3; add $4,$5,$6; nop; beq $1,$4,L` (the add $1 is 3 before)	0
(ii) ALU producer immediately before, or load 2nd before	`lw $1,addr; add $4,$5,$6; beq $1,$4,L`	1
(iii) load immediately before	`lw $1,addr; beq $1,$0,L`	2

Reason: an ALU result is available at the end of its EX, a loaded value at the end of its MEM, but the branch needs the operand at the start of its ID.

Past Exam Question (2023–24 Q1(b), 15 marks)

```
     lw $t0, 0($s0)
     add $t1, $t0, $s1
     beq $t1, $zero, L1
     add $t2, $t1, $s2
     bne $t2, $zero, L2
     sub $t3, $t2, $t1
L1: and $t4, $t1, $t2
L2: or $t5, $t3, $t4
```

Five-stage pipeline, full forwarding, branches resolved in ID with a 1-cycle penalty when taken, load-use costs 1 stall, no rescheduling. Find the stalls and total cycles if (a) neither branch is taken, (b) the first branch is taken, (c) the first is not taken and the second is taken.

Solution. Stalls caused by data (branch operand from the immediately preceding ALU instruction costs 1; load-use costs 1):

- `add $t1` uses `$t0` from the `lw` just before: 1 stall.
- `beq $t1,...` uses `$t1` from the `add` just before: 1 stall (branch in ID).
- `bne $t2,...` uses `$t2` from the `add $t2` just before: 1 stall.
- `sub`, `and`, `or`: the sources come from at least two instructions earlier, or from the preceding ALU instruction (forwarding): no stall.

Case	Executed path	Instr.	Stalls (data + taken)	Cycles = instr. +4+stalls
(a) NT, NT	1–8	8	$3+0$	$8+4+3=$ **15**
(b) first taken	1,2,3,and,or	5	$2+1$	$5+4+3=$ **12**
(c) NT, then taken	1–5,or	6	$3+1$	$6+4+4=$ **14**

In (b) the `bne` is never reached; the penalty of the taken branch is the 1 flushed instruction (the `add $t2` fetched behind the `beq`).

Past Exam Question (2022–23 CSE 305 Q4(b), 12 marks)

Which four components must be added in the ID stage to reduce the branch stalls? How are the decision and the target available by the end of ID?

Solution. (The scan names no figure, so this is the standard answer of slide 78.) Added to ID: (1) a **shift-left-2** unit applied to the sign-extended offset (the offset counts words), (2) a **branch**

target adder PC+4 + (offset×4), (3) an **equality comparator** on the two register outputs (bitwise XOR then NOR/OR), and (4) **flush logic** `IF.Flush` which turns the instruction in IF/ID into a `nop` when the branch is taken (plus forwarding paths into the comparator and the hazard unit for operands, as in the data-hazard table). Because the register file is read in the first half of ID, the comparator and adder finish before the end of ID; the PC is loaded with the target at the end of the cycle, so the target is fetched in the next cycle: only one wrong instruction (in IF) must be flushed.

Past Exam Question (2022–23 CSE 209 Q7, 15 marks (scan paraphrased))

Three friends discuss the jump instruction. Chandler claims a `j` always causes a 1-cycle stall in the pipeline. Joey proposes moving the control logic to the IF stage. Ross says there is a better solution. Evaluate.

Solution. *Caution: the exact statements were taken from my reading of the scanned page; the reasoning below stands on its own.*
Chandler is right. A jump is identified only when it is decoded in ID. By then the next sequential instruction is already in IF; it is wrong-path and must be flushed, so the target is fetched one cycle late: 1 bubble per jump (`j`, `jal`), the same as a branch decided in ID.
Joey's idea (decode in IF) is not attractive: the instruction only arrives at the end of the instruction-memory access, so decoding in the same stage lengthens the IF stage and hence the clock cycle of the whole pipeline, which costs more than the occasional bubble.
Ross's better solution: use a **branch target buffer** (or other predictor) indexed by the PC in IF: it recognises the jump and supplies its target in the very same cycle, so the correct instruction is fetched next with no bubble (a wrong entry is flushed like a misprediction). A cheaper alternative is to store predecode bits in the instruction cache. Delayed jumps (executing the next instruction regardless) also hide the bubble but are not a modern approach (slide 94 pitfall).

15.3 P6: Branch prediction

Past Exam Question (2021–22 Q1(d), 5; 2017–18 Q2(d), 5; 2013–14 Q8(d), 4; 2023–24 Q1(c), 10)

Nested loops: `outer: ... inner: ... beq $t0,$zero,inner ... beq $t1,$zero,outer`. What is the problem with a 1-bit branch predictor? How can you solve it? (2023–24 also: why does even a perfect predictor still cost a cycle?)

Solution. A 1-bit predictor remembers only the *last* outcome of the branch. The inner-loop branch is taken on every iteration but one (at the exit it is not taken). The predictor mispredicts **twice** per execution of the inner loop: (1) at the last iteration (predicts taken, actual not taken) and (2) at the first iteration of the next execution of the loop (bit says not taken, actual taken). For an inner loop of 10 iterations repeated 5 times this gives 9 mispredictions in 50 executions (1 at the very first exit, then 2 per later round), i.e. about 80% accuracy instead of 90%.
Solution: a 2-bit predictor (Figure 12.4) changes the prediction only after two successive mispredictions. At the loop exit the state falls from strongly taken to weakly taken but still predicts taken, so the first iteration of the next round is predicted correctly: **one** misprediction per round (5 in 50 executions: 90%). I verified these counts by simulation.
Perfect predictor and the 1-cycle penalty: predicting "taken" tells the processor *that* the branch is taken but not *where* to go; the target address is computed only in ID, so the target is fetched one cycle late even when the direction is predicted correctly. The cure is a **branch target buffer** (a cache of target addresses indexed by the PC during IF) so the target is available in the IF stage (2015–16 Q2(e), 2017–18 Q1(d)).

Past Exam Question (2019–20 Q8(b), 8 marks; 2014–15 Q3(b), 10 marks)

Draw the finite state machine for 2-bit dynamic branch prediction / describe dynamic branch prediction techniques.

Solution. The four-state machine is Figure 12.4: states *strongly taken* and *weakly taken* predict taken; *weakly not taken* and *strongly not taken* predict not taken. A taken outcome moves one state toward strongly taken, a not-taken outcome one state toward strongly not taken, saturating at the ends. The prediction therefore changes only after two successive mispredictions.
Techniques (from the lecture): (1) the *branch prediction buffer* (branch history table), a small table indexed by low bits of the branch address, holding the last outcome (1-bit) or a 2-bit counter; (2) the *2-bit predictor*; (3) the *branch target buffer* supplying the target in IF. The slides mention that real processors (Cortex-A8, Core i7) use *two-level* predictors; the details of correlating predictors are not in the slides.

Past Exam Question (undated paper Q4(e), 7 marks)

Branch outcomes repeat T, T, NT, T. What is the accuracy of the 2-bit predictor for this sequence, and in steady state if the pattern repeats forever? The predictor starts in the bottom-left state (predict not taken) of the figure.

Solution. The figure is not in the scan, so the exact meaning of "bottom left" is uncertain; I give both possible starting states (counter 0 = strongly not taken, 1 = weakly not taken; the counter moves $+1$ on T, -1 on NT):

Start state	Correct in the first pass T,T,NT,T	Steady state
strongly not taken (0)	0 of 4 (0%)	3 of 4 (75%)
weakly not taken (1)	2 of 4 (50%)	3 of 4 (75%)

Walk-through from *strongly not taken*: T: predict NT, wrong, $\rightarrow$ weakly NT; T: predict NT, wrong, $\rightarrow$ weakly T; NT: predict T, wrong, $\rightarrow$ weakly NT; T: predict NT, wrong, $\rightarrow$ weakly T. In steady state the counter oscillates between weakly taken and strongly taken, the NT is always mispredicted and the three T are correctly predicted: 75%. These figures were checked by a short simulation.

15.4 P7: Pipelined datapath and control

Past Exam Question (2019–20 Q8(a), 15 marks)

For `add`, `sw`, `beq`, give the values (0/1) of RegDst, Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite.

Solution.

	RegDst	Branch	MemRead	MemtoReg	MemWrite	ALUSrc	RegWrite
`add`	1	0	0	0	0	0	1
`sw`	X	0	0	X	1	1	0
`beq`	X	1	0	X	0	0	0

X means "don't care": the value does not matter because nothing is written to a register. `add`: destination field is Rd (RegDst=1), second ALU operand a register (ALUSrc=0), result from the

ALU (MemtoReg=0). `sw`: the ALU adds the base and the offset (ALUSrc=1), memory is written. `beq`: the ALU subtracts two registers (ALUSrc=0) and `Branch`=1 lets the zero flag change the PC. (MemRead for `sw`/`beq` may also be a don't care in some designs.)

Past Exam Question (2015–16 Q2(c) 10; 2013–14 Q8(a) 15; 2016–17 Q4(b) 15; 2014–15 Q3(c) 15)

Draw the complete MIPS pipelined datapath and control for `lw` with the pipeline registers; modify the single-cycle datapath step by step (pipeline registers, forwarding unit, hazard detection unit).

Solution. Draw the pipelined datapath of Chapter 10 with these modifications, in this order:

1. **Cut the datapath into 5 stages with four pipeline registers**: IF/ID (64 bits: PC+4 and the instruction), ID/EX (128 bits of data: PC+4, two register values, sign-extended offset), EX/MEM (97 bits: branch target, ALU result, store data, Zero flag), MEM/WB (64 bits: memory data, ALU result), each with the register numbers and plus the control bits.
2. **Pass the destination register number** down to WB, so that the write happens to the register of the *right* instruction rather than the one now in ID: the Write register input of the register file comes from MEM/WB.
3. **Generate control signals in ID** and carry them in the pipeline registers, grouped in EX (RegDst, ALUOp, ALUSrc), MEM (Branch, MemRead, MemWrite) and WB (RegWrite, MemtoReg); each stage uses its own group and passes the rest on.
4. **Add the forwarding unit** and the 3-input multiplexers on both ALU inputs.
5. **Add the hazard detection unit**: inputs `ID/EX.MemRead`, `ID/EX.RegisterRt`, `IF/ID.RegisterRs/Rt`; outputs `PCWrite`, `IF/IDWrite`, and a multiplexer that zeroes the control signals into ID/EX.

For `lw` only: IF: instruction fetched, PC+4 computed; ID: registers read, offset sign-extended, control signals set (RegDst=0, ALUSrc=1, MemRead=1, MemtoReg=1, RegWrite=1); EX: ALU adds base and offset; MEM: data memory read; WB: data written into register Rt (number carried from ID/EX through to MEM/WB).

15.5 P8: Multiple issue, scheduling and superscalar

Past Exam Question (2021–22 Q2(c), 10; 2023–24 Q4(b), 15)

Schedule the loop `Loop: lw $t0,0($s1); addu $t0,$t0,$s2; sw $t0,0($s1); addi $s1,$s1,-4; bne $s1,$zero,Loop` for the dual-issue MIPS (one ALU/branch + one load-/store per cycle). Find the IPC, and show what loop unrolling gives.

Solution. This is exactly the lecture example (slides 95–97). Explanation in Chapter 13: the plain schedule needs 4 cycles for 5 instructions, **IPC** $= 5/4 = 1.25$; four-fold unrolling with renamed registers `$t0`-`$t3` needs 8 cycles for 14 instructions, **IPC** $= 14/8 = 1.75$. Do not forget the offset change of the `sw` ($0 \to 4$) after moving the `addi` earlier, and the correction `addu $t3,$t3,$s2`.

Past Exam Question (2020–21 Q5, 15 marks)

```
Loop: lw $t0, 0($s0)
      lw $t1, 0($s1)
      addu $t0, $t0, $t1
      sw $t0, 0($s1)
      addi $s0, $s0, -4
```

```
addi $s1, $s1, -4
bne $s1, $zero, Loop
```

Schedule on the dual-issue MIPS to maximize IPC.

Solution. **Constraints.** Two loads need two load/store slots, so the second load is in cycle 2 at the earliest; `addu` needs it and, with a one-cycle load-use latency, goes in cycle 4 at the earliest; `sw` needs `addu`: cycle 5. Hence at least 5 cycles. Both `addi` can be placed in the free ALU slots of cycles 1 and 2 (the loads in the same packet read the *old* pointer values), and then the `sw` offset becomes 4:

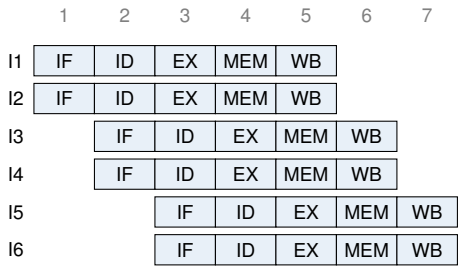
	ALU/branch	Load/store	Cycle
`Loop:`	`addi $s0,$s0,-4`	`lw $t0,0($s0)`	1
	`addi $s1,$s1,-4`	`lw $t1,0($s1)`	2
	`nop`	`nop`	3
	`addu $t0,$t0,$t1`	`nop`	4
	`bne $s1,$zero,Loop`	`sw $t0,4($s1)`	5

IPC $= 7/5 =$ **1.4** (peak 2). Check: `bne` in cycle 5 sees the new `$s1` (written in cycle 2); the `sw` gets `$t0` from the `addu` of the previous packet by forwarding; `sw` address `4($s1)` with the decremented `$s1` equals the original `0($s1)`. Unrolling would raise the IPC further, as in the lecture.

Past Exam Question (2021–22 Q2(b), 10; 2014–15 Q1(d), 3+8+2)

Draw the block diagram of a dynamically scheduled superscalar CPU; show the time-space diagram for pipelining in a superscalar processor; mention the differences between a superscalar and a VLIW processor.

Solution. **Block diagram:** slide 100 (Figure 13.5.1 in Chapter 13): in-order *instruction fetch and decode unit* → *reservation stations* in front of each functional unit (integer, floating point, load/store) → *out-of-order execution* → *commit unit* with a *reorder buffer*, which writes registers/memory in program order and can supply operands.
Time-space diagram of a two-issue pipeline: two instructions enter each stage in every cycle.



Six instructions finish in 7 cycles instead of 10 for a single pipeline (IPC up to 2 in steady state).
Superscalar vs. VLIW:

	Superscalar (dynamic)	VLIW (static)
Who finds parallelism	hardware at run time	compiler at compile time
Instruction format	ordinary scalar instructions	long packets with several operations (nop padding)
Hazards	resolved by the CPU	avoided by the compiler
Hardware	complex (reservation stations, reorder buffer, speculation)	simple
Compatibility	same code runs on a different machine	code is tied to the issue width and latencies

15.6 P9: CISC versus RISC

Past Exam Question (2016–17 Q4(c), 6 marks; 2014–15 Q1(c), 8 marks; 2013–14 (undated) Q2(e): three examples of complexity of IA-32)

Compare typical CISC and RISC architectures.

Solution.

	CISC (e.g. VAX, IA-32)	RISC (e.g. MIPS)
Instruction set	large, many complex instructions	small, simple, regular
Instruction length	variable (1 to 15 bytes in IA-32)	fixed (32 bits)
Addressing modes	many, complex (memory indirection, side effects)	few (register, base+offset, immediate, PC-relative, pseudodirect)
Memory operands	ALU instructions may access memory	only load/store access memory (load-store architecture)
Registers	fewer	many (32)
Control	often microprogrammed	hardwired
Pipelining	hard: IA-32 needs translation to micro-operations	easy, one instruction per cycle
Code size / CPI	smaller code, higher and variable CPI	larger code, CPI close to 1

The lecture's pitfall slide (109–110) states that poor ISA design, such as complex instruction sets (VAX, IA-32), makes pipelining harder: complex addressing modes, register update side effects and memory indirection; IA-32 overcomes it by the micro-op approach (instructions are translated into simple internal operations). Comparison details beyond this slide come from the standard CISC/RISC treatment.

PART III

Floating-Point Arithmetic

CHAPTER 16

Floating-Point Numbers: Representation

Chapter goal

Integers cannot represent 0.375, 3.14159 or 6×10^{23} in a fixed number of bits. *Floating point* solves this the way scientists do, with scientific notation. By the end of this chapter you can encode a decimal number in IEEE 754 single or double precision, decode a bit pattern back into a number, find the smallest and largest numbers of any format with given field sizes, and explain denormals, infinity and NaN. *Lecture source: Arithmetic slides 9–18 (§3.5).*

16.1 Prerequisites

16.1.1 Binary fractions

In the decimal number 13.625 each digit to the right of the point has weight $10^{-1}, 10^{-2}, 10^{-3}$. Binary works in the same way with powers of 2: after the **binary point** the weights are $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, …

Worked Example: Reading and writing binary fractions

$110.011_2 = 4 + 2 + 0 + 0 + 0.25 + 0.125 = 6.375$.
To convert the decimal fraction 0.375 to binary, repeatedly *double* the fraction and record the integer part of each result: $0.375 \times 2 = 0.75 \to 0$; $0.75 \times 2 = 1.5 \to 1$; $0.5 \times 2 = 1.0 \to 1$; stop when the fraction is 0. Reading the recorded digits from the top: $0.375 = 0.011_2$. Check: $0.25 + 0.125 = 0.375$. Some fractions never terminate: $0.1_{10} = 0.0001100110011\ldots_2$ (the group 0011 repeats forever), which is why 0.1 cannot be stored exactly in a binary floating-point format.

16.1.2 Scientific notation and normalization

Scientific notation writes a number as (significand)$\times$(base)^{exponent}, for instance -2.34×10^{56}. A number is **normalized** if there is exactly one non-zero digit before the point: -2.34×10^{56} is normalized, $+0.002 \times 10^{-4}$ and $+987.02 \times 10^{9}$ are not [FP slide 9]. In binary the only non-zero digit is 1, so a normalized binary number has the form

$$\pm 1.xxxxxxx_2 \times 2^{yyyy}.$$

Worked Example: Normalizing

$110.011_2 = 1.10011_2 \times 2^2$ (move the point 2 places left, exponent $+2$). $0.011_2 = 1.1_2 \times 2^{-2}$ (move the point 2 places right, exponent -2).

Every non-zero number has exactly one normalized form, so only three things must be stored: the sign, the exponent, and the digits after the point (the **fraction**).

16.2 The IEEE 754 standard [FP slide 10–11]

Before the standard (IEEE Std 754-1985) every computer maker used its own floating-point format, which made scientific programs non-portable. The standard is now almost universal and defines two main formats, **single precision** (32 bits, type `float` in C) and **double precision** (64 bits, type `double`).

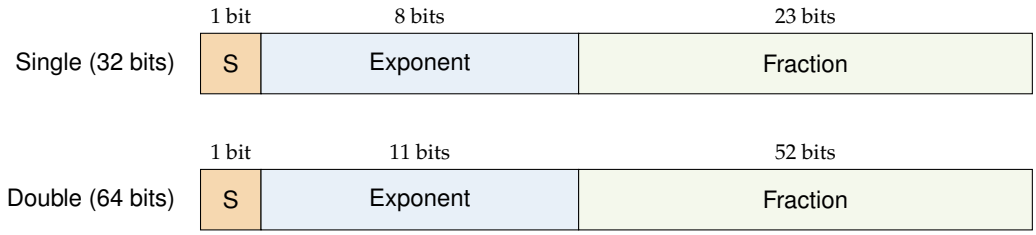


Figure 16.1: IEEE 754 field layout (the boxes are not to scale).

- **S (sign bit):** 0 means non-negative, 1 means negative. The number's size is stored separately from its sign (**sign–magnitude**), so $+x$ and $-x$ differ only in this bit.
- **Fraction:** the digits after the binary point of the normalized significand. Because a normalized number always starts with "1.", that leading 1 does not need to be stored: it is the **hidden bit**. The **significand** (the full number $1.xxx$ that is multiplied by the power of two) is the fraction with the "1." restored. Hidden bit: 23 stored bits give 24 significant bits in single precision.
- **Exponent:** stored in **excess** (biased) form: stored value = actual exponent + **bias**, with bias = 127 for single and 1023 for double precision.

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent}-\text{Bias})}.$$

16.2.1 Why a bias?

The exponent can be negative (numbers smaller than 1). Storing it in two's complement would make comparisons awkward. A bias is added so that the stored exponent is always an *unsigned* number: the smallest actual exponent maps to the smallest stored value. Then, because the exponent sits to the left of the fraction, the bit patterns of two positive floating-point numbers are ordered like unsigned integers (Section 16.6), and a sorting routine or integer comparator can be used directly.

16.3 Encoding and decoding

Worked Example: Encode -0.75 [FP slide 15]

Step 1: sign. The number is negative, so $S = 1$.
Step 2: binary and normalize. $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$, so the significand is 1.1_2 and the actual exponent is -1.
Step 3: fraction. Drop the hidden 1: Fraction $= 1000\ldots0$ (the single digit 1 followed by zeros up to 23 bits).
Step 4: exponent. Single: $-1 + 127 = 126 = 01111110_2$. Double: $-1 + 1023 = 1022 = 01111111110_2$.
Result. Single: `1 01111110 10000000000000000000000` = `0xBF400000`. Double: 1

`01111111110 1000...0` = `0xBFE8000000000000`.

Worked Example: Decode `11000000101000...0` [FP slide 16]

Split the 32 bits: $S = 1$, exponent field $= 10000001_2 = 129$, fraction $= 0100\ldots0_2$.
Actual exponent $= 129 - 127 = 2$. Significand $= 1 + 0.01_2 = 1.25$.
Value $= (-1)^1 \times 1.25 \times 2^2 = -5.0$.

Worked Example: A number that is not exact: 0.1

$0.1 = 1.1001100110011\ldots_2 \times 2^{-4}$ (the pattern 1001 repeats). The stored 23 fraction bits are rounded: single-precision 0.1 is `0x3DCCCCCD`, whose exact value is 0.100000001490116.... Floating point stores a *nearby* number, not always the exact one.

Encoding recipe (any format with E exponent bits and F fraction bits).

1. Sign bit from the sign of the number; work with the absolute value.
2. Convert to binary and normalize to $1.f \times 2^e$.
3. Exponent field $= e +$ bias, written with E bits; bias $= 2^{E-1} - 1$.
4. Fraction field $= f$, padded with zeros (or rounded) to F bits.

Common mistake

Do not forget the hidden bit: for 1.25 the fraction field is `01000...` (for the digits after the point), not `101000...`. Do not store the biased exponent in two's complement, and do not apply the bias to the fraction.

16.4 Range and precision [FP slide 12–14]

16.4.1 The reserved exponents

Two exponent fields are not used for ordinary numbers: all zeros (denormals and zero) and all ones (infinity and NaN, Section 16.5). So normalized numbers use the stored exponents $1, \ldots, 2^E - 2$.

16.4.2 Single precision

- **Smallest normalized:** exponent field 00000001, actual exponent $1 - 127 = -126$, fraction 0 (significand 1.0): $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$.
- **Largest:** exponent field 11111110 $= 254$, actual exponent $254 - 127 = +127$, fraction all ones (significand $= 2 - 2^{-23} \approx 2.0$): $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$. (The slide writes "$2.0 \times 2^{127}$"; the exact value is slightly below that.)

16.4.3 Double precision

Smallest: $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$ (exponent field 00000000001). Largest: exponent field 11111111110 $= 2046$, actual exponent $2046 - 1023 = +1023$: $\pm(2 - 2^{-52}) \times 2^{1023} \approx \pm 1.8 \times 10^{308}$.

16.4.4 Precision

All fraction bits are significant, so the relative precision (the gap between neighbouring numbers divided by the size of the number) is about 2^{-23} in single precision and 2^{-52} in double. Converting bits to decimal digits with $\log_{10} 2 \approx 0.3$: single $23 \times 0.3 \approx 6$ decimal digits, double $52 \times 0.3 \approx 16$ digits (exact values: 6.9 and 15.6 using the hidden bit's 24 and 53 bits).

	Single	Double
bias	127	1023
smallest normalized	$2^{-126} \approx 1.18 \times 10^{-38}$	$2^{-1022} \approx 2.23 \times 10^{-308}$
largest	$(2 - 2^{-23})2^{127} \approx 3.40 \times 10^{38}$	$(2 - 2^{-52})2^{1023} \approx 1.80 \times 10^{308}$
smallest denormal	$2^{-149} \approx 1.4 \times 10^{-45}$	$2^{-1074} \approx 4.9 \times 10^{-324}$

Figure 16.2: Summary of the ranges (values checked with a short Python script).

16.5 Denormals, zero, infinity and NaN [FP slide 17–18]

Using the formula $(1 + \text{fraction}) \times 2^e$ for all patterns leaves a hole around zero: the smallest normalized number is 2^{-126}, and the next smaller number representable is 0. Also, the number zero cannot be written as $1.x \times 2^e$. The standard therefore gives special meanings to the reserved exponent fields:

Exponent field	Fraction	Meaning	Value
1 to $2^E - 2$	anything	normalized number	$(-1)^S(1 + \text{fraction})2^{\text{exp}-\text{bias}}$
0 (all zeros)	0	zero (± 0)	± 0.0
0 (all zeros)	$\neq 0$	**denormal**	$(-1)^S(0 + \text{fraction}) \times 2^{1-\text{bias}}$
all ones	0	infinity	$\pm\infty$
all ones	$\neq 0$	NaN	not a number

Denormal numbers. With exponent field 0 the hidden bit is taken to be **0**, not 1, and the exponent is fixed at $1 - \text{bias}$ (not $0 - \text{bias}$, so that denormals continue smoothly from the smallest normalized number). They are smaller than normalized numbers and allow **gradual underflow**: as numbers shrink below the normalized range the precision decreases gradually instead of the result suddenly becoming 0. The smallest positive denormal has fraction $0 \ldots 01$: $2^{-23} \times 2^{-126} = 2^{-149}$ in single precision. A denormal with fraction 0 is zero; since the sign bit is free there are **two zeros**, $+0.0$ and -0.0 [FP slide 17].

Infinity. Exponent all ones with fraction 0 is $\pm\infty$. It is the result of overflow (and of dividing a non-zero number by 0), and can be used in further calculations ($1/\infty = 0$), avoiding an explicit overflow check after every operation. Single precision: $+\infty =$ `0x7F800000`, $-\infty =$ `0xFF800000`.

NaN (Not-a-Number). Exponent all ones with a non-zero fraction indicates an illegal or undefined result such as $0.0/0.0$ or $\infty - \infty$. It can be used in subsequent calculations (the result stays NaN) [FP slide 18].

Worked Example: A toy format shows everything at once

Take a tiny 6-bit format with 1 sign bit, $E = 3$ exponent bits (bias $2^2 - 1 = 3$) and $F = 2$ fraction bits. Positive values:

Exponent field	Fraction 00, 01, 10, 11	Kind
000	0, 0.0625, 0.125, 0.1875	zero and denormals (frac/4 $\times 2^{-2}$)
001	0.25, 0.3125, 0.375, 0.4375	normalized, 2^{-2}
010	0.5, 0.625, 0.75, 0.875	2^{-1}
011	1, 1.25, 1.5, 1.75	2^{0}
100	2, 2.5, 3, 3.5	2^{1}
101	4, 5, 6, 7	2^{2}
110	8, 10, 12, 14	2^{3}
111	∞, NaN, NaN, NaN	reserved

Observe: (i) the *gap* between neighbours doubles every time the exponent increases: it is $1/16$ near 0 but 2 near 14: the relative precision is constant, the absolute one is not; (ii) the denormals (gap $1/16$) fill the hole between 0 and the smallest normalized number $1/4$ with the same gap as the first normalized range; (iii) the largest finite number is $14 = (2 - 2^{-2}) \times 2^3$, the smallest normalized is $\frac{1}{4} = 2^{1-3}$.

16.6 Comparing floating-point numbers as integers

Because the exponent is stored before the fraction and is biased to be unsigned, a larger positive float has a larger bit pattern. For two *non-negative* floats $f_1 < f_2$, the integers $i_1 < i_2$ whether read as signed or unsigned (the sign bit is 0). With negative numbers the sign–magnitude representation spoils this: a negative float has its top bit set. This is examined in the past questions (Chapter 18).

16.7 General formulae for any format

For a format with 1 sign bit, E exponent bits and F fraction bits (IEEE-style conventions, as assumed in the exam questions):

Bias	$2^{E-1} - 1$
Exponent field of normalized numbers	$1, \ldots, 2^E - 2$; actual exponents $1 - \text{bias} \ldots 2^E - 2 - \text{bias}$
Smallest normalized	$1.0 \times 2^{1-\text{bias}}$
Largest	$(2 - 2^{-F}) \times 2^{2^E - 2 - \text{bias}}$
Smallest denormal	$2^{-F} \times 2^{1-\text{bias}}$
Decimal digits of precision	$\approx (F+1) \log_{10} 2$ (or $F \log_{10} 2$ if you ignore the hidden bit, as the slides do)

Check with single precision ($E = 8, F = 23$): bias 127; smallest 2^{-126}; largest $(2 - 2^{-23})2^{127}$; smallest denormal 2^{-149}. *(These general formulae follow the standard IEEE convention; the slides give the single and double cases only.)*

16.8 Practice questions

Past Exam Question (practice)

(a) Encode 6.25 in IEEE single precision. (b) Decode `0x40490000`. (c) In the toy format of the example above, what is the value of the pattern `0 100 10`? (d) Which toy-format pattern has the value -0.3125?

Solution. (a) $6.25 = 110.01_2 = 1.1001_2 \times 2^2$; exponent field $2 + 127 = 129 = 10000001$; fraction `1001000...0`. Bits: `0 10000001 10010000000000000000000` = `0x40C80000` (checked by script).
(b) In the fields S, exponent, fraction: `0 10000000 10010010000000000000000`. Exponent: $128 - 127 = 1$. Fraction `1001001`$\ldots = 0.5 + 0.0625 + 0.0078125 = 0.5703125$, so the significand is 1.5703125 and the value is $1.5703125 \times 2 = 3.140625$ (checked by script).
(c) Exponent field $100 = 4$, actual $4 - 3 = 1$; significand $1 + 0.10_2 = 1.5$; value $1.5 \times 2 = 3$.
(d) $0.3125 = 0.0101_2 = 1.01_2 \times 2^{-2}$; exponent field $-2 + 3 = 1 = 001$; fraction 01; sign 1: `1 001 01`.

Lecture source

Arithmetic slides 9–18 (§3.5 Floating Point). The toy-format example, the general formulae for arbitrary E, F, and the remark on non-terminating fractions are added explanations (standard IEEE 754 conventions).

CHAPTER 17

Floating-Point Addition

Chapter goal

Adding two integers is a single binary addition. Adding two floating-point numbers is harder because the numbers have different exponents: their binary points are not lined up. This chapter explains the four-step algorithm, works through decimal and binary examples, shows the hardware, states the overflow and underflow conditions, and explains the extra bits used for correct rounding. *Lecture source: Arithmetic slides 19–22.*

17.1 The idea

You cannot add 9.999×10^{1} and 1.610×10^{-1} digit by digit, because the digits have different weights. The way you do it by hand is the algorithm: first line up the decimal points, then add, then tidy up the result. The same four steps are used in binary.

1. **Align**: shift the significand of the number with the *smaller* exponent to the right until the exponents are equal (increase its exponent by one for every place it is shifted).
2. **Add** the significands (with the signs taken into account).
3. **Normalize** the result and **check for overflow or underflow**.
4. **Round** the result to the available number of digits, and renormalize if rounding broke the normalization.

Why shift the *smaller* number? Shifting right loses only the least significant digits of the smaller number. Shifting the larger number would push its most important digits out of the register.

17.2 A decimal example [FP slide 19]

Four significant decimal digits: $9.999 \times 10^{1} + 1.610 \times 10^{-1}$.

Step	Work	Result
1. Align	the smaller exponent is -1; shift 1.610×10^{-1} right by 2 places to exponent 1: 0.01610×10^{1}, kept to 4 digits: 0.016×10^{1}	$9.999 \times 10^{1} + 0.016 \times 10^{1}$
2. Add	$9.999 + 0.016 = 10.015$	10.015×10^{1}
3. Normalize	shift right 1 place, exponent $1 + 1 = 2$; no over/underflow	1.0015×10^{2}
4. Round	only 4 digits are kept: $1.0015 \to 1.002$	1.002×10^{2}

The exact answer is 100.151, the computed one 100.2 (4 significant digits): the error comes from the digits discarded in steps 1 and 4.

17.3 A binary example [FP slide 20]

Four significant binary digits (a 1 before the point and 3 fraction bits):

$$1.000_2 \times 2^{-1} + (-1.110_2 \times 2^{-2}) \qquad (0.5 + (-0.4375)).$$

1. **Align**: the smaller exponent is -2. Shift $-1.110_2 \times 2^{-2}$ right one place: $-0.111_2 \times 2^{-1}$. Now both have exponent -1.
2. **Add** significands: $1.000_2 + (-0.111_2) = 0.001_2$, so the sum is $0.001_2 \times 2^{-1}$.
3. **Normalize**: shift the significand *left* until a single 1 is before the point: three places gives 1.000_2, and the exponent decreases by 3: $1.000_2 \times 2^{-4}$. The exponent -4 is within the allowed range, so there is no overflow or underflow.
4. **Round**: 1.000_2 already fits in 4 digits; no change. Result $1.000_2 \times 2^{-4} = 0.0625$.

Check: $0.5 - 0.4375 = 0.0625$. In IEEE single precision the same operands are `0x3F000000` and `0xBEE00000` and the sum is `0x3D800000` (checked with a script).

Common mistake

When normalizing a result that came out smaller than 1, shift left and *decrease* the exponent; when it came out as $10.xx$ shift right and *increase* the exponent. A frequent mistake is to adjust the exponent in the wrong direction.

17.4 Sign handling

In IEEE format the significands are stored as sign–magnitude. If the two signs are equal, add the magnitudes and keep the sign. If they differ, *subtract* the smaller magnitude from the larger and give the result the sign of the larger magnitude (this is what happens in the example above: the -0.111 is subtracted from 1.000, and the result is positive because $0.5 > 0.4375$). The hardware does this by complementing the significand of the number to be subtracted when the signs differ.

17.5 The flowchart

17.6 Overflow and underflow [FP slide 19]

After step 3 the exponent of the result may be outside the representable range:

- **Overflow** occurs when the exponent of the result is *too large* (a positive exponent greater than the maximum: for single precision, actual exponent > 127, i.e. stored exponent > 254). The result is too large in magnitude to represent; IEEE gives $\pm\infty$ (or raises an exception).
- **Underflow** occurs when the exponent of the result is *too small* (a negative exponent less than the minimum: actual exponent < -126 in single precision). The result is non-zero but smaller in magnitude than the smallest normalized number; IEEE gives a denormal number or zero.

Overflow can only happen after normalization when adding two numbers of the same sign (the exponent grows by one), or after rounding up when the number was already the largest ($1.11\ldots1_2 \times 2^{127}$ rounds to 2^{128}). Underflow can happen when subtracting two nearly equal numbers: the difference is tiny and normalization decreases the exponent a lot.

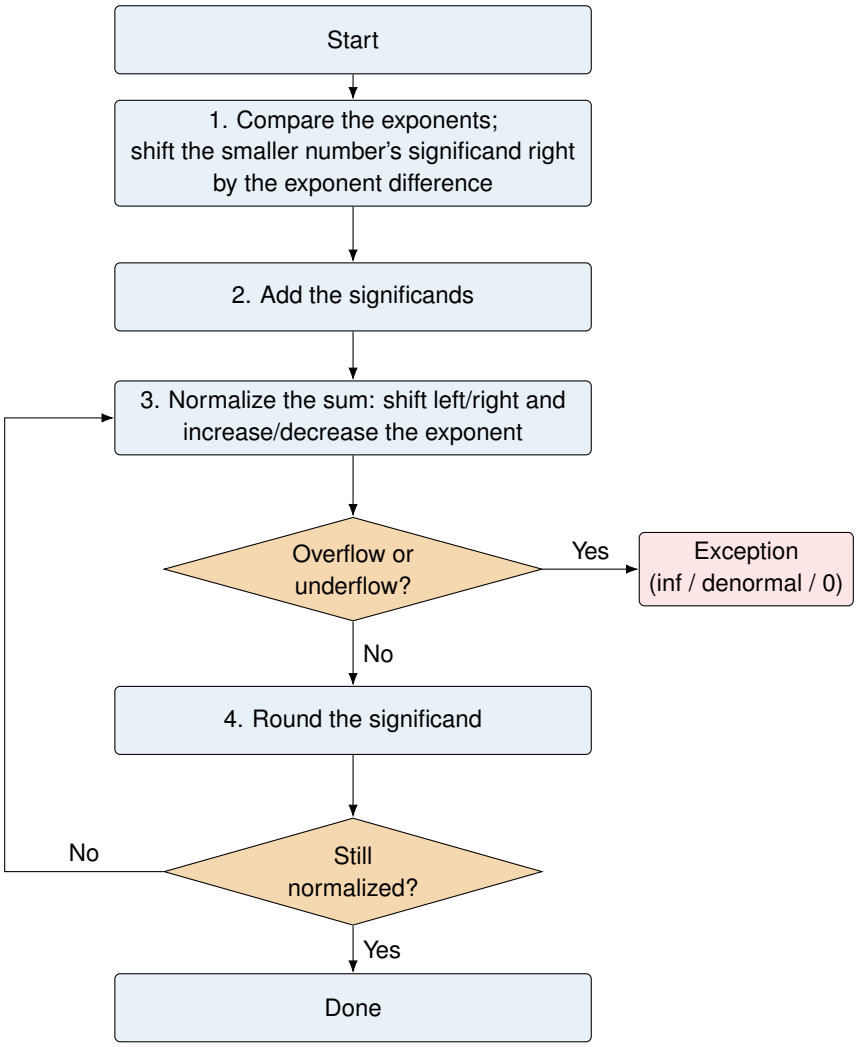


Figure 17.1: Flowchart of floating-point addition (after slides 19–22 and Patterson and Hennessy).

Other special cases. If either operand is NaN the result is NaN; $\infty + x = \infty$; $\infty + (-\infty) =$ NaN; $x + 0 = x$.

17.7 Floating-point adder hardware [FP slide 21–22]

A floating-point adder is much more complex than an integer adder. Doing the whole addition in one clock cycle would take too long (much longer than an integer addition) and a slower clock would penalize every other instruction. So the FP adder normally takes **several cycles**, and it can be *pipelined*. The four steps of the algorithm correspond to the four blocks of the figure:

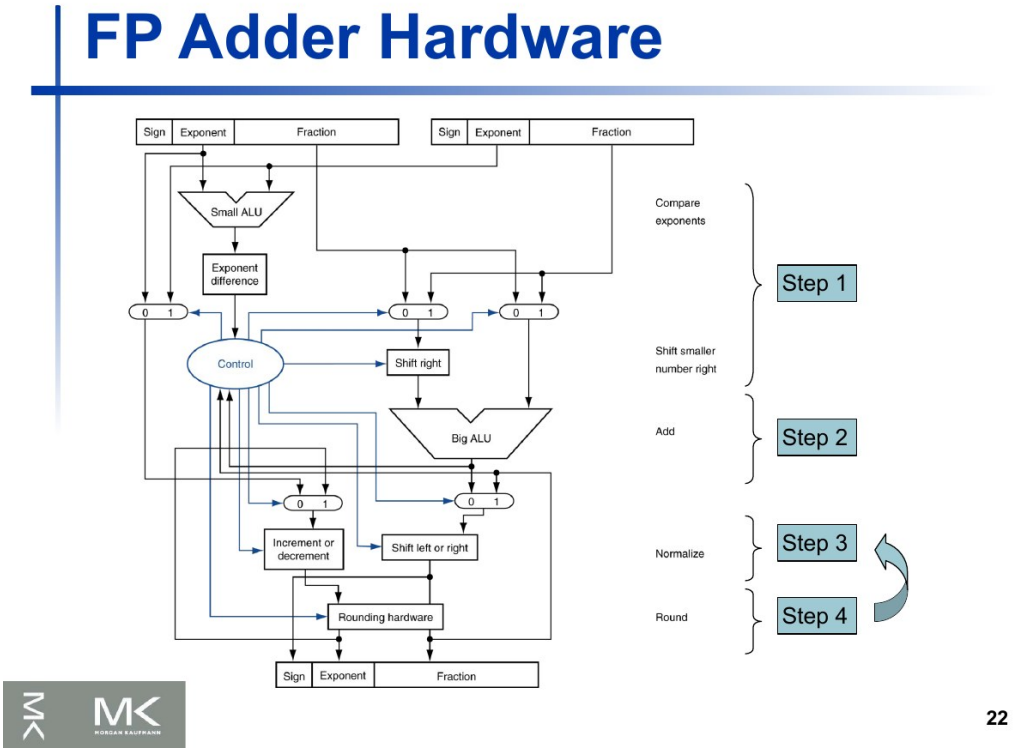


Figure 17.2: Floating-point adder hardware (slide 22). The figure is read from the top: Step 1 (exponent compare and shift), Step 2 (add), Step 3 (normalize), Step 4 (round).

Step 1 A small ALU subtracts the two exponents; the sign of the difference and its magnitude drive multiplexers that select the *larger exponent* and the significand of the number with the *smaller* exponent; the *shift right* unit shifts this significand by the difference.

Step 2 The "big ALU" adds (or subtracts) the significands.

Step 3 A normalization block shifts the result left or right (increment/decrement logic adjusts the exponent) and checks for overflow/underflow.

Step 4 The rounding hardware rounds the significand; if rounding produced a non-normalized number, the result goes through step 3 again.

17.8 Extra bits for accurate rounding

(This section goes beyond the slides but is required by one past question. The explanation follows the standard IEEE 754 approach.) In the alignment step the smaller number is shifted right and some

of its low-order bits would be discarded, making the sum wrong in the last digit. The hardware keeps extra bits to the right of the significand:

- **Guard bit**: the first extra bit to the right of the least significant bit of the (aligned) significand.
- **Round bit**: the second extra bit.
- **Sticky bit**: a bit that is the OR of *all* bits shifted out beyond the round bit; once set it stays set.

Worked Example: Why guard and round digits are needed, decimal analogue

Three significant digits: $6.78 \times 10^3 + 4.56 \times 10^1$. The exact sum is 6825.6, which rounds to 6.83×10^3. Without extra digits, 4.56×10^1 is shifted to 0.04×10^3 (the digits 56 are lost), giving $6.78 + 0.04 = 6.82 \times 10^3$: wrong in the last place. With a guard digit (5) and a round digit (6), the shifted number is 0.0456×10^3, the sum is 6.8256, and rounding gives 6.83×10^3.

Worked Example: Why a sticky bit is needed

Binary with 3 fraction bits: $1.000_2 \times 2^0 + 1.0000001_2 \times 2^{-4}$. The exact sum is 1.00010000001_2. Its bits after the 3 fraction bits are: guard $= 1$, round $= 0$, sticky $= 1$ (a later 1). The remainder is a little *more* than half a unit in the last place, so the correct rounded result is 1.001_2. With only guard and round bits the remainder looks like exactly half (guard 1, round 0) and "round to even" would wrongly round down to 1.000_2. The sticky bit records that something non-zero was shifted out, so the remainder is known to be more than half.

With guard, round and sticky bits the result is the same as if the addition were done exactly and then rounded, which is the accuracy the IEEE standard requires.

17.9 Floating-point addition is not associative

Because every addition rounds, the order of additions matters. In single precision, with $a = 10^8$, $b = -10^8$, $c = 1$: $(a + b) + c = 0 + 1 = 1$, but $a + (b + c)$: $b + c = -10^8 + 1$ is rounded back to -10^8 (the gap between neighbouring single-precision numbers near 10^8 is 8), so the result is 0 (checked in float32). A related fact: in single precision $2^{24} + 1 = 2^{24}$ (Chapter 18).

17.10 Practice question

Past Exam Question (practice)

Add $1.100_2 \times 2^3$ and $1.001_2 \times 2^1$ (4 significant binary digits, round to nearest).

Solution. 1. Align: the smaller exponent is 1; shift 1.001_2 right by 2 places: $0.01001_2 \times 2^3$ (keep the extra digits for now). 2. Add: $1.10000_2 + 0.01001_2 = 1.11001_2$, times 2^3. 3. Normalize: already normalized, exponent 3, no overflow. 4. Round to 3 fraction bits: $1.11001 \to 1.110$ (the bits `01` dropped are below half). Result $1.110_2 \times 2^3 = 14$. Check: $12 + 2.25 = 14.25$; the 4-digit result 14 differs from 14.25 by the discarded bits.

Lecture source

Arithmetic slides 19–22. The flowchart is drawn to match the four steps of the slides; the guard/round/sticky material, the special cases, and the non-associativity example are added explanations.

CHAPTER 18

Floating Point: Solved Past Examination Questions

How this chapter is organised

All questions come from the supplied CSE 305 / CSE 209 papers (2012–13 to 2023–24) and are grouped by topic: **F1** bias and special encodings, **F2** ranges of formats, **F3** encoding and decoding, **F4** integers in floating point and comparisons, **F5** floating-point addition, **F6** MIPS floating-point code (outside the slides). All numbers were recomputed with a Python script (exact fractions for custom formats, and `struct`/NumPy for IEEE single precision); bit patterns below were produced or checked by it.

18.1 F1: Bias and special encodings

Past Exam Question (2012–13 Q8(a), 5 marks; 2015–16 Q7(c), 10; 2018–19 Q7(c), 5; 2020–21 Q13(a)(i)(ii), 8)

What is the purpose of biased notation for the exponent? What is the encoding of floating-point infinity?

Solution. **Purpose of the bias.** The exponent must represent both positive and negative powers of two. Adding a constant *bias* to the actual exponent makes the stored exponent an *unsigned* number: the most negative exponent is stored as the smallest value. Consequences: (i) no separate sign handling for the exponent; (ii) because the exponent is the most significant part after the sign bit, two positive floating-point numbers can be compared as unsigned integers with ordinary integer hardware; (iii) the all-zeros and all-ones exponent fields are left free for zero/denormals and infinity/NaN. Single precision bias $= 127$, double $= 1023$; for E exponent bits the bias is $2^{E-1} - 1$ (for 7 bits: 63; for 4 bits: 7).
Infinity. Exponent field all ones, fraction all zeros: $+\infty =$ `0 11111111 000...0` $=$ `0x7F800000`, $-\infty =$ `0xFF800000` in single precision (double: exponent 11 ones, fraction 52 zeros). Non-zero fraction with the same exponent is NaN.

18.2 F2: Range of a format

Past Exam Question (2023–24 Q7(b), 15 marks)

Explain the use of bias and give the smallest and largest numbers in single and double precision.

Solution. Bias: as above. Ranges (normalized, positive):

	Smallest	Largest
Single (bias 127)	$1.0 \times 2^{1-127} = 2^{-126} \approx 1.18 \times 10^{-38}$	$(2 - 2^{-23}) \times 2^{254-127} = (2 - 2^{-23})2^{127} \approx 3.40 \times 10^{38}$
Double (bias 1023)	$2^{-1022} \approx 2.23 \times 10^{-308}$	$(2 - 2^{-52})2^{1023} \approx 1.80 \times 10^{308}$

The smallest exponent field used for normalized numbers is 00000001 (single) because 00000000 is reserved for zero and denormals; the largest is $11111110 = 254$ because 11111111 is reserved for ∞ and NaN. If denormals are counted, the smallest positive values are $2^{-149} \approx 1.4 \times 10^{-45}$ (single) and $2^{-1074} \approx 4.9 \times 10^{-324}$ (double).

Past Exam Question (2022–23 CSE 305 Q8(b), 8 marks)

A 32-bit floating-point format has 19 fraction bits, 1 sign bit and 12 exponent bits. Find its precision, range, smallest and largest numbers and smallest denormal.

Solution. $E = 12$, $F = 19$ ($1 + 12 + 19 = 32$). Bias $= 2^{11} - 1 = 2047$. Normalized exponent fields $1, \ldots, 4094$, actual exponents $-2046, \ldots, +2047$.

- **Smallest normalized:** $1.0 \times 2^{-2046} \approx 1.24 \times 10^{-616}$.
- **Largest:** $(2 - 2^{-19}) \times 2^{2047} \approx 2^{2048} \approx 3.23 \times 10^{616}$.
- **Range:** about 1.2×10^{-616} to 3.2×10^{616} in magnitude (either sign).
- **Precision:** 19 stored fraction bits give a relative precision of $2^{-19} \approx 1.9 \times 10^{-6}$, i.e. $19 \times 0.3 \approx 5.7$ decimal digits (about 6 digits counting the hidden bit, $20 \times 0.301 = 6.0$).
- **Smallest denormal:** fraction $0 \ldots 01$, exponent $1 - 2047 = -2046$: $2^{-19} \times 2^{-2046} = 2^{-2065} \approx 2.36 \times 10^{-622}$.

(Compared with IEEE single precision, this format trades 4 bits of precision for a vastly larger range.)

Past Exam Question (2020–21 Q13(a), 16 marks)

A 32-bit system uses 24 bits for the fraction, 1 for the sign and the rest for the exponent. (i) Why is bias used? (ii) What should the bias be and why? (iii) Smallest and largest absolute values? (iv) Smallest denormal?

Solution. Exponent bits: $32 - 24 - 1 = 7$. (i) See F1. (ii) With 7 bits the field runs $0 \ldots 127$; the fields 0 and 127 are reserved, so normalized fields are $1 \ldots 126$. Choosing bias $= 2^6 - 1 = \mathbf{63}$ makes the actual exponents $-62, \ldots, +63$, almost symmetric around 0 (and the standard IEEE choice). (iii) Smallest normalized: $1.0 \times 2^{-62} \approx 2.17 \times 10^{-19}$. Largest: $(2 - 2^{-24}) \times 2^{63} \approx 2^{64} \approx 1.84 \times 10^{19}$. (iv) Smallest denormal: $2^{-24} \times 2^{1-63} = 2^{-86} \approx 1.29 \times 10^{-26}$.

Past Exam Question (2019–20 Q4(a), 15 marks)

A 16-bit floating-point system has a largest absolute value of $1.11 \ldots 1_2 \times 2^{63}$ and a smallest absolute (normalized) value of $1.00 \ldots 0_2 \times 2^{-62}$. (i) Show the format. (ii) Represent $110.11_2 \times 2^1$ (assume an appropriate bias). (iii) Smallest denormal?

Solution. (The scan of (ii) is garbled; I read it as $110.11_2 \times 2^1$, and also give the alternative reading $10.11_2 \times 2^1$.)
(i) Format. The actual exponents run from -62 to $+63$: that is $63 + 62 + 1 = 126$ different exponent values, and these are the stored values $1, \ldots, 126$ (0 and 127 are reserved). So the exponent has **7 bits** ($2^7 - 2 = 126$) with bias 63 (stored $1 \leftrightarrow -62$, stored $126 \leftrightarrow +63$). The

remaining bits: $16 - 1 - 7 = \mathbf{8}$ fraction bits. Format: 1 sign + 7 exponent + 8 fraction. Largest $= (2 - 2^{-8})2^{63} \approx 1.84 \times 10^{19}$.
(ii) $110.11_2 \times 2^1 = 1101.1_2 = 13.5 = 1.1011_2 \times 2^3$. Sign 0; exponent $3 + 63 = 66 = 1000010_2$; fraction `10110000`. Bits `0 1000010 10110000` = `0x42B0`. (Alternative reading $10.11_2 \times 2^1 = 5.5 = 1.011_2 \times 2^2$: exponent $65 = 1000001$, fraction `01100000`: `0 1000001 01100000` = `0x4160`.)
(iii) Smallest denormal: $2^{-8} \times 2^{1-63} = 2^{-70} \approx 8.47 \times 10^{-22}$.

18.3 F3: Encoding and decoding

Past Exam Question (2013–14 Q3(b), 10 marks)

Show the IEEE 754 single-precision representation of -0.75.

Solution. $-0.75 = -1.1_2 \times 2^{-1}$: $S = 1$, exponent $-1 + 127 = 126 = 01111110$, fraction `1000...0`. Bits: `1 01111110 10000000000000000000000` = `0xBF400000` (script confirms). Double: `0xBFE8000000000000`.

Past Exam Question (2018–19 Q5(b), 8 marks)

A 16-bit floating-point system has a 4-bit exponent. Represent -1.25_{10} and 0.375_{10} using an appropriate bias.

Solution. Format: 1 sign, 4 exponent, $16 - 5 = 11$ fraction bits; bias $2^3 - 1 = 7$.
$-1.25 = -1.01_2 \times 2^0$: $S = 1$, exponent $0 + 7 = 7 = 0111$, fraction `01000000000`: `1 0111 01000000000` = `0xBA00`.
$0.375 = 0.011_2 = 1.1_2 \times 2^{-2}$: $S = 0$, exponent $-2 + 7 = 5 = 0101$, fraction `10000000000`: `0 0101 10000000000` = `0x2C00`.
(Both decoded back by script to -1.25 and 0.375.)

Past Exam Question (2018–19 Q5(c), 5 marks)

Determine the decimal value of the IEEE 754 single-precision number `0xDF807500`.

Solution. Binary: `1101 1111 1000 0000 0111 0101 0000 0000`.
Sign = 1 (negative). Exponent = next 8 bits = $10111111_2 = 191$; actual exponent $191 - 127 = 64$.
Fraction = last 23 bits = $000\,0000\,0111\,0101\,0000\,0000_2$ = `0x007500` = 29952, so the significand is $1 + 29952/2^{23} = 1 + 117/32768 \approx 1.003570557$.
Value $= -1.003570557 \times 2^{64} = -(32885) \times 2^{49} \approx -\mathbf{1.851} \times \mathbf{10^{19}}$ (script: $-18\,512\,609\,218\,259\,845\,120$).

Past Exam Question (2017–18 Q5(c), 5 marks)

Interpret the single-precision pattern `0x7F800005`.

Solution. `0 11111111 00000000000000000000101`: the exponent field is all ones (255) and the fraction ($= 5$) is non-zero, so the value is **NaN** (not a number), not a finite number or infinity. (For infinity the fraction would have to be zero, `0x7F800000`.)

18.4 F4: Integers in floating point; comparing floats

Past Exam Question (2021–22 Q8(b), 14 marks (as I read the scan))

In IEEE single precision, up to which integer n are all the consecutive integers $0, 1, \ldots, n$ exactly representable?

Solution. Single precision has a 24-bit significand (23 stored bits plus the hidden 1). An integer in $[2^k, 2^{k+1})$ is $1.xxx \times 2^k$ and needs k fraction bits, so it fits when $k \leq 23$: all integers up to $2^{24} - 1$ are exact, and $2^{24} = 1.0 \times 2^{24}$ is exact too. The next integer, $2^{24} + 1 = 1.00\ldots01_2 \times 2^{24}$, would need 24 fraction bits (25 significant bits), so it is *not* representable; it rounds to 2^{24} (float32 check: $2^{24} + 1 \to 16777216$). Above 2^{24} the gap between neighbouring floats is 2 (above 2^{25} it is 4, and so on). So the largest such n is $\mathbf{2^{24} = 16\,777\,216}$. (For double precision it is 2^{53}.)

Past Exam Question (2022–23 CSE 305 Q8(c), 20 marks)

f_1 and f_2 are IEEE single-precision numbers with $f_1 < f_2$. Their bit patterns are read (a) as unsigned integers i_1, i_2, (b) as signed (two's-complement) integers j_1, j_2. Is $i_1 < i_2$? Is $j_1 < j_2$?

Solution. The pattern is `S | exponent | fraction`. For positive numbers a bigger exponent, or an equal exponent and bigger fraction, means a bigger number and a bigger bit pattern, so order is preserved. The sign bit spoils it for negative numbers. Cases:

Case ($f_1 < f_2$)	Unsigned $i_1 < i_2$?	Signed $j_1 < j_2$?
both ≥ 0	yes	yes
$f_1 < 0 \leq f_2$	**no** (i_1 has the top bit set, so $i_1 > i_2$)	yes ($j_1 < 0 \leq j_2$)
both < 0	**no** (larger magnitude = larger pattern, so $i_1 > i_2$)	**no** (negative floats use sign–magnitude, so more magnitude bits means a *larger* signed value: $j_1 > j_2$)

Answer: unsigned comparison is correct only when both numbers are non-negative; signed comparison is correct unless both numbers are negative. I confirmed this with 20 000 random pairs. Other cases to mention: -0.0 (`0x80000000`) and $+0.0$ are equal as floats but differ as integers, and NaNs have no ordering. Fix for all cases: if the sign bit is set, flip all bits; otherwise flip only the sign bit; then compare as unsigned.

18.5 F5: Floating-point addition

Past Exam Question (2013–14 Q3(a), 10; 2012–13 undated Q5(a), 8+2; 2015–16 Q6(a), 12; 2018–19 Q8(b), 10; 2022–23 CSE 209 Q1(a), 10)

Write down the steps of floating-point addition (with an example / a flowchart). State the conditions of overflow and underflow (and NaN, infinity).

Solution. **Steps** (Figure 17.1): (1) compare the exponents and shift the significand of the smaller number right by the difference, so that the exponents agree; (2) add the significands (subtract if the signs differ); (3) normalize the sum, adjusting the exponent, and check for overflow/underflow; (4) round, and renormalize if necessary. **Example:** $1.000_2 \times 2^{-1} + (-1.110_2 \times 2^{-2})$: align to $1.000 \times 2^{-1} - 0.111 \times 2^{-1}$; add $= 0.001 \times 2^{-1}$; normalize $= 1.000 \times 2^{-4}$; round: unchanged; $= 0.0625$ (Chapter 17).
Conditions:

Event	Condition (single precision)
Overflow	after normalization/rounding the exponent is larger than the maximum (actual exponent > 127, stored > 254): result $\pm\infty$ (or exception)
Underflow	exponent smaller than the minimum (actual exponent < -126, stored < 1): result denormal or 0 (or exception)
Infinity as result	overflow; or an operand is $\pm\infty$ and the other finite
NaN as result	an operand is NaN, or $(+\infty) + (-\infty)$

Past Exam Question (2016–17 Q7(b), 15 marks)

Using the given FP adder hardware, simulate the addition of $-(0.25)_{10}$ and $(0.625)_{10}$ by filling in the steps.

Solution. (The hardware figure and table in the scan are not legible; I fill the steps of the slide-22 hardware, with single-precision numbers.)
$-0.25 = -1.0_2 \times 2^{-2}$ (stored exponent 125); $0.625 = 1.01_2 \times 2^{-1}$ (stored exponent 126).

Step	Work
1. Small ALU, shift	exponent difference $126 - 125 = 1$; the smaller exponent belongs to -0.25, so its significand 1.000 is shifted right 1 place: -0.100×2^{-1}; larger exponent $= -1$
2. Big ALU	signs differ: $1.010 - 0.100 = 0.110$ (positive, as 0.625 has the larger magnitude): $0.110_2 \times 2^{-1}$
3. Normalize	shift left 1 place: $1.10_2 \times 2^{-2}$; exponent field 125: within 1..254, no overflow or underflow
4. Round	no bits were lost: unchanged

Result $= 1.1_2 \times 2^{-2} = 0.375$: `0 01111101 1000...0` $=$ `0x3EC00000`, equal to $0.625 - 0.25$.

Past Exam Question (2017–18 Q5(b), 15 marks)

Add 0.75_{10} and 1.25_{10} in a 16-bit ("half-precision") floating-point format with a 4-bit exponent, step by step.

Solution. Format: 1 sign, 4 exponent, 11 fraction bits; bias 7; normalized exponent fields $1, \ldots, 14$.
$0.75 = 1.1_2 \times 2^{-1}$: `0 0110 10000000000`. $1.25 = 1.01_2 \times 2^0$: `0 0111 01000000000`.
1. Align: exponents -1 and 0; shift 1.1 right 1 place: $0.11_2 \times 2^0$.
2. Add: $1.01 + 0.11 = 10.00_2$, so $10.00_2 \times 2^0$.
3. Normalize: shift right 1, exponent $0 + 1 = 1$: $1.000_2 \times 2^1$; stored exponent $1 + 7 = 8$; $1 \leq 8 \leq 14$: no overflow or underflow.
4. Round: nothing to round (all fraction bits 0).
Result: `0 1000 00000000000` $=$ `0x4000` $= 2.0 = 0.75 + 1.25$.

Past Exam Question (2020–21 Q12(c), 9 marks)

Explain guard, round and sticky bits.

Solution. (Not in the lecture slides; standard IEEE 754 explanation.) When the smaller operand is shifted right to align the exponents, bits fall off the end of the significand register, and the result of the addition may be wrong in the last place. Hardware keeps up to three extra bits: the **guard** bit and the **round** bit are the first two bits shifted out below the least significant bit (they allow the result to be normalized by one left shift after subtraction and still have a correct rounding bit); the **sticky** bit is the OR of all the remaining bits shifted out. Together they say

whether the discarded part is $< \frac{1}{2}$, exactly $\frac{1}{2}$ or $> \frac{1}{2}$ of a unit in the last place, which is what round-to-nearest-even needs. Example: $1.000_2 + 1.0000001_2 \times 2^{-4}$ has guard 1, round 0, sticky 1: the remainder is more than half, so the result is 1.001_2; without the sticky bit it would look like an exact tie and round (wrongly) to 1.000_2. See Chapter 17.

18.6 F6: MIPS code with floating-point instructions (outside the lecture slides)

Common mistake

The three questions below use MIPS floating-point instructions (`lwc1`, `add.s`, `mul.d`, `c.lt.s`, `bc1t`, ...). These instructions are *not* covered by the supplied slides (MIPS Chapter 2 and Arithmetic slides 9–22 contain none); the answers use the standard MIPS floating-point instruction set from the textbook (Patterson and Hennessy, §3.5). Facts used: single-precision registers `$f0`-`$f31`; a double occupies an even–odd register pair named by the even register; `c.lt.s fs,ft` sets the FP condition flag if fs < ft; `bc1t`/`bc1f` branch if the flag is true/false. I did not run these programs in my simulator (it has no FP support); I checked them by hand.

Past Exam Question (2019–20 Q4(c), 8 marks)

`if (a>b) c = c - (c*e)/d; else c = c + d;` where `a, b, c, d, e` are floats in `$f2, $f4, $f6, $f8, $f10` (the scan prints "`$6`" for c; I read `$f6`). Fewest instructions.

Solution.

```
     c.lt.s $f4, $f2 # flag = (b < a) i.e. a > b
     bc1f Else # if not (a > b) go to Else
     mul.s $f0, $f6, $f10 # c * e (temporary in $f0)
     div.s $f0, $f0, $f8 # (c*e)/d
     sub.s $f6, $f6, $f0 # c = c - (c*e)/d
     j Exit
Else: add.s $f6, $f6, $f8 # c = c + d
Exit:
```

7 instructions.

Past Exam Question (2020–21 Q13(c), 7 marks)

`float a,b,c; double w,x,y,z; if (a+b>c) x = (y*z)/w;` Efficient MIPS code, assuming suitable registers.

Solution. Assume a, b, c in `$f2,$f4,$f6`; doubles w, x, y, z in the register pairs `$f8, $f10, $f12, $f14` (each even register names a pair).

```
     add.s $f0, $f2, $f4 # a + b
     c.lt.s $f6, $f0 # flag = (c < a+b)
     bc1f Exit # skip if not (a+b > c)
     mul.d $f16, $f12, $f14 # y * z
     div.d $f10, $f16, $f8 # x = (y*z) / w
Exit:
```

5 instructions.

Past Exam Question (2017–18 Q8(b), 10 marks)

```
float convert(float cels) { float ferh = (9.0*cels)/5.0; ferh = ferh +
32.0; return ferh; }
```

Solution. The argument `cels` arrives in `$f12`; the result is returned in `$f0`. The constants 9.0, 5.0 and 32.0 cannot be immediates, so they are kept in static memory at labels `const9`, `const5`, `const32`, addressed from `$gp`.

```
convert: lwc1 $f16, const9($gp) # 9.0
       mul.s $f16, $f16, $f12 # 9.0 * cels
       lwc1 $f18, const5($gp) # 5.0
       div.s $f16, $f16, $f18 # (9.0*cels)/5.0
       lwc1 $f18, const32($gp) # 32.0
       add.s $f0, $f16, $f18 # + 32.0 -> return value
       jr $ra
```

CHAPTER A

Topics Left Out, Assumptions and Sources

Past-paper topics not solved here

The supplied papers (CSE 209 2022–23, 2023–24; CSE 305 2012–13 to 2022–23) also contain questions on topics that are outside the three parts of this book, as you requested. They are *not* solved here:

- **Chapter 1:** performance metrics (CPI, clock rate, Amdahl's law, SPEC ratio), Moore's law, the "great ideas" questions.

- **Single-cycle and multicycle datapath design:** drawing the datapath for `add`/`lw`/`sw`/`beq`, extending it with `bne`, `j`, `jr`, `jal`, `lui`, `ori`, microprogramming, hard-wired versus microprogrammed control (the pipelined-datapath questions are solved in Chapter 15).

- **Computer arithmetic other than floating point:** integer multiplication and division hardware, signed multiplication, ALU and overflow-detector design, status-generator modules.

- **Exceptions and interrupts:** EPC/Cause registers, precise exceptions, DMA, I/O, RAID, buses.

- **Memory hierarchy and multiprocessors:** caches, virtual memory, TLB, coherence, UMA/NUMA, interconnects, multithreading — these belong to the first book (memory hierarchy and GPUs) or to other chapters.

Assumptions made where a scan was unreadable or ambiguous

Question	Assumption or limitation
2014–15 Q7(c), 2018–19 Q8(c)	`beq $s0,$s1,30500` at 005000: read both as printed; the offset is out of the 16-bit range under the hex reading (44351 words), and fits under the decimal reading; both are discussed
2016–17 Q6(a)	bne offset taken as 1
2022–23 CSE 305 Q5(c)	X and Y assumed at offset 0 of their data segments
2017–18 Q7(b)	target `0x40000001` is not word-aligned; the answer says so
2018–19 Q8(a)	"return 0" at $b = 0$ is taken as printed (the function then always returns 0); the corrected "return 1" version is also given
2019–20 Q4(a)(ii)	the number is read as $110.11_2 \times 2^1$; the alternative reading $10.11_2 \times 2^1$ is also given
2019–20 Q4(c)	register `$6` for c read as `$f6`
2019–20 Q8(c)	figure not legible; the standard `sub/and/or` sequence of slide 53 is assumed
2016–17 Q7(b)	hardware figure not legible; the steps of the slide-22 adder are filled in with single-precision numbers
undated paper Q4(d),(e)	Q4(d) is cut off in the scan and is not answered; for Q4(e) the starting state in the figure is not visible, so both possible start states are given
2013–14 Q7(d)	stage times taken to be the same as in 2015–16 Q2(b)
2022–23 CSE 209 Q7, 2021–22 Q8(b)	the question text was paraphrased from a scan; the reasoning is stated independently of the exact wording
2022–23 CSE 305 Q4(b)	standard answer of slide 78 given, since the figure is not in the scan

Lecture misprints corrected in the text

- MIPS Chapter 2: the `sll` shamt example (see the correction box in Chapter 5).
- Processor slide 97: `addu $t3,$t4,$s2` should be `addu $t3,$t3,$s2` (Chapter 13).
- Arithmetic slide 12: the largest single-precision number is written "2.0×2^{127}"; the exact value is $(2 - 2^{-23}) \times 2^{127}$ (Chapter 16).

Sources

Primary sources (syllabus boundary).

- Lecture slides: *Chapter 2 — Instructions: Language of the Computer* (69 slides); *Chapter 4 Part 2 — The Processor* (111 slides; Part 1 and Chapter 1 were attached but used only for prerequisites); *Chapter 3 — Arithmetic for Computers*, slides 9–22. The slides are based on Patterson and Hennessy, *Computer Organization and Design* (5th edition, "The Hardware/Software Interface").
- Past papers: CSE 305 (2012–13 to 2022–23) and CSE 209 (2022–23, 2023–24) as supplied.

Background textbook (not reproduced). D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann — the standard treatment of MIPS, pipelining and IEEE 754 on which the slides are based; used as general background for the clearly marked "beyond the slides" explanations (MIPS floating-point instructions, guard/round/sticky bits, CISC/RISC comparison).

External source consulted

[1] D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, March 1991 (edited reprint as Appendix D of Oracle's *Numerical Computation Guide*). `https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html` — consulted for: single/double bias 127 and 1023; $e_{\max}$ and $e_{\min}$; infinity and NaN encoded with exponent $e_{\max} + 1$; guard digits and the sticky bit ("the result is the same as if the difference were computed exactly and then rounded").